Devin AI Export - PDF Version
Generated on: 11/4/2025, 10:05:42 PM
Source: Devin AI Platform

Overview

Relevant source files

- [

  README.md](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md))

- [

  package.json](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/package.json (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/package.json))

Purpose and Scope

The TradeX Configuration Service is a Node.js microservice that serves as the central configuration authority for the TradeX trading platform ecosystem. This service manages client authentication, role-based permissions, multi-language localization, system settings, and administrative functions across the entire platform.

This document provides a high-level overview of the service's architecture, core capabilities, and integration patterns. For detailed API specifications, see API Reference. For database schema information, see Database Schema. For deployment procedures, see Deployment.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L1-L8)

README.md1-8 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L1-L8)[

package.json2-4](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/package.json#L2-L4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/package.json#L2-L4))

Core Capabilities

The Configuration Service provides six primary functional domains:

Domain	Purpose	Key Entities
Client Management	OAuth client registration and authentication	`Client`, `LoginMethod`
Access Control	Role-based permissions and scope management	`Scope`, `ScopeGroup`
Localization	Multi-language resource management	`LangResource`
Content Management	FAQ and help content administration	`FAQ`, `FAQGroup`
System Configuration	Holiday calendars and interest rates	`Holiday`, `Service`
Navigation	Dynamic menu system with role-based access	`Menu`, `MenuGroup`

Technical Architecture Features

- **Event-Driven Processing**: Kafka consumers handle asynchronous configuration synchronization
- **TypeORM Integration**: Database abstraction with entity management and migrations
- **Multi-Storage Support**: AWS S3 and MinIO for file storage operations
- **Dependency Injection**: TypeDI container for service management
- **API Management**: OpenAPI specification handling and Swagger integration

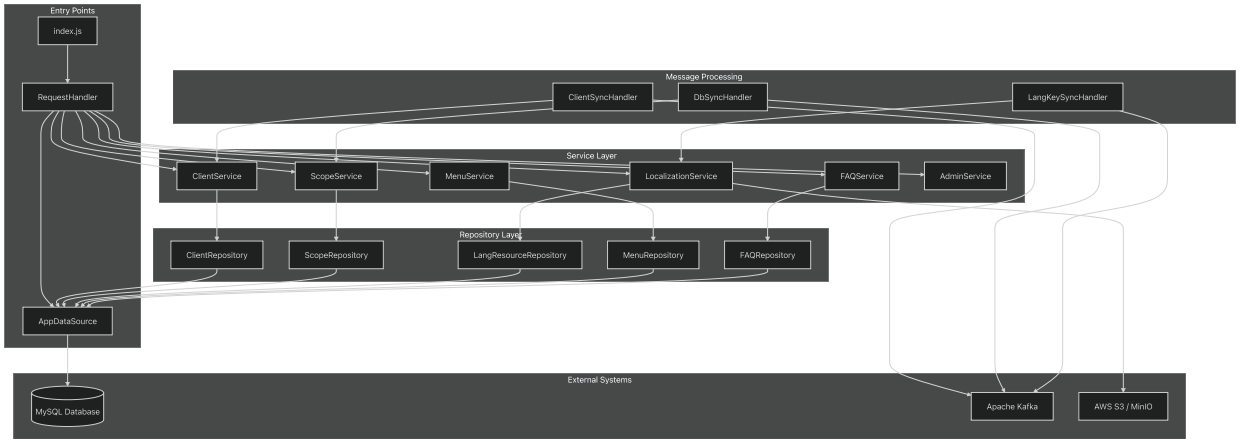
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L9-L26)

README.md9-26 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L9-L26)[

README.md158-169](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L158-L169 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L158-L169))

System Architecture

Core Service Structure



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L29-L43)

README.md29-43 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L29-L43)[

README.md172-187](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L172-L187 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L172-L187))

Technology Stack

The service is built on a modern Node.js stack with enterprise-grade dependencies:

Category	Technology	Version	Purpose
Runtime	Node.js	16.x	JavaScript execution environment
Language	TypeScript	5.3.3	Type-safe development
ORM	TypeORM	0.3.19	Database abstraction and migrations
Messaging	Apache Kafka	-	Event-driven communication
Storage	AWS S3 / MinIO	-	File storage and management
Validation	AJV	8.12.0	JSON schema validation
DI Container	TypeDI	0.10.0	Dependency injection

TradeX Platform Dependencies

The service integrates with several TradeX-specific libraries:

- **tradex-common**: Core utilities and framework components

- **tradex-models-ts**: Shared data model definitions
- **tradex-models-configuration**: Configuration-specific models
- **tradex-models-configuration-validator**: Validation schemas

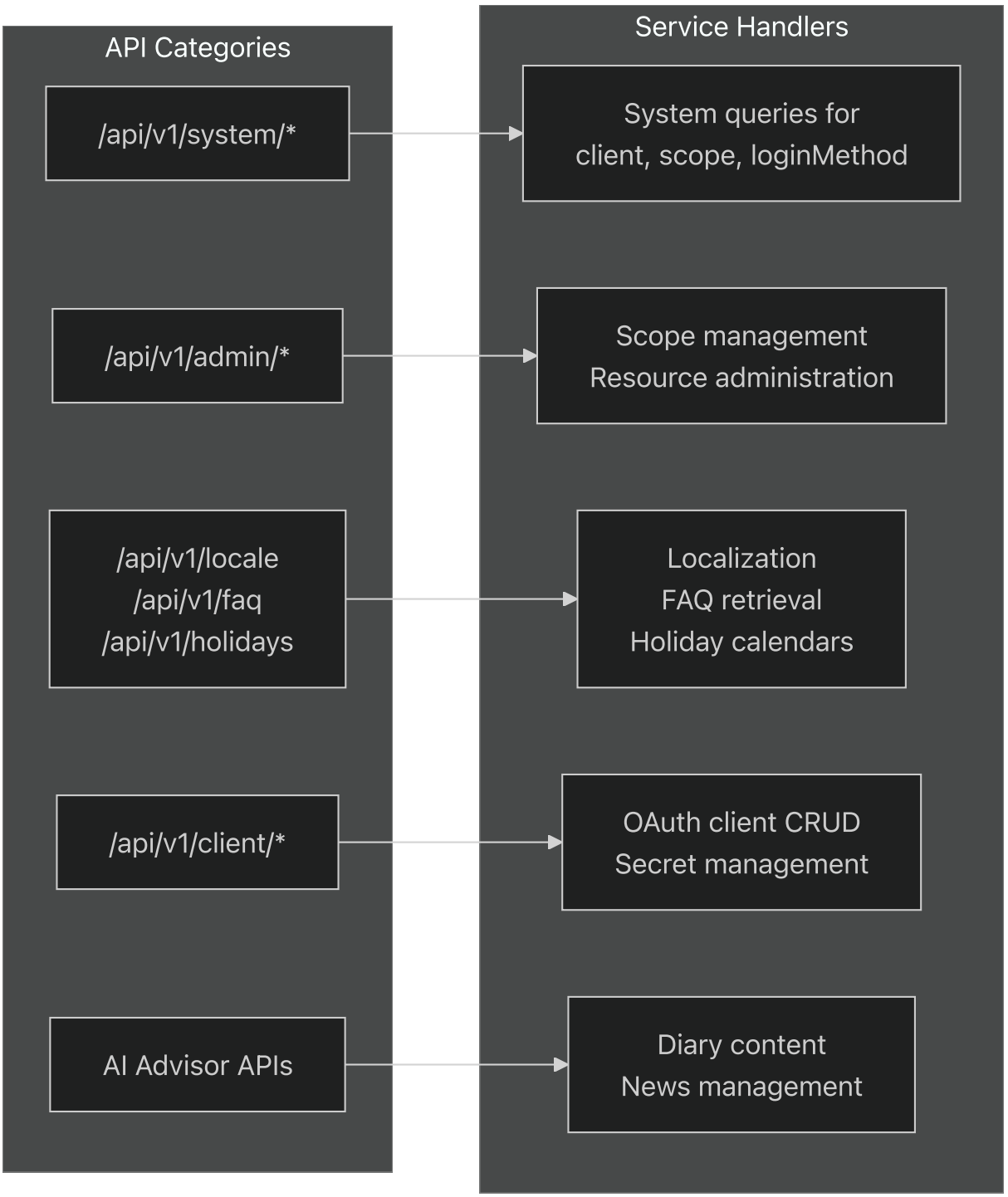
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L45-L56)

README.md45-56 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L45-L56)[

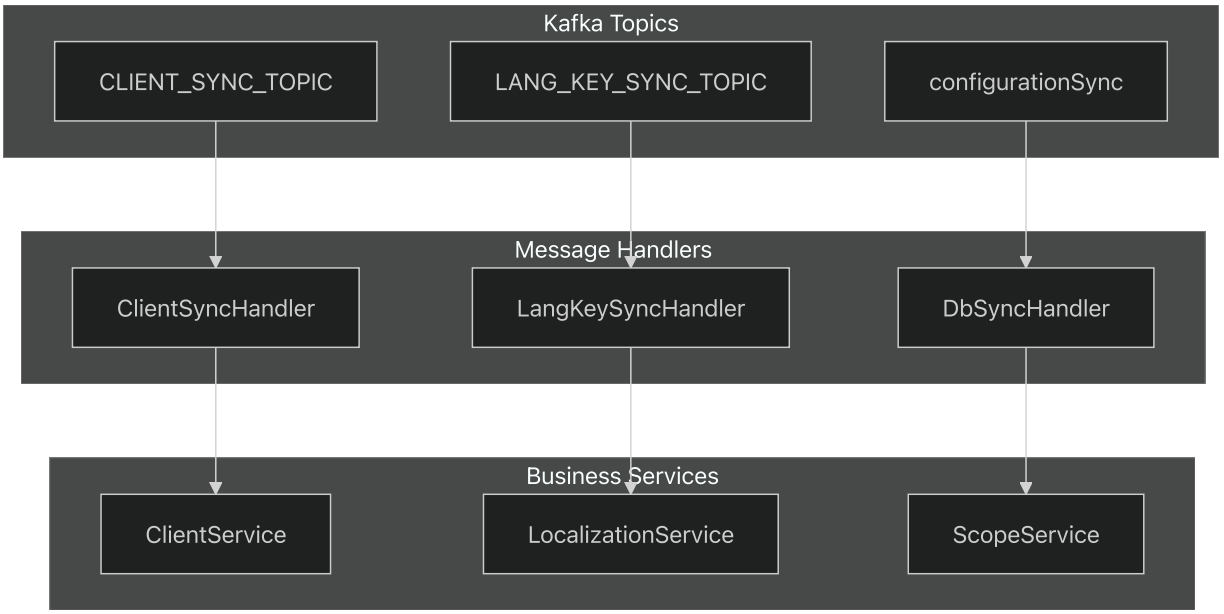
package.json19-34](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/package.json#L19-L34 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/package.json#L19-L34))

Integration Patterns

API Endpoint Categories



Kafka Event Processing



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L129-L157)

README.md129-157 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L129-L157)[

README.md21-22](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L21-L22 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L21-L22))

Project Structure

The codebase follows a layered architecture pattern with clear separation of concerns:

```
src/
├── consumers/          # Kafka message handlers (ClientSyncHandler, etc.)
├── models/            # TypeScript models and DTOs
│   ├── db/           # Database entities (Client, Scope, LangResource)
│   ├── request/      # API request schemas
│   └── response/     # API response schemas
├── repositories/     # Data access layer (ClientRepository, etc.)
├── services/        # Business logic (ClientService, ScopeService)
│   └── admin/       # Administrative service implementations
└── utils/           # Utility functions and helpers
```

Database Migration Management

The service uses Liquibase for database schema management:

- **dbmigration/dbchangelog.xml**: Master migration orchestrator
- **Migration Scripts**: Versioned schema changes and data initialization
- **Entity Synchronization**: TypeORM entities reflect current schema state

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L172-L187)

README.md172-187 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L172-L187)[

README.md158-169](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L158-L169 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L158-L169))

Deployment and Operations

Container Architecture

The service is containerized using a multi-stage Docker build optimized for production:

1. **Base Stage**: Node.js Alpine runtime with system dependencies
2. **Builder Stage**: TypeScript compilation and dependency installation
3. **Production Stage**: Minimal runtime image with compiled artifacts

Health Monitoring

- **Health Endpoint**: HTTP health checks on port 3000
- **Service Registration**: Kafka-based service discovery
- **Logging**: Structured logging with file rotation

Environment Configuration

Configuration is managed through multiple layers:

- **Base Configuration**:[

  src/config.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts)) with sensible defaults

- **Environment Overrides**: External `env.js` files for deployment-specific settings

- **Runtime Parameters**: Docker environment variables and startup scripts

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L205-L225)

README.md205-225 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L205-L225)[

README.md98-109](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L98-L109 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L98-L109))

Architecture

Overview

Relevant source files

- [

  README.md](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md))

- [

  src/AppDataSource.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts))

- [

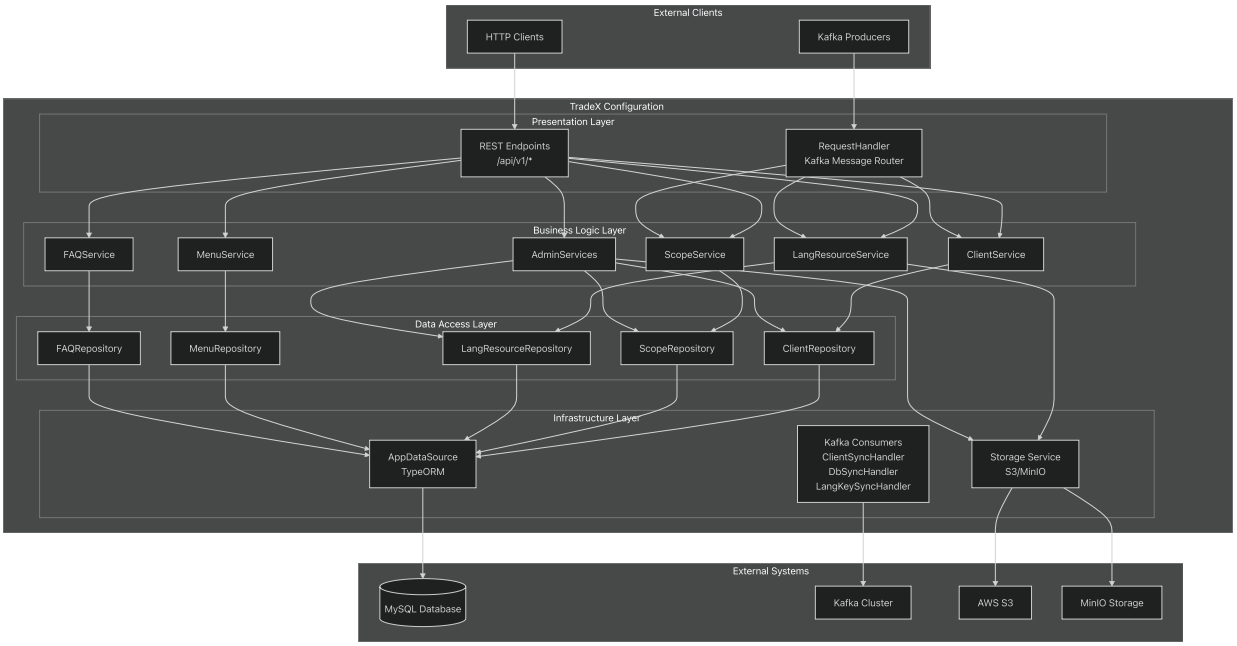
  src/config.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts))

This document provides a detailed overview of the TradeX Configuration Service system architecture, including service layers, data flow patterns, and integration mechanisms. It covers the technical structure of how components interact within the service and with external systems.

For information about specific database schemas and entity relationships, see Database Schema. For detailed API endpoint documentation, see API Reference. For configuration specifics, see Configuration.

Service Layer Architecture

The TradeX Configuration Service follows a layered architecture pattern with clear separation of concerns across multiple tiers.



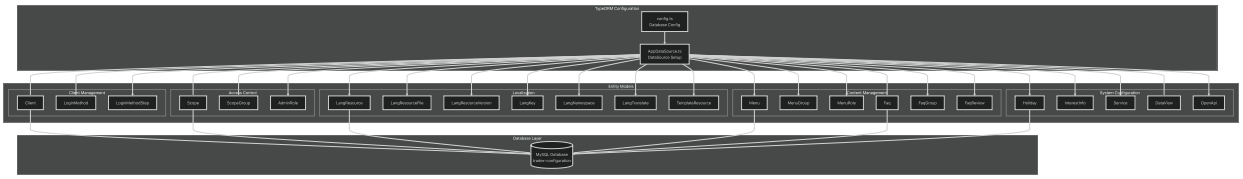
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L29-L44)

README.md29-44 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L29-L44)[

README.md173-187](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L173-L187 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L173-L187))

Data Persistence Architecture

The service uses TypeORM as the Object-Relational Mapping layer to manage database interactions with MySQL. The `AppDataSource` serves as the central database connection manager.



TypeORM Configuration Details

The `AppDataSource` configuration establishes the connection parameters and entity registration:

Configuration	Value	Purpose
`type`	"mysql"	Database type specification
`synchronize`	false	Prevents automatic schema updates
`logging`	true	Enables SQL query logging
`timezone`	"Z"	UTC timezone configuration

The entities array in `AppDataSource` registers 21 database entities covering all functional areas of the configuration service.

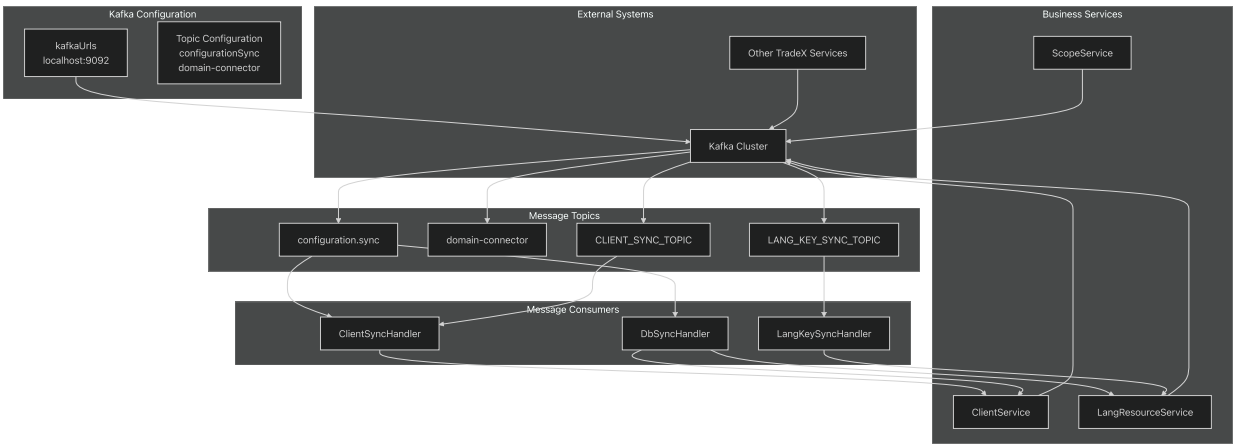
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L1-L68)

src/AppDataSource.ts1-68 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L1-L68)[

src/config.ts14-23](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L14-L23 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L14-L23))

Message Processing Architecture

The service implements an event-driven architecture using Apache Kafka for asynchronous message processing and inter-service communication.



Kafka Integration Configuration

The service includes comprehensive Kafka configuration with multiple option layers:

Configuration Layer	Purpose
`kafkaCommonOptions`	Shared settings across producers and consumers
`kafkaConsumerOptions`	Consumer-specific configurations
`kafkaProducerOptions`	Producer-specific configurations
`kafkaTopicOptions`	Topic-level configurations

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L50-L56)

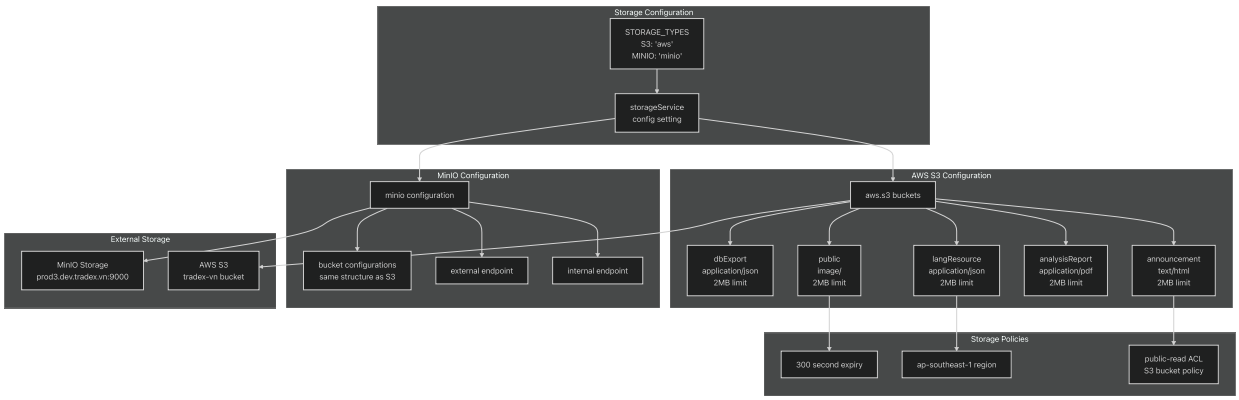
src/config.ts50-56 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L50-L56)[

src/config.ts233-237](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L233-L237)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L233-L237)%5B)

src/config.ts300-308](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L300-L308 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L300-L308))

Storage Architecture

The service supports dual storage backends for file management: AWS S3 and MinIO, configurable through the `storageService` setting.



Storage Bucket Configuration

Each storage bucket has specific configurations for content types and size limits:

Bucket Type	Content Type	Max Size	ACL
`announcement`	text/html	2MB	public-read
`analysisReport`	application/pdf	2MB	public-read
`langResource`	application/json	2MB	public-read
`public`	image/	2MB	public-read
`dbExport`	application/json	2MB	public-read

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L7-L10)

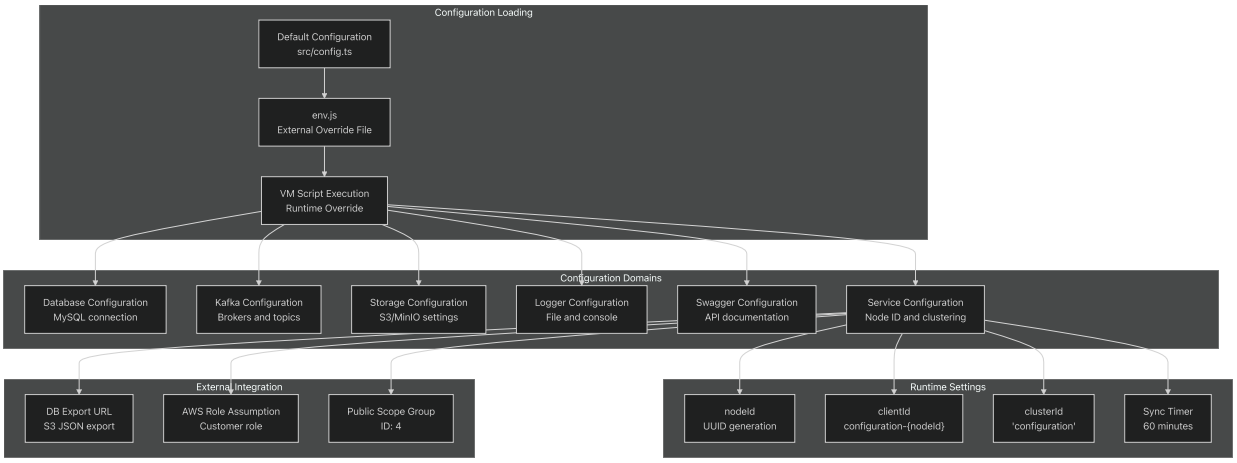
src/config.ts7-10 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L7-L10)[

src/config.ts68-123](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L68-L123)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L68-L123)%5B)

src/config.ts124-223](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L124-L223 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L124-L223))

Configuration Management Architecture

The service implements a sophisticated configuration system that supports environment-specific overrides and runtime configuration loading.



Configuration Override Mechanism

The configuration system uses Node.js VM module to safely execute external configuration files:

1. **Default Configuration**: Defined in `src/config.ts` with production defaults
2. **External Override**: `env.js` file execution in sandboxed VM context
3. **Runtime Merging**: Kafka options merged with common configurations
4. **Logger Initialization**: Conditional logger setup based on configuration success

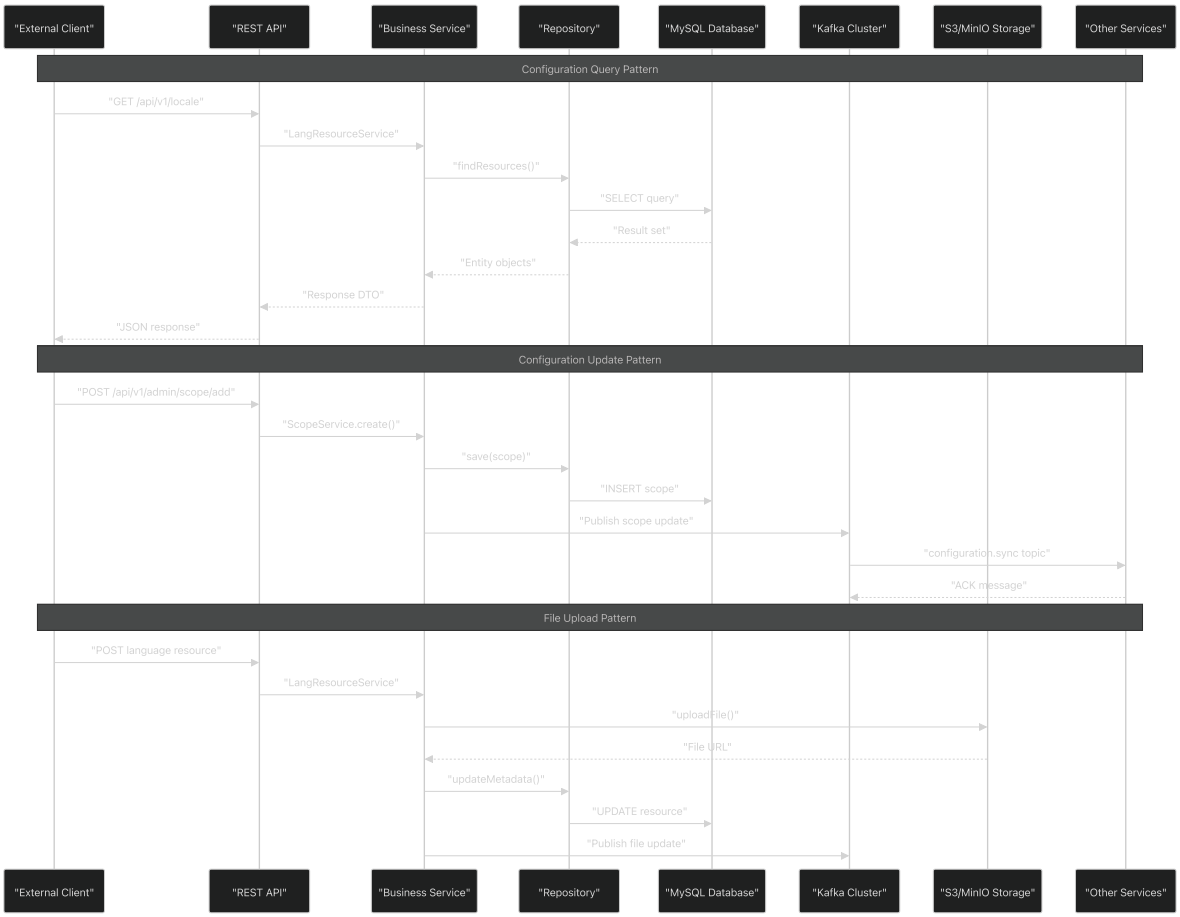
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L275-L309)

src/config.ts275-309 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L275-L309)[

src/config.ts11-273](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L11-L273 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L11-L273))

Integration Patterns

The service demonstrates several key integration patterns for microservice communication and data synchronization.



Key Integration Mechanisms

Pattern	Implementation	Purpose
Synchronous API	REST endpoints with TypeORM	Direct configuration queries
Asynchronous Messaging	Kafka topics with event handlers	Cross-service synchronization
File Storage Integration	S3/MinIO with metadata tracking	Resource file management
Database Transactions	TypeORM with MySQL	Data consistency
Configuration Synchronization	Timer-based sync with JSON export	Distributed configuration updates

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L130-L157)

README.md130-157 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L130-L157)[

src/config.ts53-67](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L53-L67 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L53-L67))

Database Layer

Relevant source files

- [

  src/AppDataSource.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts))

- [

  src/config.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts))

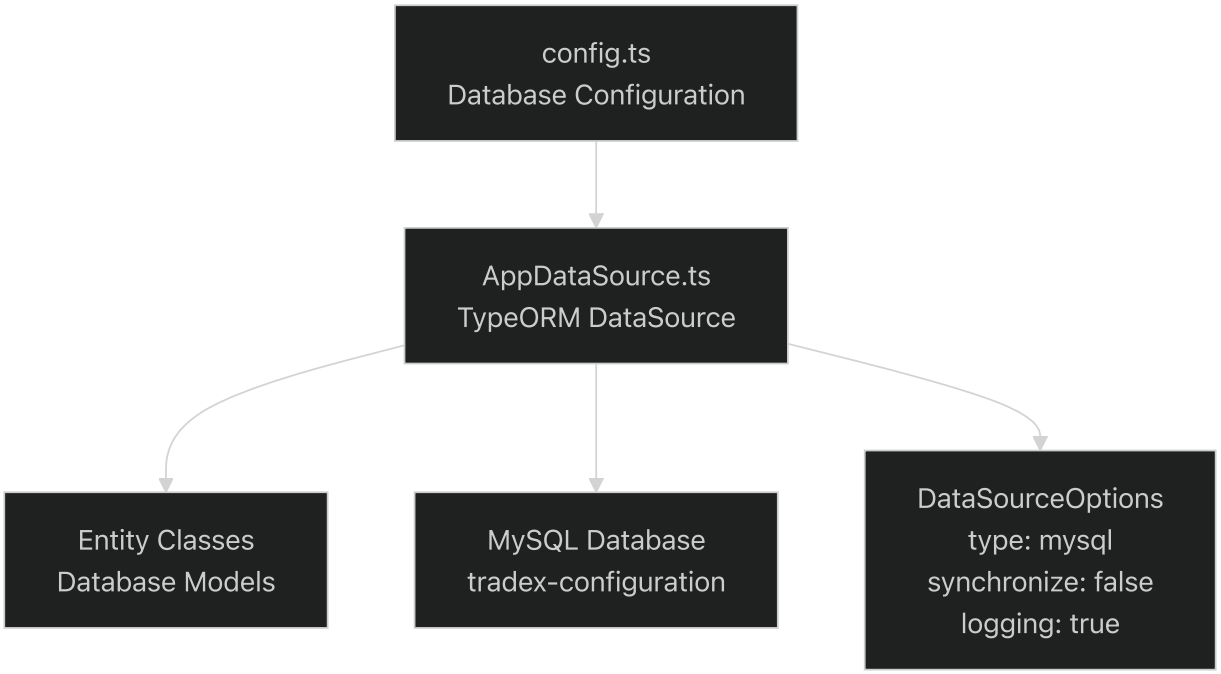
Purpose and Scope

The Database Layer provides TypeORM-based data access and entity management for the TradeX Configuration Service. This layer handles MySQL database connections, entity mapping, and data persistence operations for all configuration data including client management, localization resources, access control, and system settings.

For information about database schema structure and migrations, see Database Schema. For details about specific entity relationships and table designs, see Client Management Schema, API Registry Schema, and Access Control Schema.

TypeORM Configuration

The database layer is built on TypeORM and configured through the `AppDataSource` class, which serves as the central data source for all database operations.



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L29-L65)

src/AppDataSource.ts29-65 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L29-L65)[

src/config.ts14-23](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L14-L23 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L14-L23))

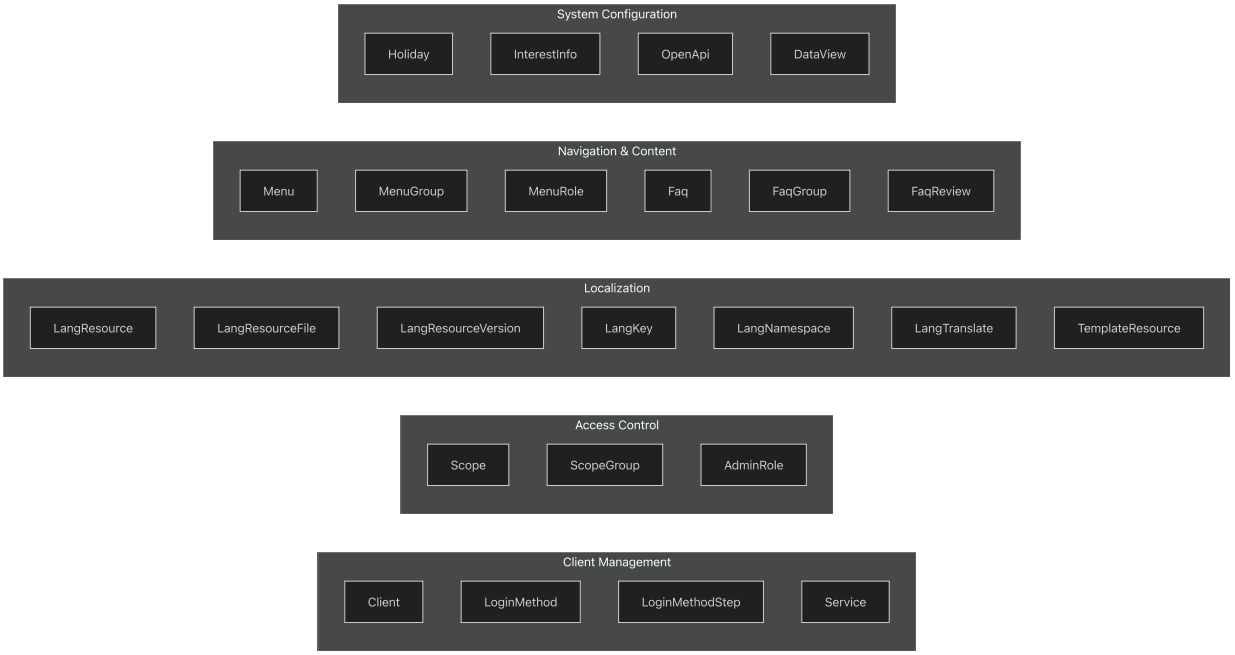
The `DataSourceOptions` configuration in (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L29-L65)

src/AppDataSource.ts29-65 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L29-L65) specifies key TypeORM settings:

Setting	Value	Purpose
`type`	`"mysql"`	Database engine type
`synchronize`	`false`	Disables automatic schema sync, relies on migrations
`logging`	`true`	Enables SQL query logging for debugging
`timezone`	`"Z"`	Uses UTC timezone for consistent date handling

Entity Management

The database layer manages 23 entity classes representing different functional areas of the configuration service. These entities are registered in the `AppDataSource.entities` array and provide TypeORM mapping to database tables.



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L4-L27)

src/AppDataSource.ts4-27 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L4-L27)[

src/AppDataSource.ts36-61](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L36-L61 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L36-L61))

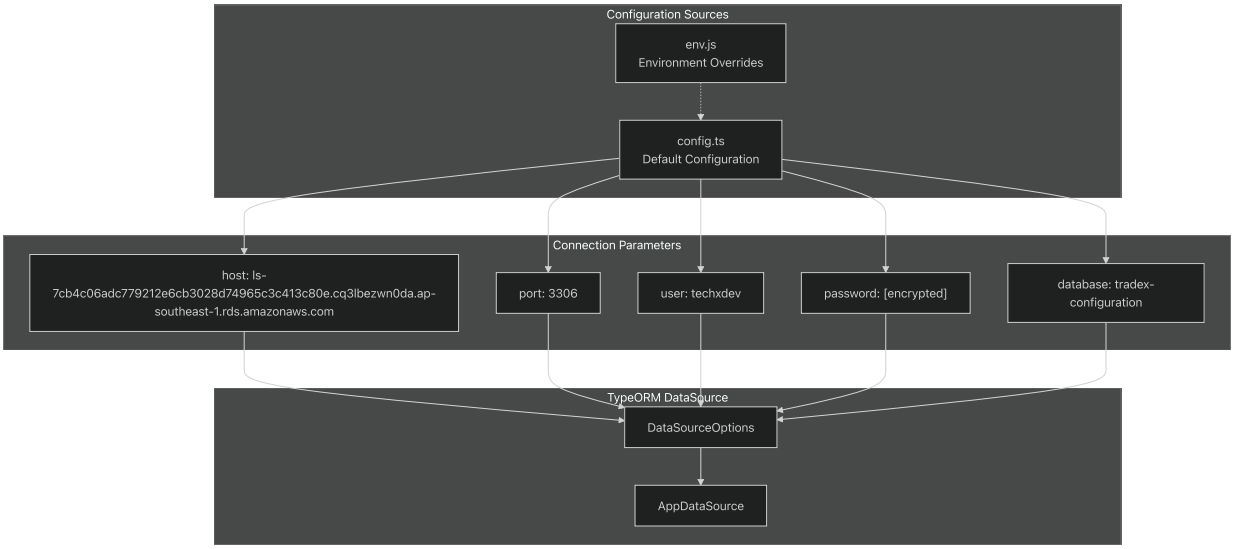
Entity Categories

The entities are organized into functional groups based on their domain responsibilities:

- **Client Management**: `Client`, `LoginMethod`, `LoginMethodStep`, `Service` - OAuth client registration and authentication methods
- **Access Control**: `Scope`, `ScopeGroup`, `AdminRole` - Permission-based access control system
- **Localization**: `LangResource`, `LangResourceFile`, `LangResourceVersion`, `LangKey`, `LangNamespace`, `LangTranslate`, `TemplateResource` - Multi-language resource management
- **Navigation & Content**: `Menu`, `MenuGroup`, `MenuRole`, `Faq`, `FaqGroup`, `FaqReview` - Dynamic menu system and FAQ management
- **System Configuration**: `Holiday`, `InterestInfo`, `OpenApi`, `DataView` - Trading calendar and API registry

Database Connection Configuration

Database connection parameters are defined in the central configuration object and referenced by the `AppDataSource` constructor.



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L14-L23)

src/config.ts14-23 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L14-L23)[

src/AppDataSource.ts31-35](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L31-L35)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L31-L35)%5B)

src/config.ts285-298](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L285-L298 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L285-L298))

The connection configuration follows this hierarchy:

1. **Default Configuration** - Base settings in[

src/config.ts16-22](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L16-L22 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L16-L22))

2. **Environment Overrides** - Runtime configuration loaded from `env.js` in[

   src/config.ts285-298](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L285-L298 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L285-L298))

3. **TypeORM Integration** - Configuration passed to `DataSourceOptions` in[

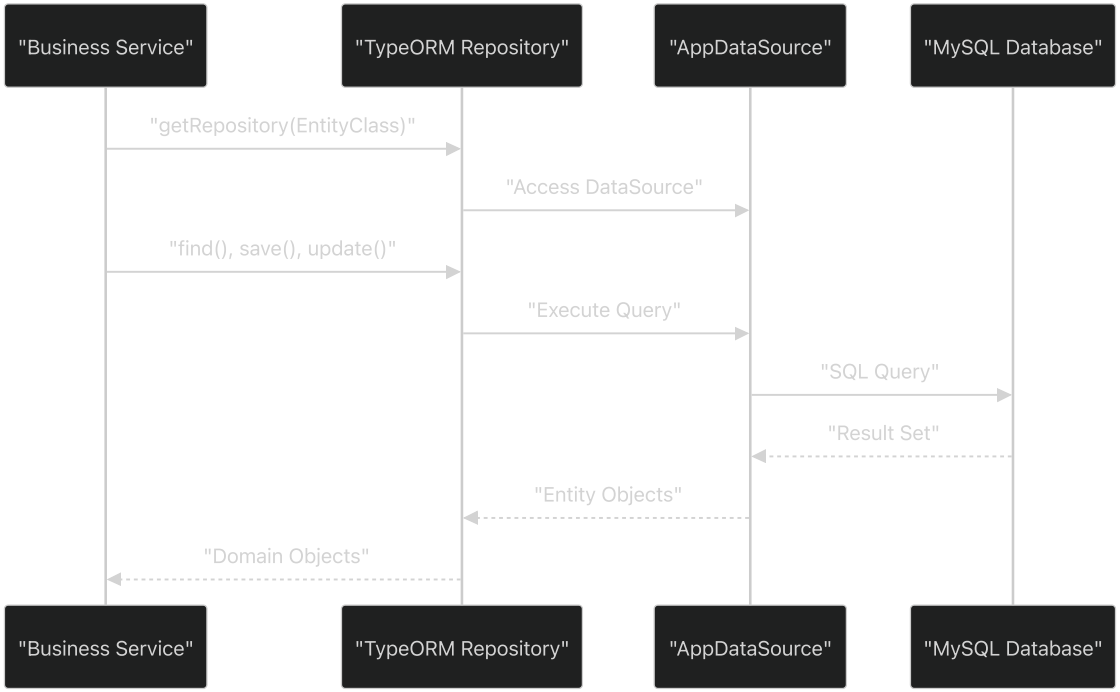
   src/AppDataSource.ts31-35](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L31-L35 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L31-L35))

Connection Properties

Property	Configuration Path	Purpose
`host`	`config.db.connection.host`	RDS MySQL endpoint
`port`	`config.db.connection.port`	Database port (3306)
`username`	`config.db.connection.user`	Database user credentials
`password`	`config.db.connection.password`	Database password
`database`	`config.db.connection.database`	Target database schema

Database Integration Patterns

The database layer follows established patterns for data access and transaction management across the service architecture.



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L67-L67)

src/AppDataSource.ts67 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L67-L67)

Key Integration Features

- **Repository Pattern**: Services access entities through TypeORM repositories obtained from `AppDataSource.getRepository()`
- **Migration-Based Schema**: Uses `synchronize: false` to rely on Liquibase migrations rather than automatic schema generation
- **Query Logging**: Enabled logging provides visibility into generated SQL queries for debugging and performance monitoring
- **UTC Timezone**: Consistent timezone handling across all timestamp operations using `timezone: "Z"`

The `AppDataSource` instance exported from (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L67-L67)

src/AppDataSource.ts67 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L67-L67) serves as the singleton data source used throughout the application for all database operations.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L29-L67)

src/AppDataSource.ts29-67 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L29-L67)[

src/config.ts14-23](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L14-L23 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L14-L23))

Messaging and Event Processing

Relevant source files

- [

  src/constants/updateTopics.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/updateTopics.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/updateTopics.ts))

- [

  src/consumers/ClientSyncHandler.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts))

- [

  src/consumers/DbSyncHandler.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts))

- [

  src/consumers/LangKeySyncHandler.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts))

Purpose and Scope

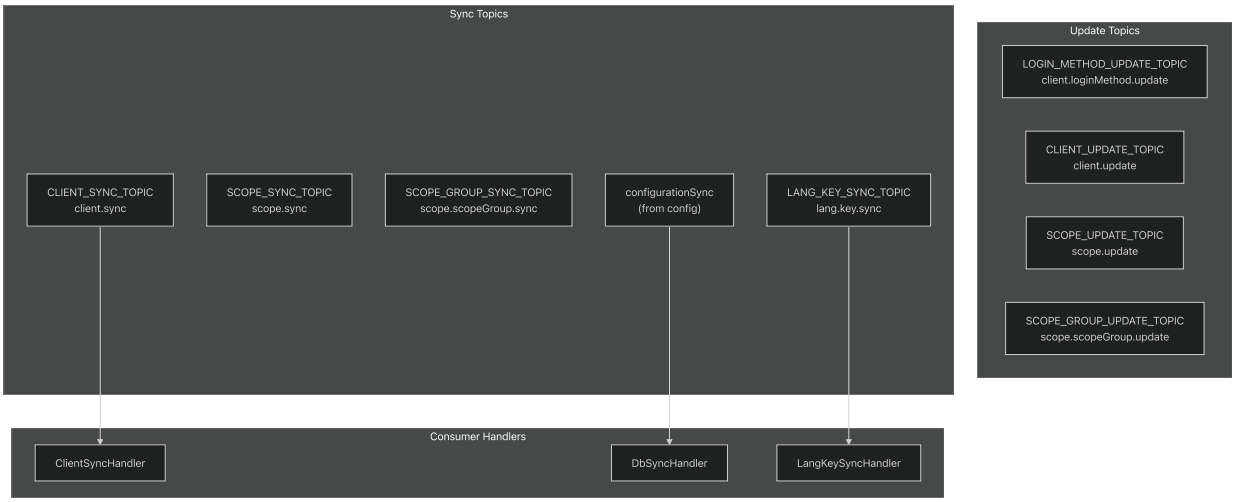
This document covers the event-driven messaging infrastructure of the TradeX Configuration Service, including Kafka topic definitions, consumer handlers, and message processing patterns. The messaging system enables asynchronous communication and data synchronization across the TradeX platform ecosystem.

For information about the overall system architecture, see Architecture. For database integration details, see Database Layer.

Messaging Architecture Overview

The TradeX Configuration Service implements an event-driven architecture using Apache Kafka for asynchronous message processing. The system consumes messages from various topics to handle configuration synchronization, client management, and localization updates across the platform.

Kafka Topic Management



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/updateTopics.ts#L1-L17)

src/constants/updateTopics.ts1-17 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/updateTopics.ts#L1-L17)

Topic Definitions

The system defines two categories of Kafka topics:

Topic Category	Purpose	Topics
Update Topics	Direct entity updates	`client.update`, `scope.update`, `scope.scopeGroup.update`, `client.loginMethod.update`
Sync Topics	Cross-service synchronization	`client.sync`, `scope.sync`, `scope.scopeGroup.sync`, `lang.key.sync`

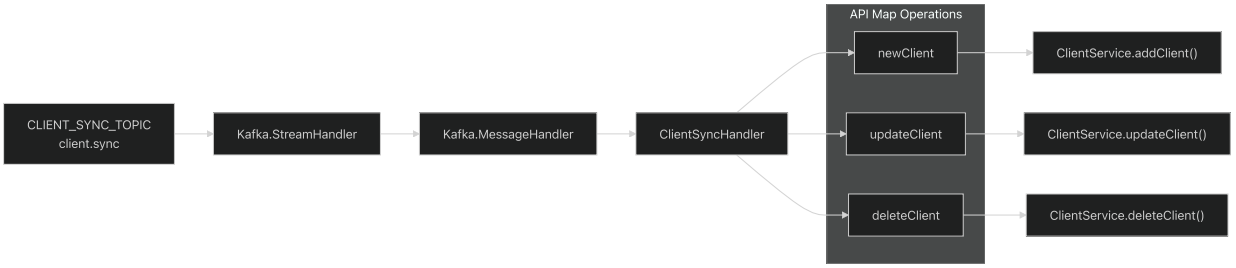
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/updateTopics.ts#L1-L17)

src/constants/updateTopics.ts1-17 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/updateTopics.ts#L1-L17)

Consumer Handlers

ClientSyncHandler

The `ClientSyncHandler` processes client-related synchronization messages on the `CLIENT_SYNC_TOPIC`. It handles client lifecycle operations through a service-oriented approach.



Key Components:

- **Topic Subscription:** `CLIENT_SYNC_TOPIC` [

  src/consumers/ClientSyncHandler.ts21](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L21-L21 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L21-L21))

- **Message Routing:** API map with operations `newClient`, `updateClient`, `deleteClient` [

  src/consumers/ClientSyncHandler.ts26-30](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L26-L30 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L26-L30))

- **Service Integration:** Delegates to `ClientService` for business logic[

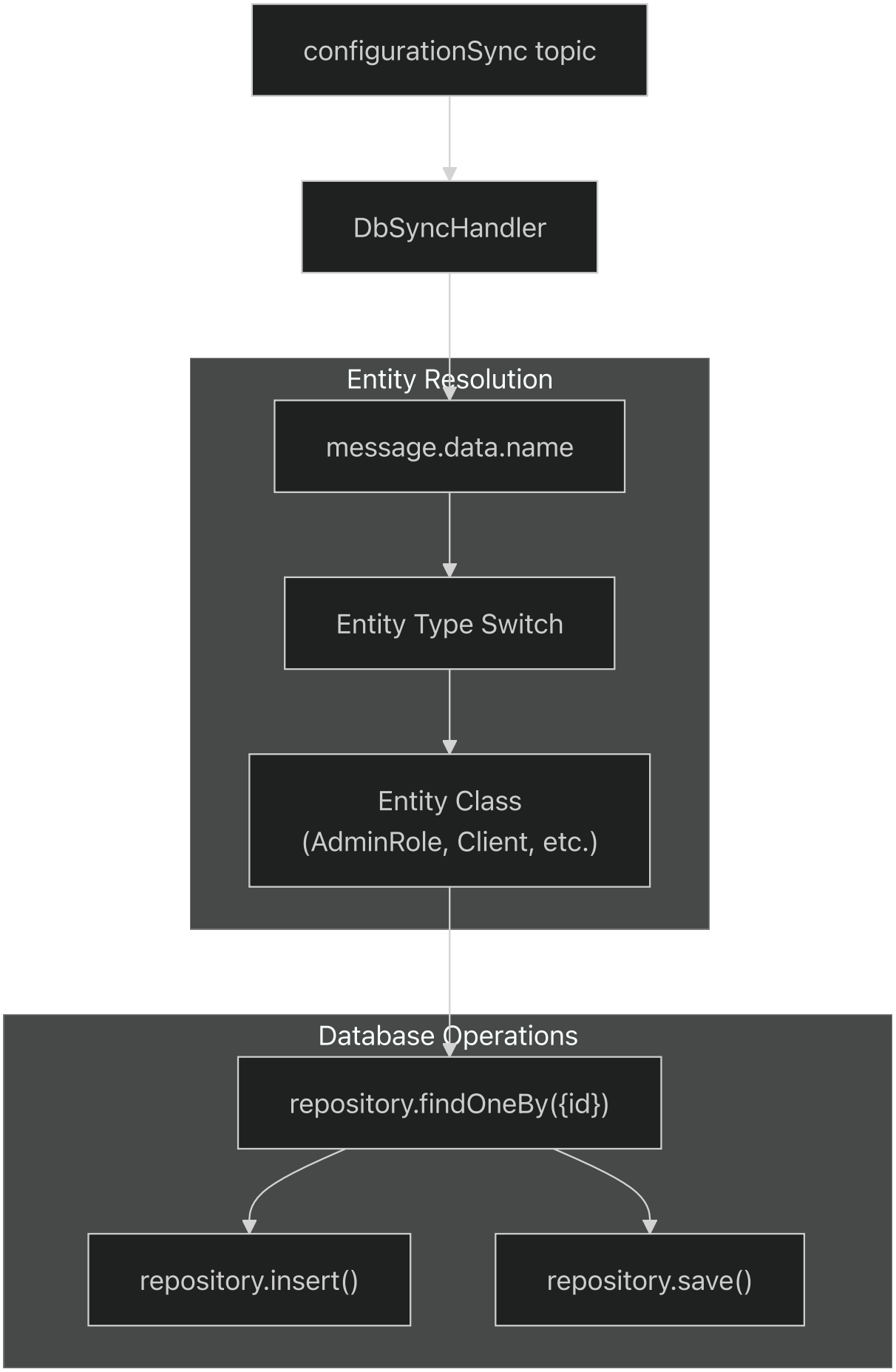
  src/consumers/ClientSyncHandler.ts27-29](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L27-L29 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L27-L29))

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L1-L46)

src/consumers/ClientSyncHandler.ts1-46 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L1-L46)

DbSyncHandler

The `DbSyncHandler` provides comprehensive database synchronization capabilities, handling messages for all entity types in the configuration database.


configurationSync topic
DbSyncHandler
Entity Resolution
message.data.name
Entity Type Switch
Entity Class
(AdminRole, Client, etc.)
Database Operations
repository.findOneBy({id})
repository.insert()
repository.save()

Supported Entities: The handler supports synchronization for 19 different entity types including `AdminRole`, `Client`, `DataView`, `Faq`, `FaqGroup`, `FaqReview`, `Holiday`, `InterestInfo`, `LangKey`, `LangNamespace`, `LangResource`, `LangResourceFile`, `LangResourceVersion`, `LangTranslate`, `LoginMethod`, `Menu`, `MenuGroup`, `MenuRole`, `OpenApi`, `Scope`, `ScopeGroup`, `Service`, and `TemplateResource`.

Upsert Logic:

1. Query existing record by ID[

   src/consumers/DbSyncHandler.ts104](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L104-L104 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L104-L104))

2. Insert if record doesn't exist[

   src/consumers/DbSyncHandler.ts107-112](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L107-L112 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L107-L112))

3. Update if record exists and differs[

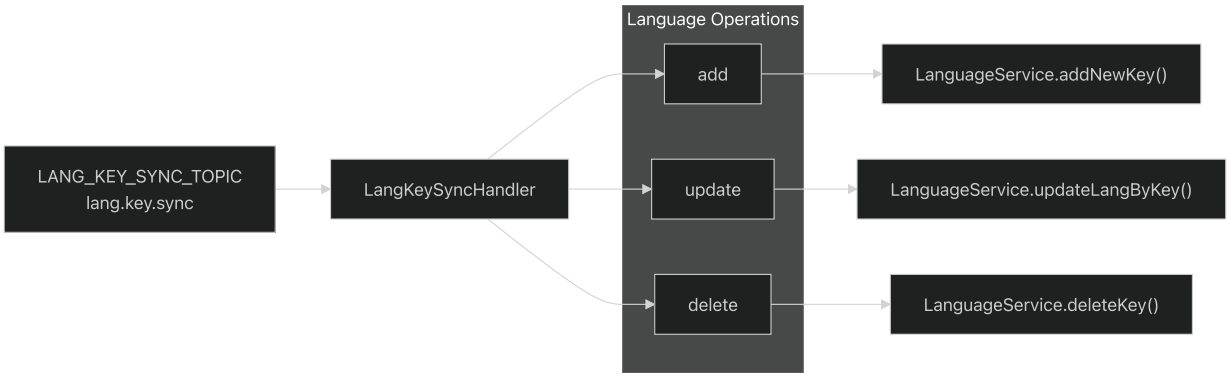
   src/consumers/DbSyncHandler.ts115-119](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L115-L119 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L115-L119))

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L1-L139)

src/consumers/DbSyncHandler.ts1-139 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L1-L139)

LangKeySyncHandler

The `LangKeySyncHandler` manages localization key synchronization for the multi-language support system.



Operations:

- **Add Key:** `LanguageService.addNewKey()` [

  src/consumers/LangKeySyncHandler.ts27](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L27-L27 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L27-L27))

- **Update Key:** `LanguageService.updateLangByKey()` [

  src/consumers/LangKeySyncHandler.ts28](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L28-L28 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L28-L28))

- **Delete Key:** `LanguageService.deleteKey()` [

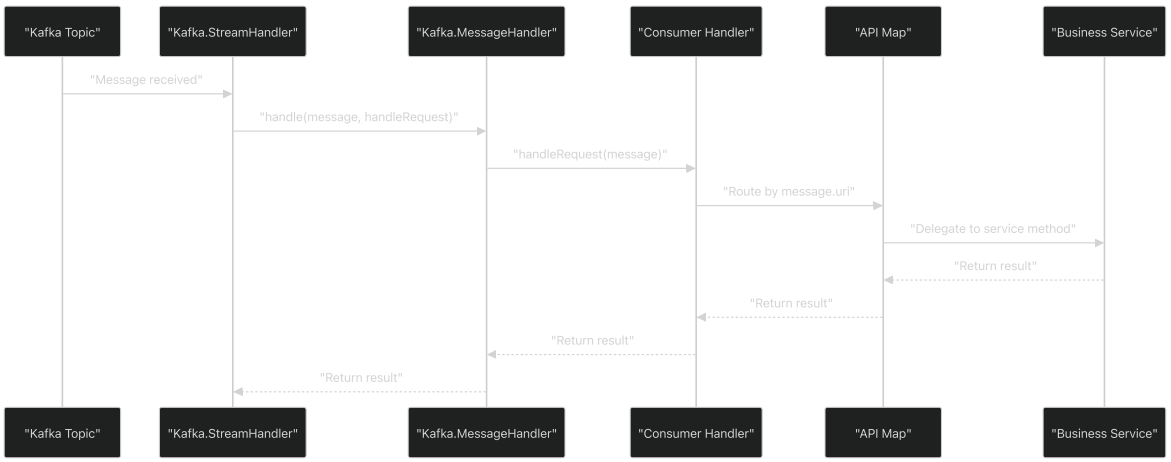
  src/consumers/LangKeySyncHandler.ts29](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L29-L29 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L29-L29))

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L1-L46)

src/consumers/LangKeySyncHandler.ts1-46 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L1-L46)

Message Processing Pattern

All consumer handlers follow a consistent message processing pattern implemented using the `tradex-common` Kafka utilities.



Common Implementation Elements

Stream Handler Initialization:

```
new Kafka.StreamHandler(
  config,
  config.kafkaConsumerOptions,
  [TOPIC_NAME],
  (message: any) => handle.handle(message, this.handleRequest),
  config.kafkaTopicOptions,
)
```

Message Router Pattern: Each handler implements an `apiMap` object that routes message URIs to corresponding service methods (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L26-L30)

src/consumers/ClientSyncHandler.ts26-30 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L26-L30)[

src/consumers/LangKeySyncHandler.ts26-30](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L26-L30 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L26-L30))

Error Handling: All handlers include validation for null messages and unknown URIs, returning `SystemError` for invalid requests (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L34-L36)

src/consumers/ClientSyncHandler.ts34-36 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L34-L36)[

src/consumers/DbSyncHandler.ts127-129](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L127-L129)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L127-L129)%5B)

src/consumers/LangKeySyncHandler.ts34-36](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L34-L36 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L34-L36))

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L16-L44)

src/consumers/ClientSyncHandler.ts16-44 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L16-L44)[

src/consumers/DbSyncHandler.ts35-137](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L35-L137)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L35-L137)%5B)

src/consumers/LangKeySyncHandler.ts16-44](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L16-L44 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L16-L44))

Integration with Business Services

The messaging layer serves as an integration point between external events and internal business logic, maintaining separation of concerns through service delegation.

Service Layer Integration

Consumer Handler	Integrated Service	Operations
`ClientSyncHandler`	`ClientService`	Client CRUD operations
`DbSyncHandler`	TypeORM Repository	Generic entity synchronization
`LangKeySyncHandler`	`LanguageService`	Localization key management

Configuration Dependencies

The messaging system relies on configuration parameters defined in the main config object:

- `kafkaConsumerOptions` - Kafka consumer configuration
- `kafkaTopicOptions` - Topic-specific settings
- `topic.configurationSync` - Primary sync topic name
- `uri.configurationSync` - Configuration sync URI pattern

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L19-L24)

src/consumers/ClientSyncHandler.ts19-24 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/ClientSyncHandler.ts#L19-L24)[

src/consumers/DbSyncHandler.ts38-47](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L38-L47)[
(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/DbSyncHandler.ts#L38-L47)%5B)

src/consumers/LangKeySyncHandler.ts17-24](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L17-L24
(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/consumers/LangKeySyncHandler.ts#L17-L24))

Constants and Enums

Relevant source files

- [

src/constants/ClientStatusEnum.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/ClientStatusEnum.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/ClientStatusEnum.ts))

- [

  src/constants/DataViewStatusEnum.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewStatusEnum.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewStatusEnum.ts))

- [

  src/constants/DataViewTypeEnum.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewTypeEnum.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewTypeEnum.ts))

- [

  src/constants/defaultRequestParameter.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/defaultRequestParameter.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/defaultRequestParameter.ts))

- [

  src/constants/errors.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/errors.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/errors.ts))

- [

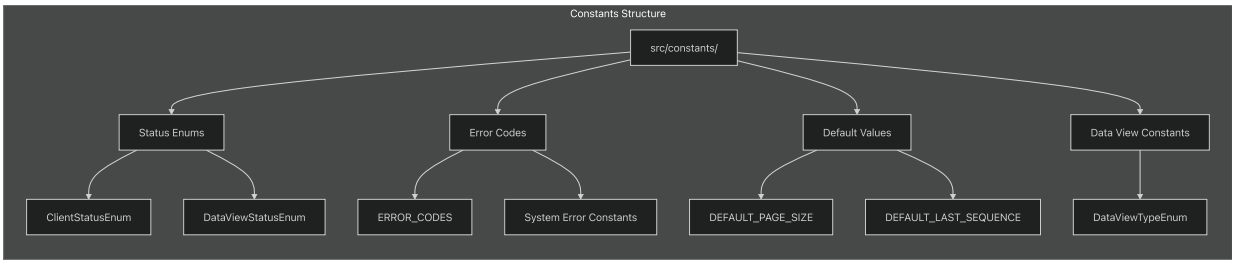
  src/constants/index.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/index.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/index.ts))

This document provides reference documentation for all system constants, error codes, and enumerated types used throughout the TradeX Configuration Service. These constants ensure type safety, standardize error handling, and maintain consistent values across the application.

For information about configuration parameters and environment setup, see Environment Setup. For database schema constants and constraints, see Database Schema.

System Constants Organization

The constants are organized into several categories based on their functional purpose within the service architecture:



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/ClientStatusEnum.ts#L1-L4)

src/constants/ClientStatusEnum.ts1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/ClientStatusEnum.ts#L1-L4)[

src/constants/DataViewStatusEnum.ts1-4](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewStatusEnum.ts#L1-L4)[

(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewStatusEnum.ts#L1-L4)%5B)

src/constants/DataViewTypeEnum.ts1-3](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewTypeEnum.ts#L1-L3)[

(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewTypeEnum.ts#L1-L3)%5B)

src/constants/defaultRequestParameter.ts1-4](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/defaultRequestParameter.ts#L1-L4)[

(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/defaultRequestParameter.ts#L1-L4)%5B)

src/constants/errors.ts1-8](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/errors.ts#L1-L8)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/errors.ts#L1-L8)%5B)

src/constants/index.ts1-2](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/index.ts#L1-L2 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/index.ts#L1-L2))

Status Enumeration Types

Client Status Enumeration

The `ClientStatusEnum` defines the operational states for OAuth clients in the system:

Value	Numeric Code	Description
`DISABLED`	`0`	Client is inactive and cannot authenticate
`ENABLED`	`1`	Client is active and can perform authentication

Implementation: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/ClientStatusEnum.ts#L1-L4)

src/constants/ClientStatusEnum.ts1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/ClientStatusEnum.ts#L1-L4)

Data View Status Enumeration

The `DataViewStatusEnum` controls the visibility and accessibility of data views:

Value	Description
`"ENABLED"`	Data view is accessible and functional
`"DISABLED"`	Data view is hidden or non-functional

Implementation: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewStatusEnum.ts#L1-L4)

src/constants/DataViewStatusEnum.ts1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewStatusEnum.ts#L1-L4)

Data View Type Enumeration

The `DataViewTypeEnum` categorizes the operational mode of data views:

Value	Description
`"SELECT"`	Data view operates in selection mode

Implementation: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewTypeEnum.ts#L1-L3)

src/constants/DataViewTypeEnum.ts1-3 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewTypeEnum.ts#L1-L3)

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/ClientStatusEnum.ts#L1-L4)

src/constants/ClientStatusEnum.ts1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/ClientStatusEnum.ts#L1-L4)[

src/constants/DataViewStatusEnum.ts1-4](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewStatusEnum.ts#L1-L4)[

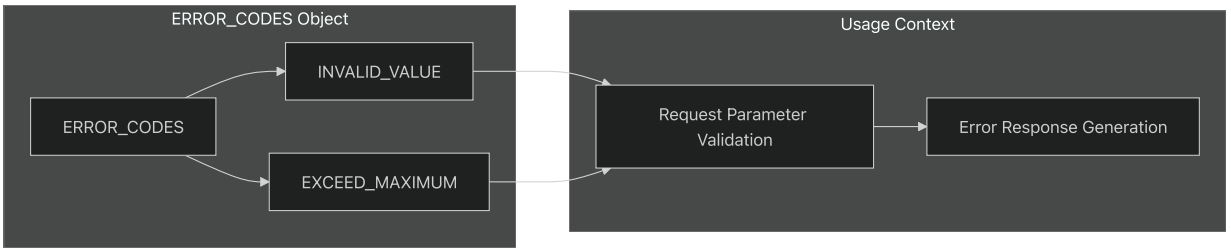
(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewStatusEnum.ts#L1-L4)%5B)

src/constants/DataViewTypeEnum.ts1-3](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewTypeEnum.ts#L1-L3 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewTypeEnum.ts#L1-L3))

Error Code Constants

Request Validation Errors

The `ERROR_CODES` object defines standardized error identifiers for request validation:



Constant	Value	Purpose
`INVALID_VALUE`	`"INVALID_VALUE"`	Parameter value does not meet validation criteria
`EXCEED_MAXIMUM`	`"EXCEED_MAXIMUM"`	Parameter value exceeds allowed maximum

Implementation: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/defaultRequestParameter.ts#L1-L4)

src/constants/defaultRequestParameter.ts1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/defaultRequestParameter.ts#L1-L4)

System Error Constants

The system maintains several specific error constants for various operational failures:

Constant	Value	Context
`FAQ_ALREADY_REVIEWED`	`"FAQ_ALREADY_REVIEWED"`	FAQ review state management
`AWS_GET_SIGNED_DATA_FAILED`	`"AWS_GET_SIGNED_DATA_FAILED"`	S3/MinIO signed URL generation
`DATA_VIEW_CODE_NOT_EXISTED`	`"DATA_VIEW_CODE_NOT_EXISTED"`	Data view lookup operations
`KEY_ALREADY_EXISTED`	`"KEY_ALREADY_EXISTED"`	Duplicate key constraint violations
`UPLOAD_LANGUAGE_RESOURCE_FAILED`	`"UPLOAD_LANGUAGE_RESOURCE_FAILED"`	Localization file operations
`INVALID_PARAMETER`	`"INVALID_PARAMETER"`	General parameter validation
`REQUIRE_AUTHENTICATE`	`"REQUIRE_AUTHENTICATE"`	Authentication requirement enforcement

Implementation: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/errors.ts#L1-L8)

src/constants/errors.ts1-8 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/errors.ts#L1-L8)

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/defaultRequestParameter.ts#L1-L4)

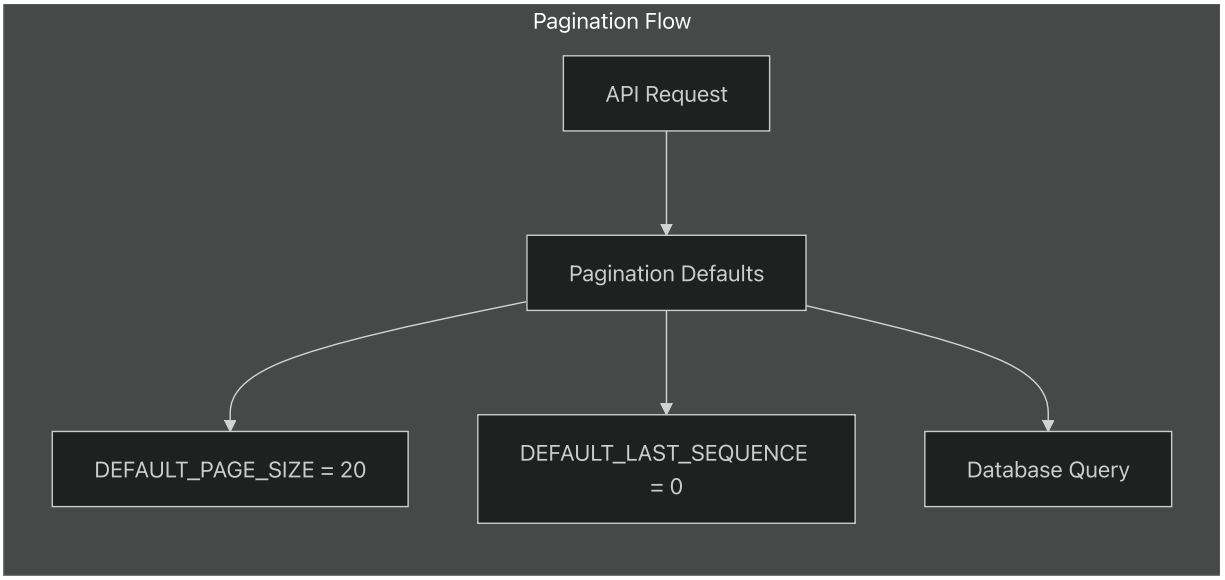
src/constants/defaultRequestParameter.ts1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/defaultRequestParameter.ts#L1-L4)[

src/constants/errors.ts1-8](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/errors.ts#L1-L8 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/errors.ts#L1-L8))

Default System Values

Pagination Constants

The service defines standard pagination parameters used across API endpoints:



Constant	Value	Usage
`DEFAULT_PAGE_SIZE`	`20`	Standard number of records per page when not specified
`DEFAULT_LAST_SEQUENCE`	`0`	Initial sequence value for pagination cursors

Implementation: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/index.ts#L1-L2)

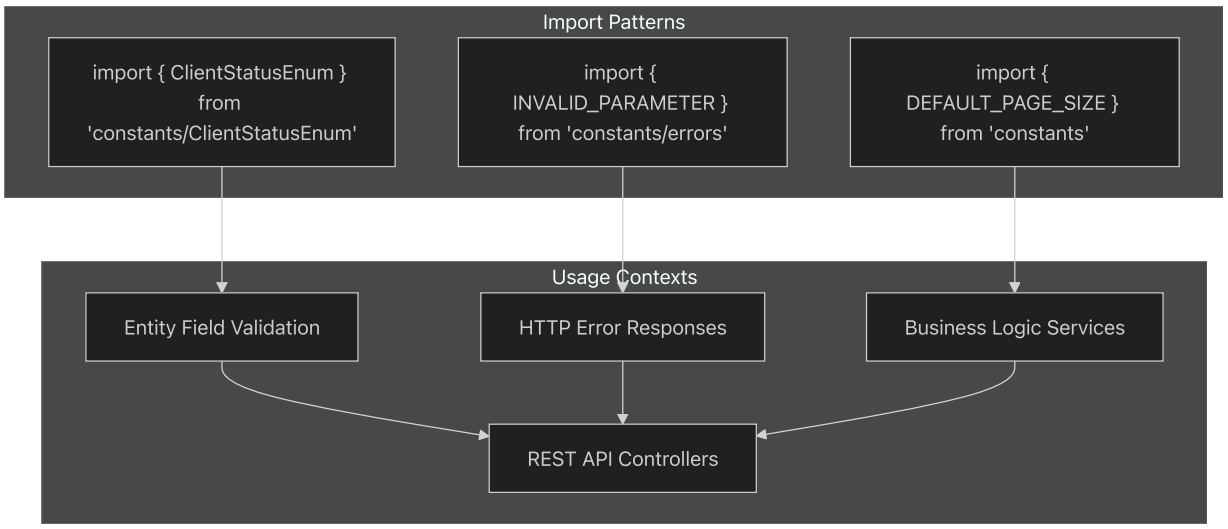
src/constants/index.ts1-2 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/index.ts#L1-L2)

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/index.ts#L1-L2)

src/constants/index.ts1-2 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/index.ts#L1-L2)

Constant Usage Patterns

The constants follow consistent patterns throughout the codebase:



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/ClientStatusEnum.ts#L1-L4)

src/constants/ClientStatusEnum.ts1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/ClientStatusEnum.ts#L1-L4)[

src/constants/errors.ts1-8](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/errors.ts#L1-L8)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/errors.ts#L1-L8)%5B)

src/constants/index.ts1-2](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/index.ts#L1-L2 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/index.ts#L1-L2))

Type Safety and Validation

These constants provide compile-time type safety and runtime validation throughout the service:

- **Enums** enforce valid state values for database entities
- **Error constants** ensure consistent error message formatting
- **Default values** provide fallback behavior for optional parameters

The centralized constant management supports maintainability and reduces the risk of inconsistent values across different service components.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/ClientStatusEnum.ts#L1-L4)

src/constants/ClientStatusEnum.ts1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/ClientStatusEnum.ts#L1-L4)[

src/constants/DataViewStatusEnum.ts1-4](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewStatusEnum.ts#L1-L4)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewStatusEnum.ts#L1-L4)%5B)

src/constants/DataViewTypeEnum.ts1-3](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewTypeEnum.ts#L1-L3)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/DataViewTypeEnum.ts#L1-L3)%5B)

src/constants/defaultRequestParameter.ts1-4](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/defaultRequestParameter.ts#L1-L4)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/defaultRequestParameter.ts#L1-L4)%5B)

src/constants/errors.ts1-8](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/errors.ts#L1-L8)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/errors.ts#L1-L8)%5B)

src/constants/index.ts1-2](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/index.ts#L1-L2 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/constants/index.ts#L1-L2))

Database Schema

Overview

Relevant source files

- [

  dbmigration/dbchangelog.xml](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml))

- [

  src/AppDataSource.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts))

This document provides a comprehensive overview of the database schema for the TradeX Configuration Service, including all tables, their relationships, and organizational structure. The schema supports client management, API registry, access control, localization, menu systems, FAQ management, and system configuration.

For detailed information about specific schema areas, see:

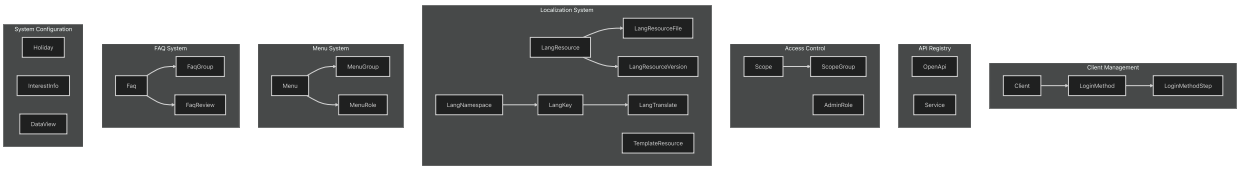
- Client management tables: Client Management Schema
- API endpoint registry: API Registry Schema
- Permission and authentication systems: Access Control Schema

- Database migration orchestration: Migration Management

Schema Overview

The database schema consists of 24 primary entities organized into distinct functional areas. The schema is managed through TypeORM entities and Liquibase migrations, providing a robust foundation for the configuration service's data persistence needs.

Entity Organization by Functional Area



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L36-L61)

src/AppDataSource.ts36-61 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L36-L61)

TypeORM Entity Registration

The database entities are registered in the TypeORM `AppDataSource` configuration, establishing the connection between code entities and database tables:

Entity Class	Primary Purpose	Related Functional Area
`AdminRole`	Administrative role definitions	Access Control
`Client`	OAuth client registration	Client Management
`DataView`	System data view configurations	System Configuration
`Faq`	FAQ content management	FAQ System
`FaqGroup`	FAQ categorization	FAQ System
`FaqReview`	FAQ review workflow	FAQ System
`Holiday`	Holiday calendar data	System Configuration
`InterestInfo`	Interest rate information	System Configuration
`LangKey`	Localization key definitions	Localization System
`LangNamespace`	Localization namespace organization	Localization System
`LangResource`	Localization resource management	Localization System
`LangResourceFile`	Localization file storage	Localization System
`LangResourceVersion`	Localization version control	Localization System
`LangTranslate`	Translation mappings	Localization System
`LoginMethod`	Authentication method definitions	Client Management
`LoginMethodStep`	Multi-step authentication flows	Client Management
`Menu`	Navigation menu items	Menu System
`MenuGroup`	Menu categorization	Menu System
`MenuRole`	Menu access control	Menu System
`OpenApi`	API endpoint registry	API Registry

Entity Class	Primary Purpose	Related Functional Area
`Scope`	Permission scope definitions	Access Control
`ScopeGroup`	Permission grouping	Access Control
`Service`	Service endpoint catalog	API Registry
`TemplateResource`	Template resource management	Localization System

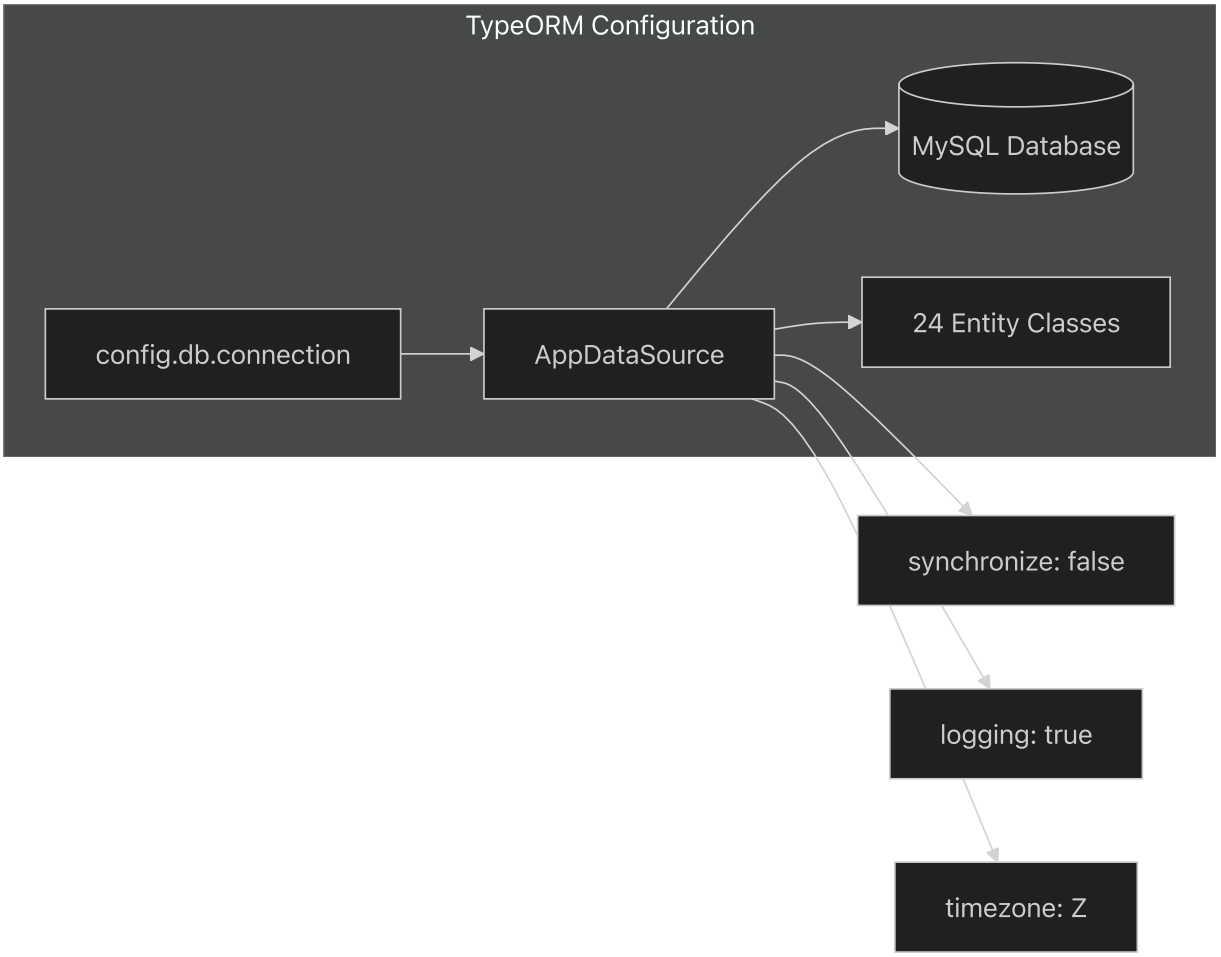
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L4-L27)

src/AppDataSource.ts4-27 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L4-L27)[

src/AppDataSource.ts36-61](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L36-L61 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L36-L61))

Database Connection Configuration

The schema operates on a MySQL database with specific TypeORM configuration settings:



Key configuration parameters:

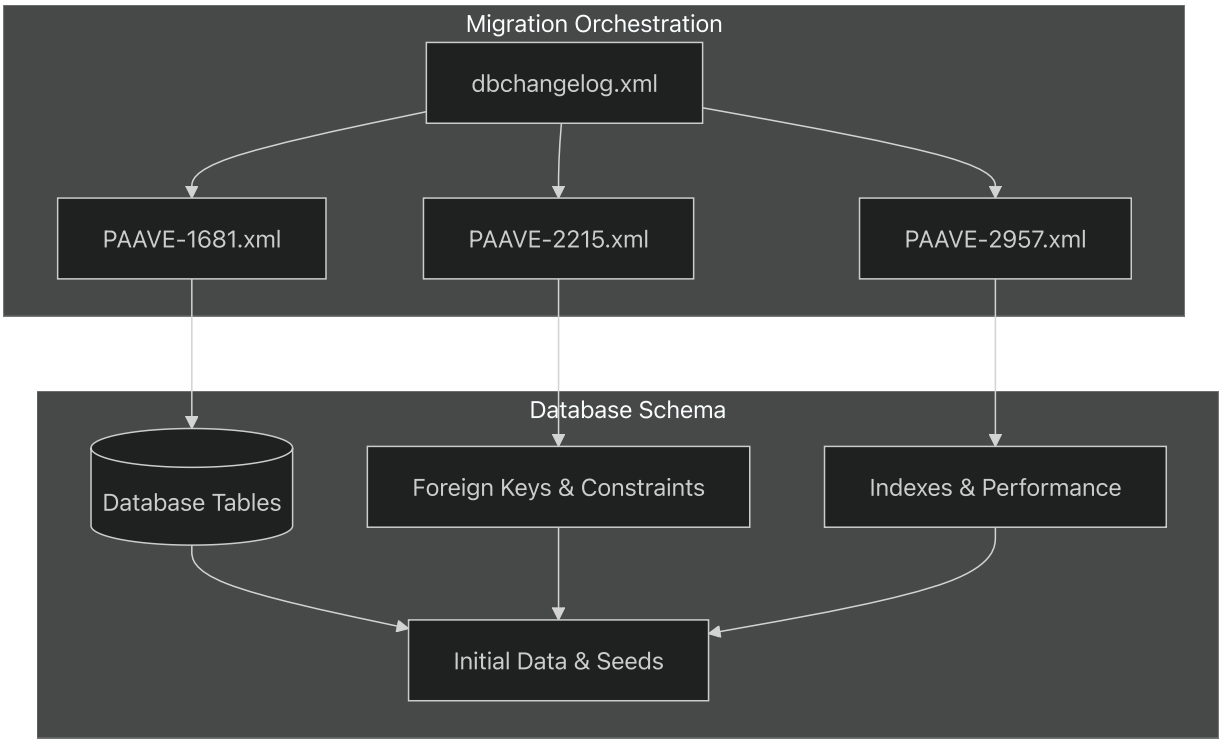
- **Database Type**: MySQL
- **Synchronization**: Disabled (schema managed via migrations)
- **Logging**: Enabled for query debugging
- **Timezone**: UTC (Z)

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L29-L65)

src/AppDataSource.ts29-65 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L29-L65)

Migration Management Structure

The database schema evolution is managed through Liquibase changesets organized in a master changelog:



The master changelog at (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml#L9-L11)

dbmigration/dbchangelog.xml9-11 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml#L9-L11) includes three migration files that collectively establish the complete database schema. For detailed migration management practices, see Migration Management.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml#L9-L11)

dbmigration/dbchangelog.xml9-11 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml#L9-L11)

Schema Integration Points

The database schema integrates with multiple system components through well-defined entity relationships and data access patterns. Each functional area maintains clear boundaries while supporting cross-functional data requirements through established foreign key relationships and shared reference tables.

The schema design supports the service's role as a central configuration hub by providing normalized storage for:

- Multi-tenant client configurations
- Dynamic API endpoint registration
- Granular permission management
- Multi-language content delivery

- Flexible menu and navigation systems
- Comprehensive help and documentation
- System-wide operational parameters

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L1-L68)

src/AppDataSource.ts1-68 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/AppDataSource.ts#L1-L68)[

dbmigration/dbchangelog.xml1-12](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml#L1-L12 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml#L1-L12))

Client Management Schema

Relevant source files

- [

  dbmigration/20200403_t_client.sql](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_client.sql (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_client.sql))

- [

  dbmigration/PAAVE-1681.sql](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.sql (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.sql))

- [

  dbmigration/PAAVE-1681.xml](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml))

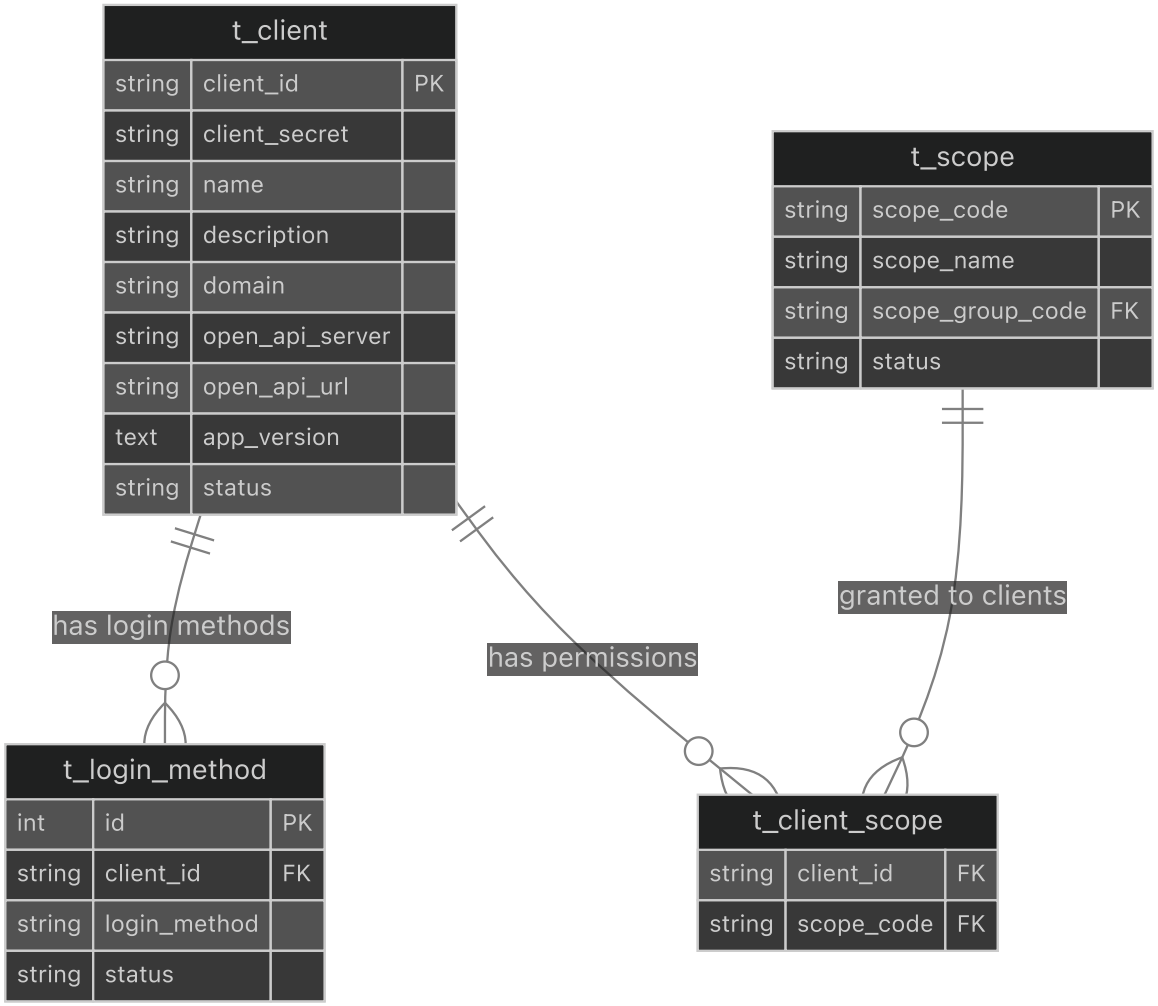
Purpose and Scope

This document covers the database schema components responsible for client management within the TradeX Configuration Service. The client management schema encompasses OAuth client registration, application version tracking, and OpenAPI endpoint integration for client applications.

For information about access control and permission scopes, see Access Control Schema. For API endpoint registration details, see API Registry Schema.

Client Management Database Architecture

The client management system centers around the `t_client` table, which stores OAuth client configurations and metadata for applications connecting to the TradeX platform.



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_client.sql#L1-L4)

dbmigration/20200403_t_client.sql1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_client.sql#L1-L4)[

dbmigration/PAAVE-1681.sql1-6](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.sql#L1-L6 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.sql#L1-L6))

Core t_client Table Structure

The `t_client` table serves as the primary registry for OAuth clients and applications within the TradeX ecosystem.

Column	Type	Purpose	Example
`client_id`	VARCHAR(255)	OAuth client identifier	`paave`, `paavetest`
`client_secret`	VARCHAR(255)	OAuth client secret	Encrypted secret string
`name`	VARCHAR(255)	Human-readable client name	`PAAVE Mobile App`
`description`	TEXT	Client description	Application details
`domain`	VARCHAR(255)	Client domain/origin	`https://app.paave.com`
`open_api_server`	VARCHAR(255)	OpenAPI server endpoint	API server URL
`open_api_url`	VARCHAR(255)	OpenAPI specification URL	Swagger/OpenAPI spec location
`app_version`	TEXT	JSON version mapping	`{"IOS":"1.6.0","ANDROID":"1.6.0"}`
`status`	VARCHAR(50)	Client status	`ACTIVE`, `INACTIVE`

The table structure supports OAuth 2.0 client credentials flow and provides metadata for client application management.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_client.sql#L1-L4)

dbmigration/20200403_t_client.sql1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_client.sql#L1-L4)[

dbmigration/PAAVE-1681.sql1-2](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.sql#L1-L2 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.sql#L1-L2))

App Version Management

The `app_version` column stores platform-specific version information as JSON text, enabling version control and compatibility checking for mobile and web applications.

The migration (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/PAAVE-1681.xml#L25-L47)

PAAVE-1681.xml25-47 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/PAAVE-1681.xml#L25-L47) demonstrates the initialization of version data for existing clients:

- `paave` client: Standard platform keys (`IOS` , `ANDROID`)
- Platform-specific keys: `M.PAAVE.OS` (iOS), `M.PAAVE.AN` (Android)

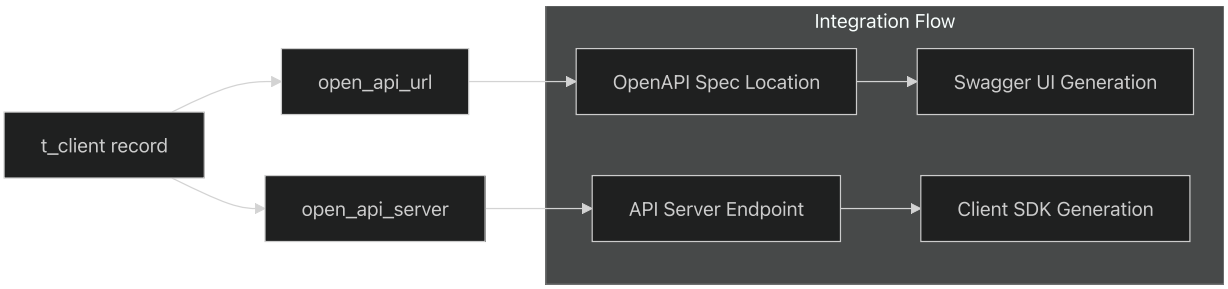
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.sql#L1-L6)

dbmigration/PAAVE-1681.sql1-6 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.sql#L1-L6)[

dbmigration/PAAVE-1681.xml10-47](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L10-L47 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L10-L47))

OpenAPI Integration Fields

The OpenAPI integration fields enable automatic API documentation and endpoint discovery for client applications.



These fields were added in migration (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/20200403_t_client.sql#L1-L4)

20200403_t_client.sql1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/20200403_t_client.sql#L1-L4) to support:

- **`open_api_server`** : Base URL for the client's API server
- **`open_api_url`** : Location of the OpenAPI/Swagger specification

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_client.sql#L1-L4)

dbmigration/20200403_t_client.sql1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_client.sql#L1-L4)

Migration History and Schema Evolution

The client management schema has evolved through several key migrations:

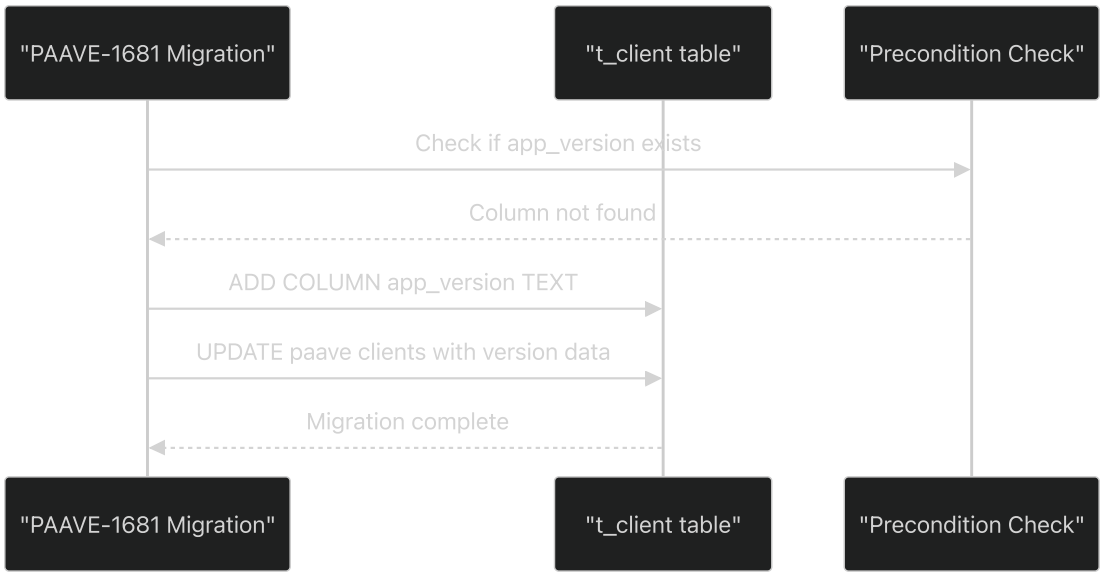
Migration	Purpose	Changes
`20200403_t_client.sql`	OpenAPI Integration	Added `open_api_server`, `open_api_url` columns
`PAAVE-1681`	App Versioning	Added `app_version` TEXT column with JSON structure

PAAVE-1681 Migration Details

The app version migration (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/PAAVE-1681.xml#L10-L47)

PAAVE-1681.xml10-47 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/PAAVE-1681.xml#L10-L47) includes:

1. **Preconditions**: Checks if `app_version` column exists before adding
2. **Column Addition**: Adds `app_version` as TEXT type after `status` column
3. **Data Population**: Updates existing `paave`, `paavetest`, and `paave-non-login` clients



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L10-L47)

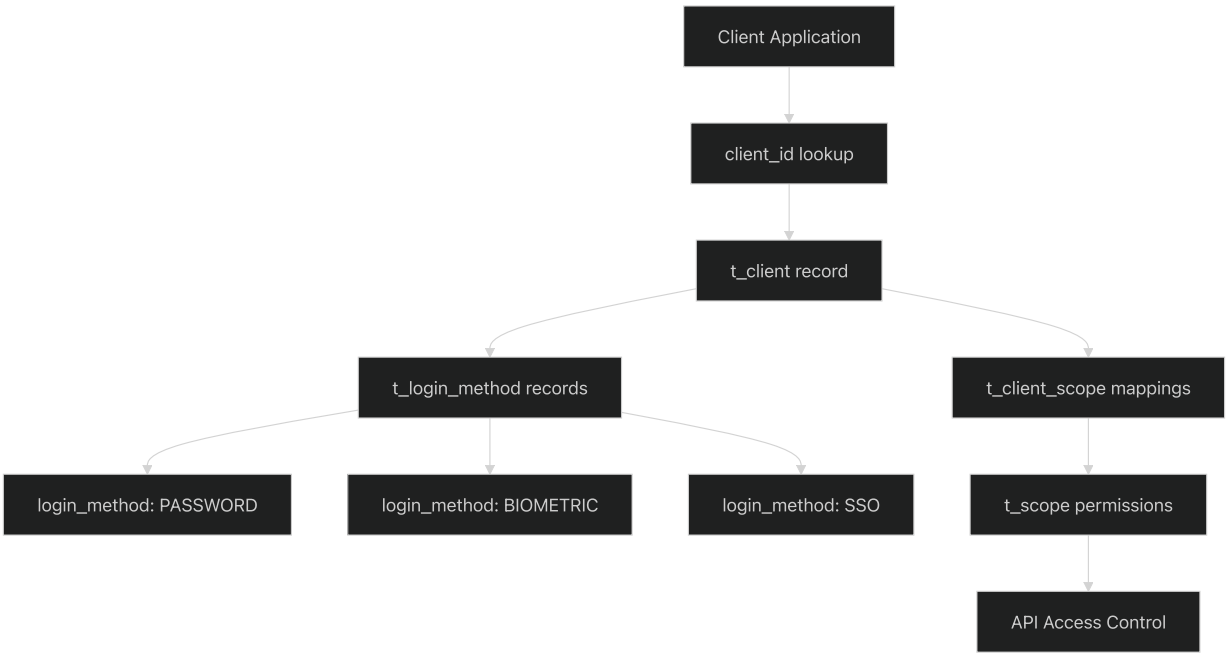
dbmigration/PAAVE-1681.xml10-47 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L10-L47)[

dbmigration/PAAVE-1681.sql1-6](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.sql#L1-L6 (https://github.com/nhsv-

tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.sql#L1-L6))

Client Authentication Flow

The client management schema supports OAuth 2.0 authentication patterns through the relationship between `t_client` and `t_login_method` tables.



Each client can have multiple login methods and scope permissions, enabling flexible authentication and authorization configurations.

Sources: Based on schema relationships inferred from migration files and table structure

---

API Registry Schema

Relevant source files

- [

  dbmigration/20200403_t_open_api.sql](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql))

- [

  dbmigration/20200526_all_all_t_open_api_ai_advisor.sql](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql))

This document covers the database schema and structure of the OpenAPI endpoint registry system (`t_open_api` table) within the TradeX Configuration Service. This registry serves as a centralized metadata store for all API endpoints across the TradeX platform, storing OpenAPI specifications and endpoint documentation.

For information about scope-based access control and permissions, see Access Control Schema. For client management and OAuth configurations, see Client Management Schema.

Purpose and Overview

The API Registry Schema provides a centralized repository for storing OpenAPI specifications, endpoint metadata, and API documentation for all services within the TradeX ecosystem. This schema enables dynamic API discovery, documentation generation, and integration management across the platform.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L18-L40)

dbmigration/20200403_t_open_api.sql18-40 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L18-L40)[

dbmigration/20200526_all_all_t_open_api_ai_advisor.sql1-4](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4))

Table Structure

Core Schema Definition

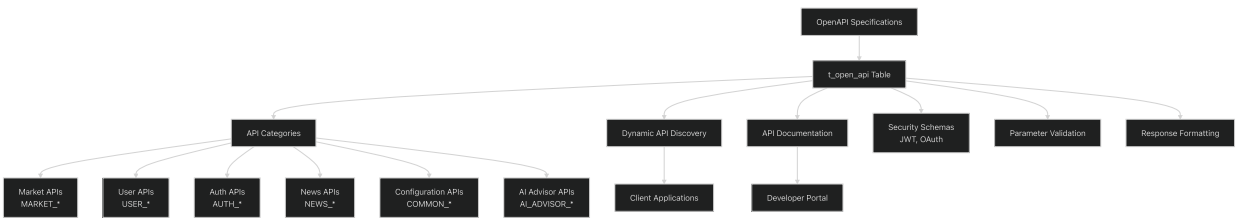
The `t_open_api` table stores comprehensive OpenAPI specification data with the following structure:

Column	Type	Description
`id`	int(11)	Primary key identifier
`operation_id`	varchar(255)	Unique operation identifier for the endpoint
`tags`	json	Categorization tags for grouping related endpoints
`uri_pattern`	varchar(255)	HTTP method and URL pattern (e.g., "get:/api/v1/market/stock")
`summary`	mediumtext	Brief description of the endpoint functionality
`security`	json	Security requirements and authentication schemas
`parameters`	json	Request parameters specification
`request_body`	json	Request body schema for POST/PUT operations
`responses`	json	Response schemas and status codes
`description`	text	Detailed endpoint description
`version`	varchar(45)	API version identifier
`created_at`	timestamp	Record creation timestamp
`updated_at`	timestamp	Last modification timestamp

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L25-L40)

dbmigration/20200403_t_open_api.sql25-40 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L25-L40)

API Registry Data Flow



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L48-L49)

dbmigration/20200403_t_open_api.sql48-49 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L48-L49)[

dbmigration/20200526_all_all_t_open_api_ai_advisor.sql1-4](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4))

API Category Organization

Primary API Categories

The registry organizes endpoints into distinct functional categories based on their `operation_id` prefixes and `tags`:

Market Data APIs

- **Prefix Pattern:** `MARKET_*`
- **Tags:** `["Market-v1"]`
- **Examples:** `MARKET_LISTED_STOCK`, `MARKET_INDEX_CURRENT`, `MARKET_STOCK_CURRENT_PRICE`
- **URI Patterns:** `/api/v1/market/*`

User Management APIs

- **Prefix Pattern:** `USER_*`
- **Tags:** `["User"]`, `["User utility v1"]`
- **Examples:** `USER_FAVORITE_LIST`, `USER_ALARM_LIST`, `USER_REGISTER_USER`
- **URI Patterns:** `/api/v1/user/*`, `/api/v1/favorite/*`, `/api/v1/alarm/*`

Authentication APIs

- **Prefix Pattern:** `AUTH_*`
- **Tags:** `["Authentication"]`
- **Examples:** `AUTH_VERIFY_OTP`
- **URI Patterns:** `/api/v1/login/*`

Configuration APIs

- **Prefix Pattern:** `COMMON_*`
- **Tags:** `["Configuration"]`
- **Examples:** `COMMON_LOCALE_RESOURCE`, `COMMON_FAQ`, `COMMON_AWS`

- **URI Patterns:** `/api/v1/locale`, `/api/v1/faq/*`, `/api/v1/aws`

AI Advisor APIs

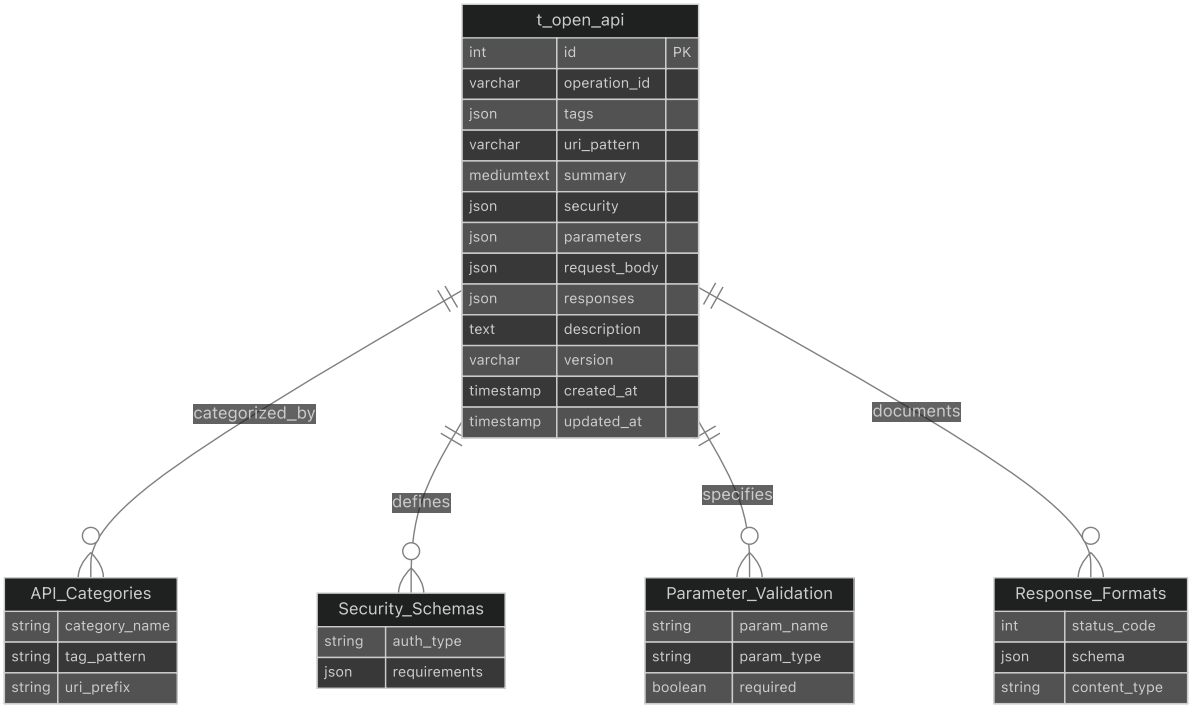
- **Prefix Pattern:** `AI_ADVISOR_*`
- **Tags:** `["Ai-advisor"]`
- **Examples:** `AI_ADVISOR_DIARY_LIST2`, `AI_ADVISOR_NEWS_LIST`
- **URI Patterns:** `/api/v2/aiAdvisor/*`

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L49-L49)

dbmigration/20200403_t_open_api.sql49 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L49-L49)[

dbmigration/20200526_all_all_t_open_api_ai_advisor.sql1-4](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4))

API Endpoint Mapping



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L25-L40)

dbmigration/20200403_t_open_api.sql25-40 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L25-L40)

Data Storage Patterns

JSON Schema Storage

The registry extensively uses JSON columns to store complex OpenAPI specifications:

Security Schema Format

```
[{"jwt": []}]
[{"jwt": []}, {"secToken": []}, {"secDomain": []}]
```

Parameters Schema Format

```
[{
  "in": "query",
  "name": "marketType",
  "schema": {"enum": ["ALL", "HNX", "HOSE", "UPCOM"], "type": "string"},
  "description": "the market to query summary info"
}]
```

Response Schema Format

```
{
  "200": {
    "content": {
      "application/json": {
        "schema": {
          "type": "array",
          "items": {"type": "object", "properties": {...}}
        }
      }
    },
    "description": "OK"
  },
  "401": {"description": "Your access token is invalid or expired"}
}
```

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L49-L49)

dbmigration/20200403_t_open_api.sql49 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L49-L49)

Version Management

The registry supports API versioning through:

- **Version Column:** Explicit version tracking (`/api/v1` , `/api/v2`)

- **URI Patterns:** Version-specific endpoint patterns
- **Evolution Tracking:** Created/updated timestamps for change history

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L36-L38)

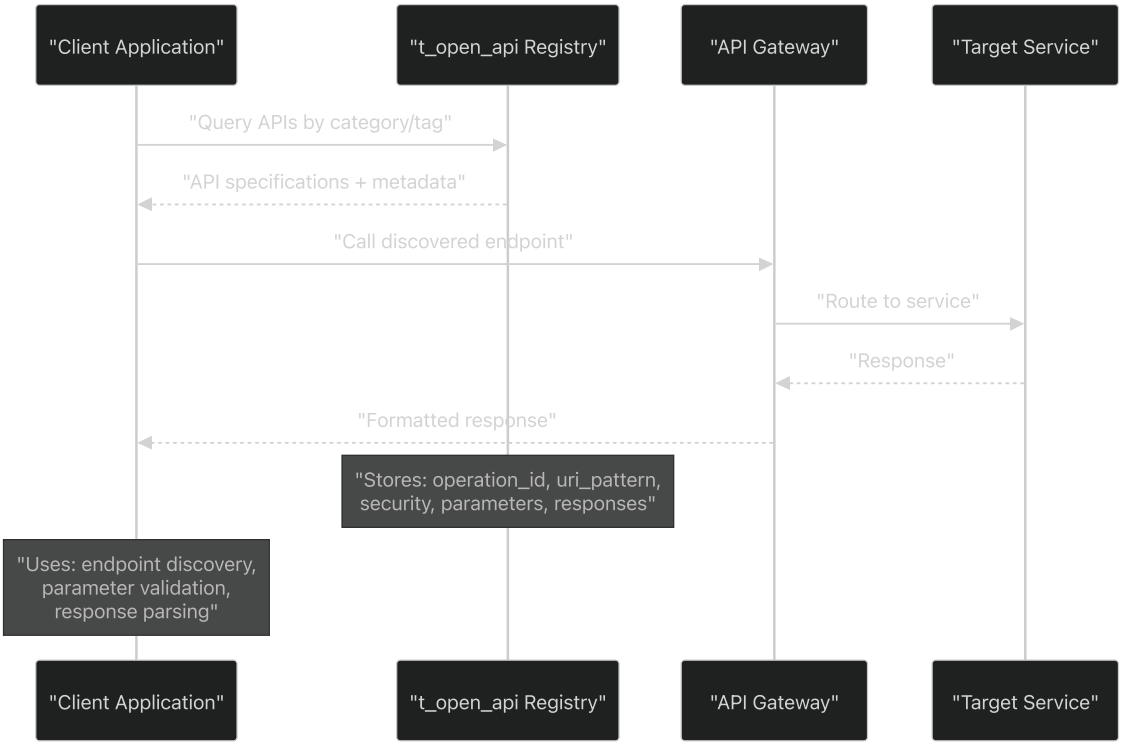
dbmigration/20200403_t_open_api.sql36-38 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L36-L38)[

dbmigration/20200526_all_all_t_open_api_ai_advisor.sql1-4](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4))

Integration Patterns

Dynamic API Discovery

The registry enables dynamic API discovery through structured queries:



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L27-L34)

dbmigration/20200403_t_open_api.sql27-34 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L27-L34)

Security Integration

The registry integrates with the access control system by storing security requirements that map to scope-based permissions:

- **JWT Authentication:** Standard token-based access
- **SEC Integration:** Securities trading system authentication
- **Domain-Specific Security:** Service-specific security contexts

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L31-L31)

dbmigration/20200403_t_open_api.sql31 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L31-L31)

AI Advisor Integration

The registry includes specialized support for AI Advisor APIs with enhanced metadata:

AI Advisor Endpoints

- **Diary Management:** `AI_ADVISOR_DIARY_LIST2`, `AI_ADVISOR_DIARY`
- **News Management:** `AI_ADVISOR_NEWS_LIST`, `AI_ADVISOR_NEWS`
- **Content Operations:** CRUD operations for diary and news content
- **Version Support:** Dedicated `/api/v2` version for AI features

Schema Extensions

The AI Advisor APIs utilize enhanced parameter and response schemas for content management, including pagination, search capabilities, and rich media support.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4)

dbmigration/20200526_all_all_t_open_api_ai_advisor.sql1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4)

Access Control Schema

Relevant source files

- [

  dbmigration/20200526_all_all_t_scope_ai_advisor.sql](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql))

- [

  dbmigration/PAAVE-2215.xml](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml (https://github.com/nhsv-

tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml))

- [

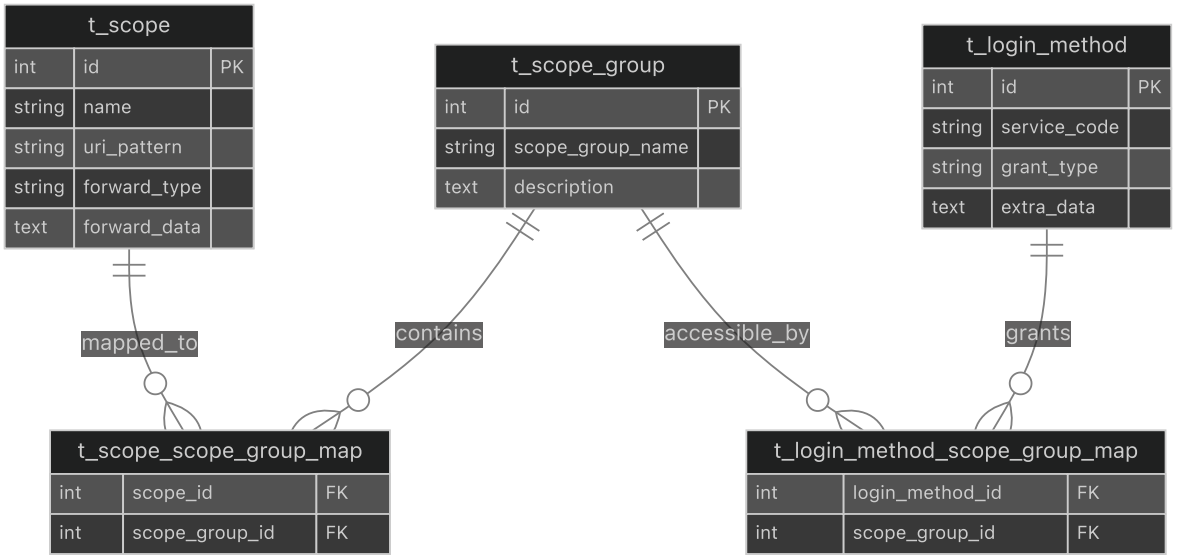
  dbmigration/PAAVE-2957.xml](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml))

This document covers the database schema components responsible for managing access control, permissions, and authentication within the TradeX Configuration Service. The access control system implements a scope-based permission model with role-based access control through scope groups and login method configurations.

For client OAuth configuration details, see Client Management Schema. For API endpoint registry and service routing, see API Registry Schema.

Core Access Control Tables

The access control schema consists of five primary tables that work together to implement a flexible permission system:



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L1-L11)

dbmigration/20200526_all_all_t_scope_ai_advisor.sql1-11 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L1-L11)[

dbmigration/PAAVE-2215.xml11-16](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L11-L16)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L11-L16)%5B)

dbmigration/PAAVE-2957.xml10-12](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml#L10-L12 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml#L10-L12))

Scope Management Schema

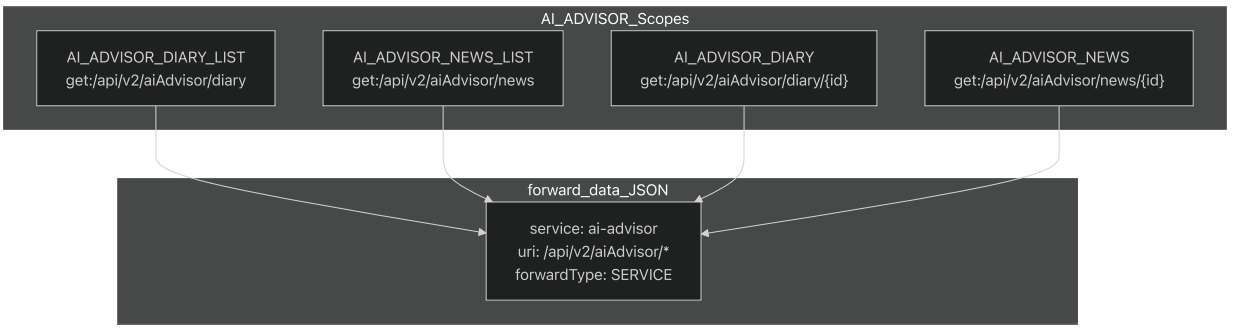
t_scope Table Structure

The `t_scope` table defines individual permissions and API endpoint access patterns:

Column	Type	Purpose
`id`	INT	Primary key identifier
`name`	VARCHAR	Human-readable scope name
`uri_pattern`	VARCHAR	HTTP method and URI pattern (e.g., "get:/api/v2/aiAdvisor/diary")
`forward_type`	VARCHAR	Routing type ('CONNECTION' for service forwarding)
`forward_data`	TEXT	JSON configuration for service routing

Scope Categories

Based on the migration data, scopes are categorized by functionality:



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L1-L4)

dbmigration/20200526_all_all_t_scope_ai_advisor.sql1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L1-L4)

t_scope_group Table Structure

The `t_scope_group` table organizes scopes into logical permission groups:

Column	Type	Purpose
`id`	INT	Primary key identifier
`scope_group_name`	VARCHAR	Group identifier (e.g., 'PUBLIC', 'LINK_ACCOUNT')
`description`	TEXT	Human-readable group description

Login Method Schema

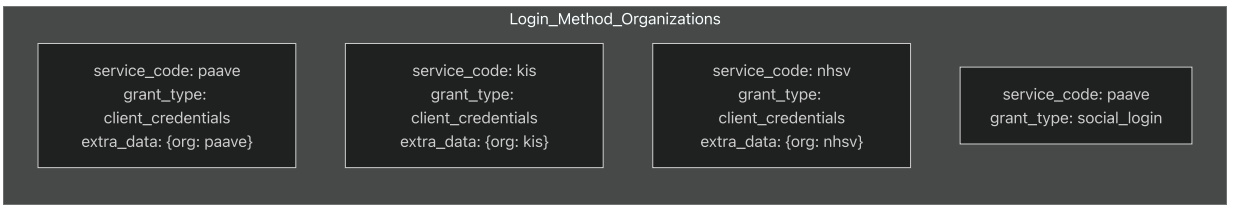
t_login_method Table Structure

The `t_login_method` table defines authentication methods and their associated metadata:

Column	Type	Purpose
`id`	INT	Primary key identifier
`service_code`	VARCHAR	Service identifier ('paave', 'kis', 'nhsv')
`grant_type`	VARCHAR	OAuth grant type ('social_login', 'client_credentials')
`extra_data`	TEXT	JSON metadata for organization mapping

Organization Mapping

Login methods include organization context through the `extra_data` JSON field:



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml#L16-L23)

dbmigration/PAAVE-2957.xml16-23 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml#L16-L23)

Permission Mapping Schema

Scope-to-Group Mapping

The `t_scope_scope_group_map` table creates many-to-many relationships between scopes and scope groups:

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L8-L11)

dbmigration/20200526_all_all_t_scope_ai_advisor.sql8-11 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L8-L11)[

dbmigration/PAAVE-2215.xml11-16](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L11-L16 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L11-L16))

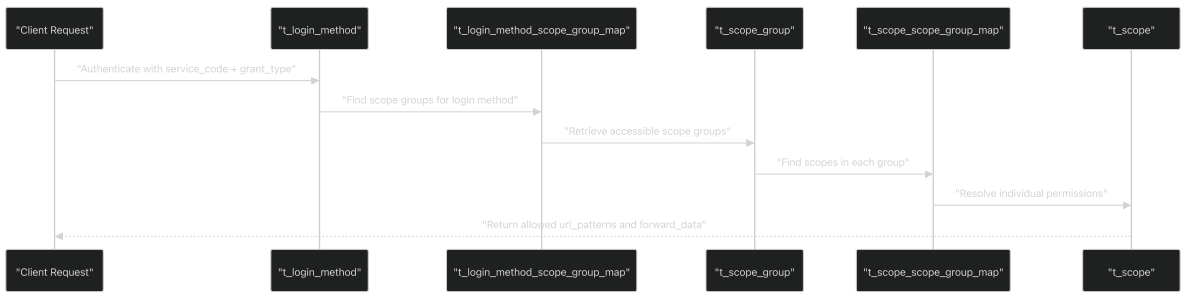
Login Method Permission Assignment

The `t_login_method_scope_group_map` table assigns scope groups to authentication methods:

Mapping Type	Login Method	Scope Group	Purpose
Social Authentication	`paave` + `social_login`	`LINK_ACCOUNT`	Account linking permissions

Access Control Data Flow

The following diagram illustrates how permissions are resolved through the schema relationships:



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L1-L11)

dbmigration/20200526_all_all_t_scope_ai_advisor.sql1-11 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L1-L11)[

dbmigration/PAAVE-2215.xml11-16](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L11-L16)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L11-L16)%5B)

dbmigration/PAAVE-2957.xml10-23](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml#L10-L23 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml#L10-L23))

Migration Management

Relevant source files

- [

  dbmigration/PAAVE-1681.xml](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml))

- [

  dbmigration/PAAVE-2215.xml](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml))

- [

  dbmigration/PAAVE-2957.xml](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml))

- [

  dbmigration/dbchangelog.xml](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml))

Purpose and Scope

This document explains the database migration management system used by the TradeX Configuration Service. The migration system uses Liquibase to orchestrate schema changes and

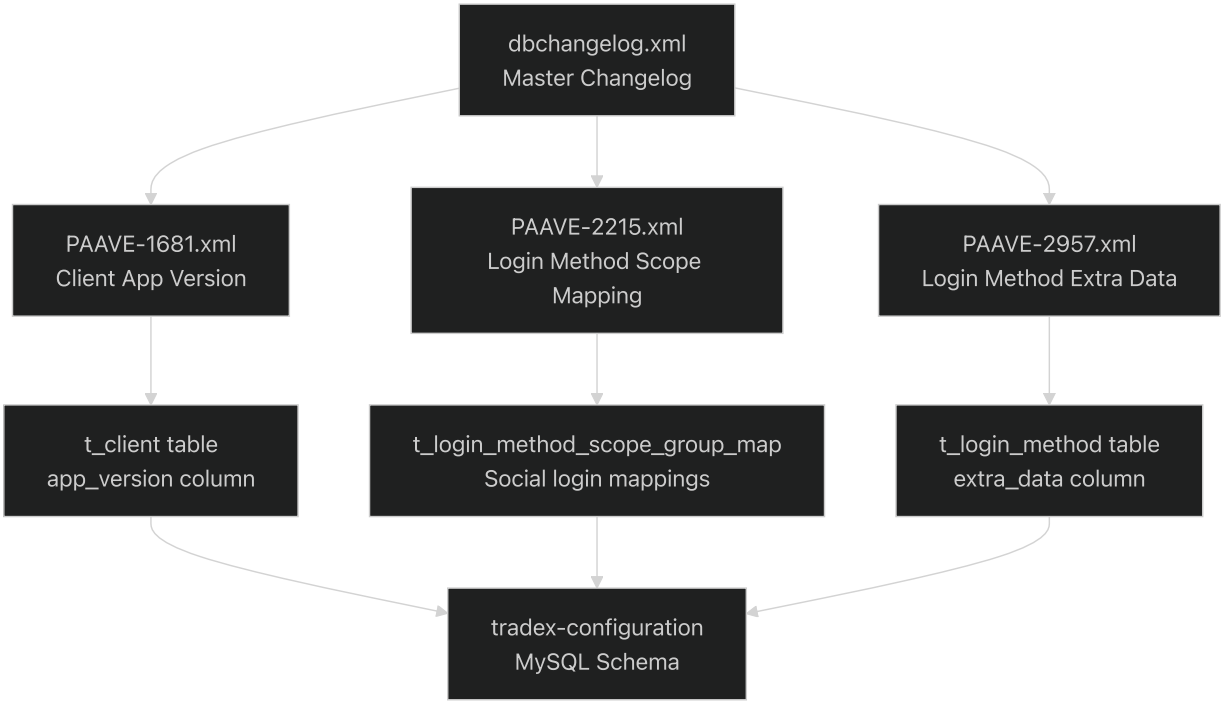
data modifications across the `tradex-configuration` database schema. This includes the structure, organization, and execution patterns of database changesets.

For information about the actual database tables and relationships created by these migrations, see Database Schema. For details about the specific schemas for client management, API registry, and access control, see Client Management Schema, API Registry Schema, and Access Control Schema.

Migration Architecture

The TradeX Configuration Service employs Liquibase as its database migration framework, organizing migrations through a master changelog that orchestrates individual feature-based migration files.

Migration Orchestration Flow



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml#L9-L11)

dbmigration/dbchangelog.xml9-11 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml#L9-L11)[

dbmigration/PAAVE-1681.xml](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml)%5B)

dbmigration/PAAVE-2215.xml](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml)%5B)

dbmigration/PAAVE-2957.xml](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml))

Migration File Structure

Master Changelog

The (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml)

dbmigration/dbchangelog.xml (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml) file serves as the migration orchestrator, defining the execution order of all database changes through `include` statements.

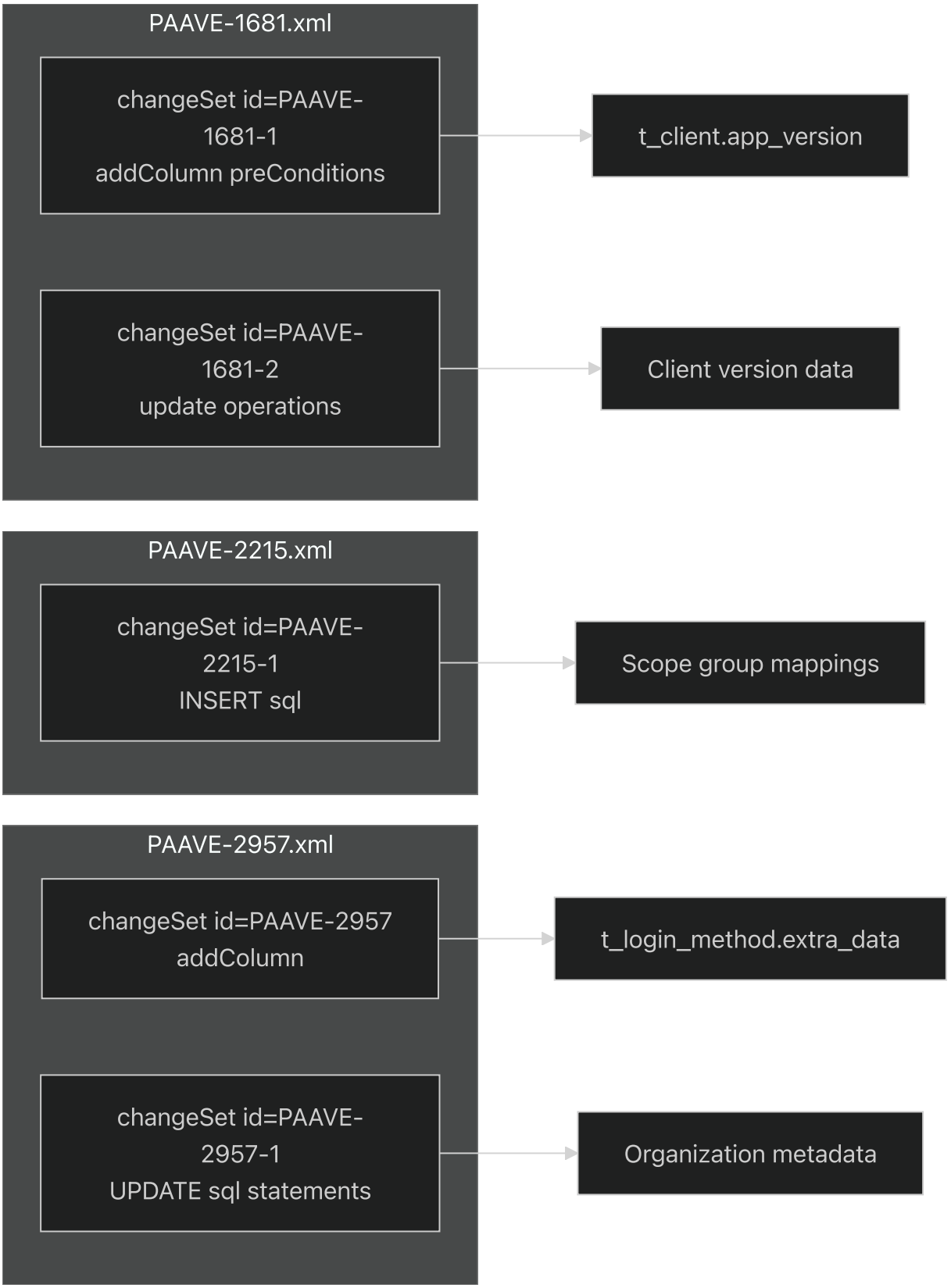
Component	Purpose	File Reference
`databaseChangeLog`	Root Liquibase namespace container	[
dbmigration/dbchangelog.xml2-8](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml#L2-L8 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml#L2-L8))		
`include` statements	Order-dependent migration file inclusion	[

dbmigration/dbchangelog.xml9-11](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml#L9-L11 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/dbchangelog.xml#L9-L11)) |

Individual Migration Files

Each migration file follows the pattern `PAAVE-{ticket-number}.xml` and contains one or more changesets targeting specific database modifications.

Migration Components



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L10-L47)

dbmigration/PAAVE-1681.xml10-47 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L10-L47)[

dbmigration/PAAVE-2215.xml10-17](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L10-L17)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L10-L17)%5B)

dbmigration/PAAVE-2957.xml9-24](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml#L9-L24 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml#L9-L24))

Migration Patterns and Best Practices

PreConditions for Idempotent Execution

The migration system implements safeguards to ensure migrations can be executed multiple times without adverse effects.

Column Existence Checking

```
<preConditions onFail="MARK_RAN">
    <not>
        <columnExists schemaName="tradex-configuration" tableName="t_client" col
    </not>
</preConditions>
```

This pattern ensures the `addColumn` operation in (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L11-L16)

dbmigration/PAAVE-1681.xml11-16 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L11-L16) only executes when the `app_version` column does not already exist in the `t_client` table.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L11-L16)

dbmigration/PAAVE-1681.xml11-16 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L11-L16)

Data Initialization Patterns

Multi-Target Updates

The system performs targeted data updates for specific client configurations:

Client ID	App Version Data	Purpose
`paave`	`{"M.PAAVE.OS":"1.6.0","M.PAAVE.AN":"1.6.0"}`	Production client versioning
`paavetest`	`{"M.PAAVE.OS":"1.6.0","M.PAAVE.AN":"1.6.0"}`	Test environment versioning
`paave-non-login`	`{"M.PAAVE.OS":"1.6.0","M.PAAVE.AN":"1.6.0"}`	Non-authenticated client versioning

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L26-L46)

dbmigration/PAAVE-1681.xml26-46 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L26-L46)

Organization Metadata Assignment

The `extra_data` column initialization demonstrates organization-specific configuration:

```
UPDATE `tradex-configuration`.`t_login_method` SET `extra_data` = '{"org": "paav
```

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml#L16-L23)

dbmigration/PAAVE-2957.xml16-23 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml#L16-L23)

Complex Query Operations

Relationship Mapping with JOIN Logic

The (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L11-L16)

dbmigration/PAAVE-2215.xml11-16 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L11-L16) migration demonstrates sophisticated data relationship establishment:

```
INSERT INTO `tradex-configuration`.`t_login_method_scope_group_map` (`login_meth
SELECT lmt.id, scg.id
FROM `tradex-configuration`.`t_login_method` lmt, `tradex-configuration`.`t_scop
WHERE lmt.grant_type = 'social_login' AND lmt.service_code = 'paave' AND scg.scc
ON DUPLICATE KEY UPDATE login_method_id = lmt.id, scope_group_id = scg.id;
```

This operation creates mappings between social login methods and scope groups while handling duplicate key conflicts gracefully.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L11-L16)

dbmigration/PAAVE-2215.xml11-16 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L11-L16)

Migration Execution Context

Database Schema Target

All migrations target the `tradex-configuration` MySQL schema, which is explicitly specified in table references and schema-qualified operations.

Change Tracking

Each changeset includes:

- `author` attribution for change ownership
- `id` for unique changeset identification
- Conditional logic for safe re-execution
- Specific database operation types (`addColumn` , `update` , `sql`)

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L10-L10)

dbmigration/PAAVE-1681.xml10 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-1681.xml#L10-L10)[

dbmigration/PAAVE-2215.xml10](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L10-L10)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2215.xml#L10-L10)%5B)

dbmigration/PAAVE-2957.xml9-14](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml#L9-L14 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/PAAVE-2957.xml#L9-L14))

Integration with Database Schema

The migration system creates and modifies core tables that support the configuration service's functionality:

- `t_client` table modifications for client application versioning
- `t_login_method` enhancements for authentication method metadata
- `t_login_method_scope_group_map` relationship establishment for permission mapping

These migrations directly support the schemas documented in Client Management Schema and Access Control Schema.

API Reference

Overview

Relevant source files

- [

  README.md](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md
  (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md))

- [

  dbmigration/20200403_t_open_api.sql](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql
  (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql))

- [

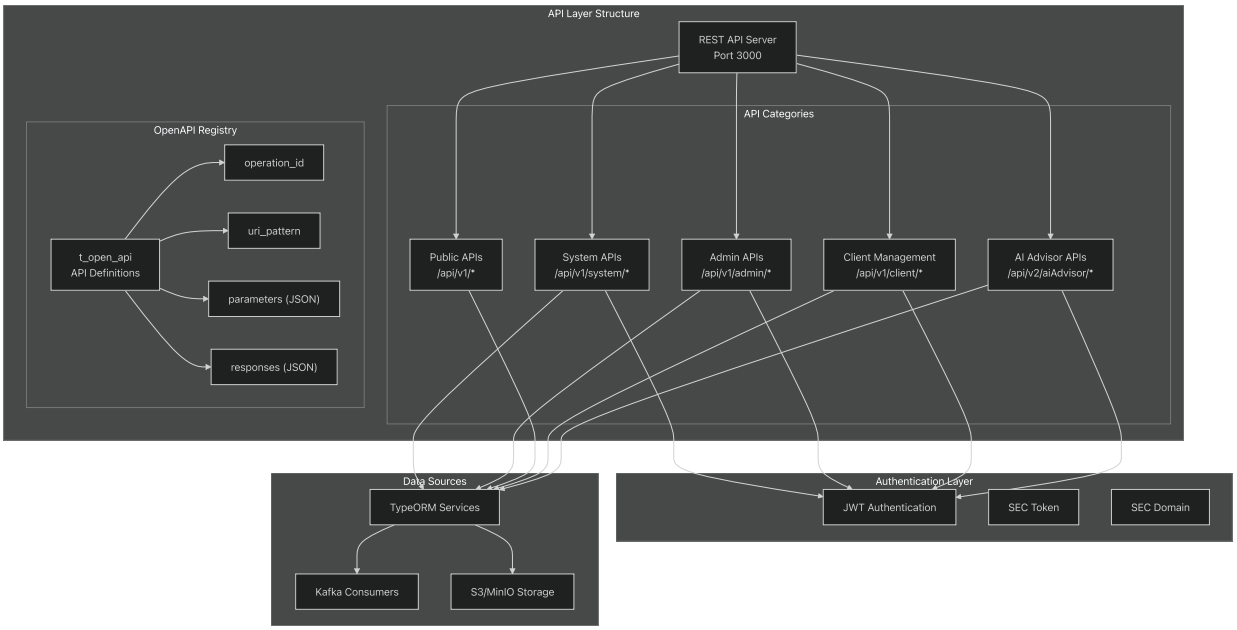
  dbmigration/20200526_all_all_t_open_api_ai_advisor.sql](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql
  (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql))

This document provides comprehensive reference documentation for all REST API endpoints exposed by the TradeX Configuration Service. The API serves as the central configuration hub for the TradeX trading platform, managing client authentication, access control, localization resources, and system settings.

For information about the underlying database schema that supports these APIs, see Database Schema. For deployment and configuration details, see Configuration and Deployment.

API Architecture Overview

The TradeX Configuration Service exposes a RESTful API organized into distinct functional categories, each serving specific aspects of platform configuration management.



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L129-L157)

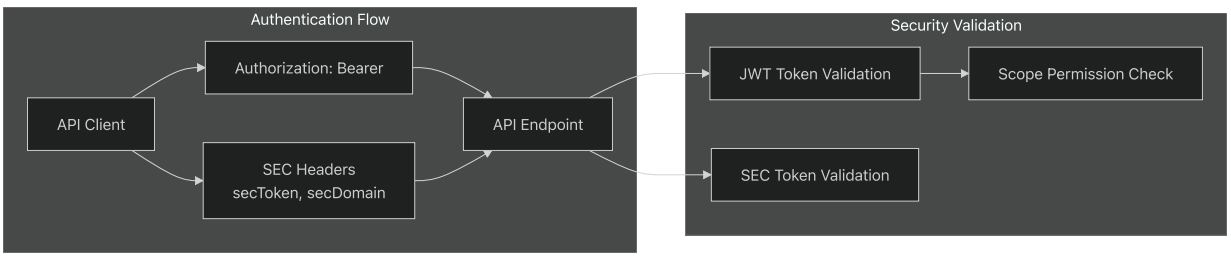
README.md129-157 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L129-L157)[

dbmigration/20200403_t_open_api.sql25-40](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L25-L40
(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L25-L40))

Authentication and Security

All API endpoints require authentication through JWT tokens, with some endpoints requiring additional SEC (Securities) authentication for trading-related operations.

Security Scheme	Usage	Format
`jwt`	Standard authentication for all endpoints	Bearer token in Authorization header
`secToken`	Additional token for trading operations	SEC-specific token
`secDomain`	Domain validation for SEC operations	SEC domain identifier



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L49-L49)

dbmigration/20200403_t_open_api.sql49 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L49-L49)[

README.md234-237](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L234-L237 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L234-L237))

System APIs

System APIs provide core platform functionality for querying clients, authentication methods, and permission scopes. These endpoints are primarily used by other services within the TradeX ecosystem.

Client Information

- **GET** `/api/v1/system/client` - Query clients for updates
- **GET** `/api/v1/system/loginMethod` - Query available login methods
- **GET** `/api/v1/system/scope` - Query permission scopes
- **GET** `/api/v1/system/scopeGroup` - Query scope groups

Response Schema Examples

Client Query Response:

```
{
  "id": "string",
  "clientId": "string",
  "clientSecret": "string",
  "name": "string",
  "description": "string"
}
```

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L132-L135)

README.md132-135 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L132-L135)

Admin APIs

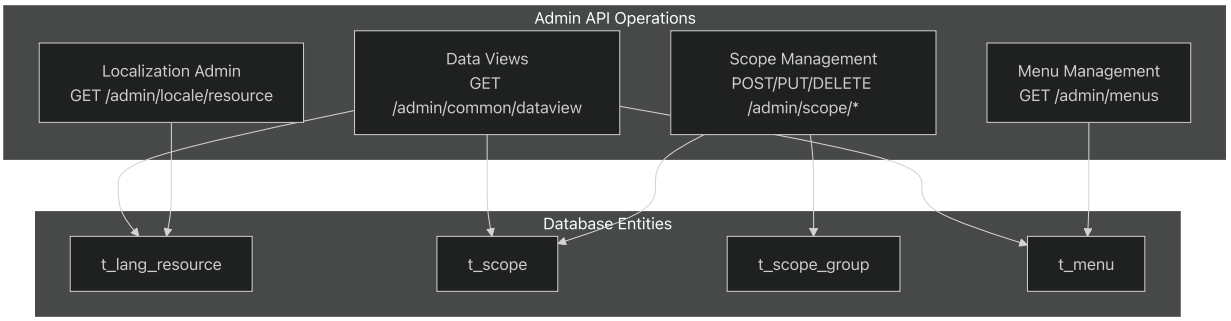
Administrative APIs provide CRUD operations for system configuration, localization resources, and access control management.

Configuration Management

- **GET** `/api/v1/admin/common/dataview` - Get data by view
- **GET** `/api/v1/admin/locale/resource` - Get all language resources
- **GET** `/api/v1/admin/menus` - Get menus by role IDs

Scope Management

- **POST** `/api/v1/admin/scope/add` - Add new scope
- **PUT** `/api/v1/admin/scope/{id}/update` - Update existing scope
- **DELETE** `/api/v1/admin/scope/{id}/delete` - Delete scope



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L137-L143)

README.md137-143 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L137-L143)

Public APIs

Public APIs provide access to localized content, FAQ information, and system configuration data that doesn't require special privileges.

Localization and Content

- **GET** `/api/v1/locale` - Get localized resources by service names
- **GET** `/api/v1/faq/{serviceName}` - Get FAQs for specific service
- **POST** `/api/v1/faq/{faqId}/review/{isUseful}` - Review FAQ usefulness

System Information

- **GET** `/api/v1/holidays` - Get holiday information
- **GET** `/api/v1/interestInfo` - Get interest rate information
- **GET** `/api/v1/common/services` - Get list of external services

File Upload Support

- **GET** `/api/v1/aws` - Get signed data for S3 uploads

Localization API Parameters

Parameter	Type	Required	Description
`msNames`	array[string]	Yes	List of service names requiring language resources

FAQ API Parameters

Parameter	Type	Required	Description
`serviceName`	string	Yes	Service name to query FAQs for
`faqId`	number	Yes	FAQ ID for review operations
`isUseful`	boolean	Yes	Review rating for FAQ

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L145-L149)

README.md145-149 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L145-L149)[

dbmigration/20200403_t_open_api.sql68-72](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L68-L72
(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L68-L72))

Client Management APIs

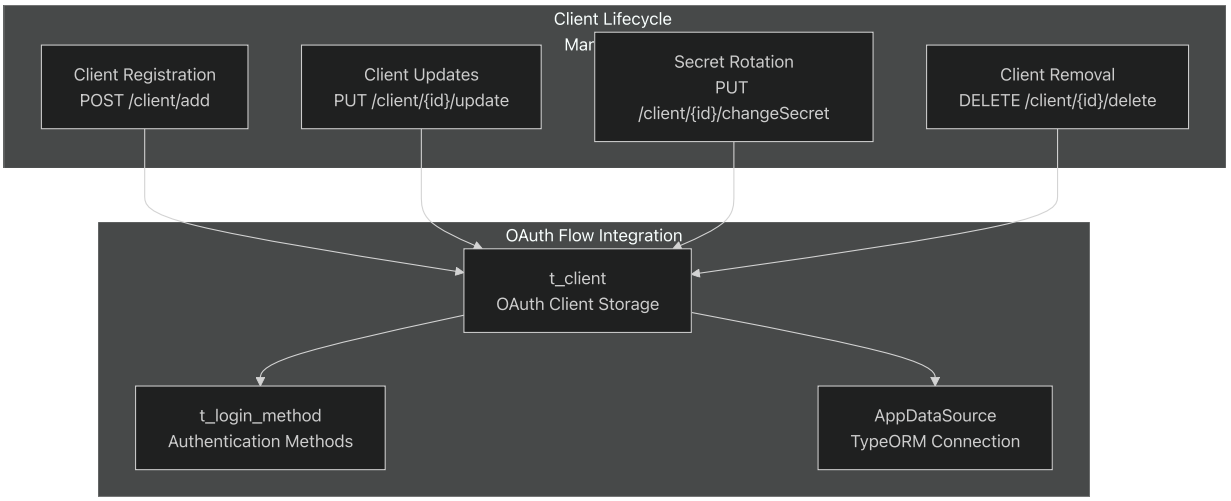
Client management APIs handle OAuth client registration, authentication, and lifecycle management for applications integrating with the TradeX platform.

Client Operations

- **GET** `/api/v1/client` - Get all clients
- **POST** `/api/v1/client/add` - Register new client
- **PUT** `/api/v1/client/{id}/changeSecret` - Change client secret
- **PUT** `/api/v1/client/{id}/update` - Update client configuration
- **DELETE** `/api/v1/client/{id}/delete` - Remove client

Client Registration Schema

```
{
  "name": "string",
  "description": "string",
  "redirectUris": ["string"],
  "scopes": ["string"],
  "grantTypes": ["authorization_code", "refresh_token"]
}
```



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L151-L156)

README.md151-156 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L151-L156)

AI Advisor APIs

AI Advisor APIs provide content management for trading insights, market analysis, and educational materials.

Content Retrieval

- **GET** `/api/v2/aiAdvisor/diary` - Query diaries list with filtering
- **GET** `/api/v2/aiAdvisor/diary/{id}` - Get specific diary content
- **GET** `/api/v2/aiAdvisor/news` - Query news list with filtering
- **GET** `/api/v2/aiAdvisor/news/{id}` - Get specific news content

Query Parameters

Parameter	Type	Description
`sort`	string	Sort specification (e.g., "id:desc")
`start`	number	Offset for pagination
`limit`	number	Number of records to fetch
`fromDate`	string	Start date filter (YYYYMMDD)
`toDate`	string	End date filter (YYYYMMDD)
`searchTitle`	string	Title search filter

Content Response Schema

Diary Response:

```
{
  "id": "integer",
  "slug": "string",
  "title": "string",
  "author": "string",
  "avatar": "string",
  "createdAt": "string",
  "description": "string",
  "content": "string"
}
```

News Response:

```
{
  "id": "integer",
  "title": "string",
  "description": "string",
  "slug": "string",
  "source": "string",
  "sourceUrl": "string",
  "avatar": "string",
  "createdAt": "string",
  "content": "string"
}
```

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4)

dbmigration/20200526_all_all_t_open_api_ai_advisor.sql1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4)

Request and Response Patterns

Standard Response Format

All API responses follow consistent patterns with appropriate HTTP status codes and JSON payloads.

Success Response (200):

```
{
  "data": {},
  "message": "Operation successful"
}
```

Error Response (4xx/5xx):

```
{
  "error": "Error description",
  "code": "ERROR_CODE"
}
```

Common HTTP Status Codes

Code	Meaning	Usage
200	OK	Successful operation
401	Unauthorized	Invalid or expired token
403	Forbidden	Insufficient permissions
404	Not Found	Resource not found
500	Internal Server Error	Server-side error

Pagination Parameters

For list endpoints supporting pagination:

Parameter	Type	Default	Description
`offset`	number	0	Starting record position
`fetchCount`	number	20	Number of records (20,40,60,80,100)
`baseDate`	string	today	Base date for time-based queries (YYYYMMDD)

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L49-L49)

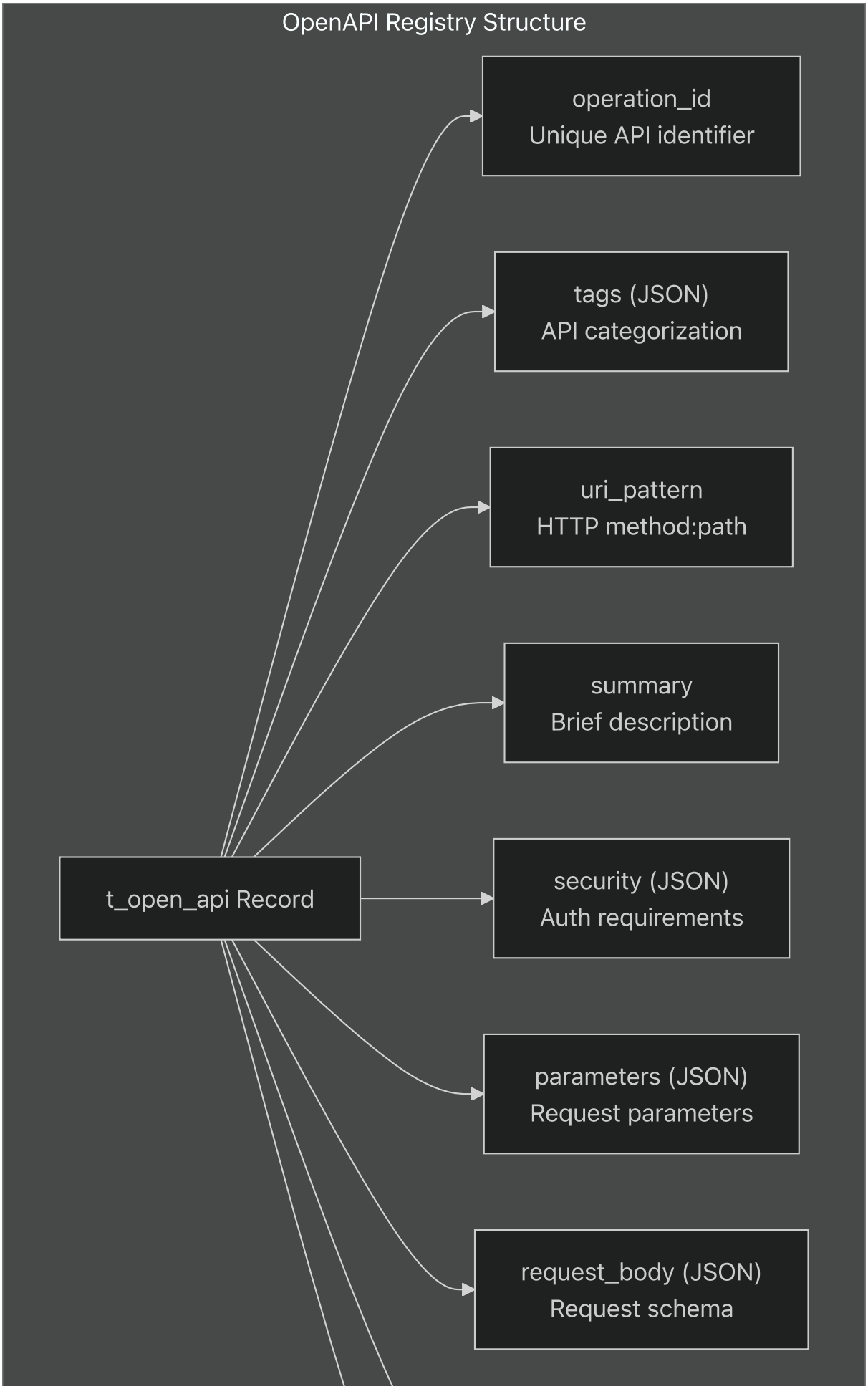
dbmigration/20200403_t_open_api.sql49 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L49-L49)[

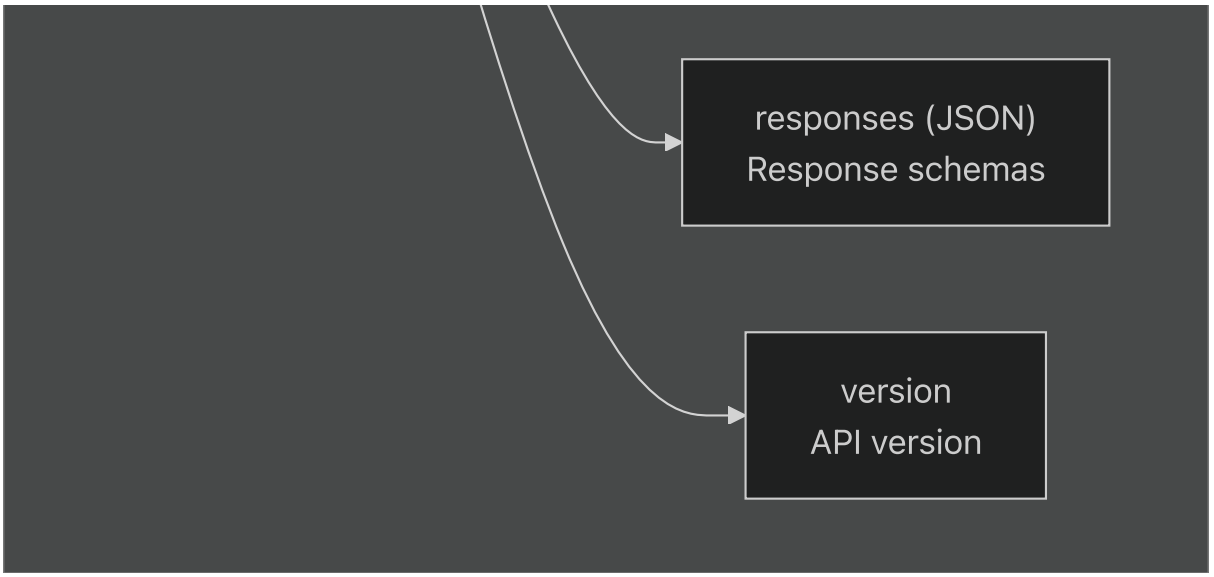
README.md129-157](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L129-L157 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L129-L157))

OpenAPI Integration

The service maintains a comprehensive OpenAPI registry in the `t_open_api` database table, storing complete API specifications for all endpoints across the TradeX platform.

OpenAPI Storage Schema


OpenAPI Registry Structure
operation_id
Unique API identifier
tags (JSON)
API categorization
uri_pattern
HTTP method:path
summary
Brief description
t_open_api Record
security (JSON)
Auth requirements
parameters (JSON)
Request parameters
request_body (JSON)
Request schema



Example OpenAPI Record

The system stores complete OpenAPI 3.0 specifications:

```
{
  "operation_id": "COMMON_LOCALE_RESOURCE",
  "tags": ["Configuration"],
  "uri_pattern": "get:/api/v1/locale",
  "summary": "Get latest version data of all Language Resource File",
  "security": [{"jwt": []}],
  "parameters": [
    {
      "in": "query",
      "name": "msNames",
      "schema": {"type": "array", "items": {"type": "string"}},
      "required": true,
      "description": "List of service names requiring language resources"
    }
  ],
  "responses": {
    "200": {
      "description": "List of language resource files",
      "content": {
        "application/json": {
          "schema": {
            "type": "array",
            "items": {"$ref": "#/components/schemas/LanguageResource"}
          }
        }
      }
    }
  }
}
```

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L25-L40)

dbmigration/20200403_t_open_api.sql25-40 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L25-L40)[

dbmigration/20200403_t_open_api.sql68-69](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L68-L69 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L68-L69))

System APIs

Relevant source files

- [

README.md](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md))

- [

  dbmigration/20200403_t_open_api.sql](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql))

This document covers the system management endpoints provided by the TradeX Configuration Service. These APIs are primarily used for internal system operations, client synchronization, and querying configuration metadata. For administrative configuration management, see Admin APIs. For public-facing endpoints, see Public APIs.

Purpose and Scope

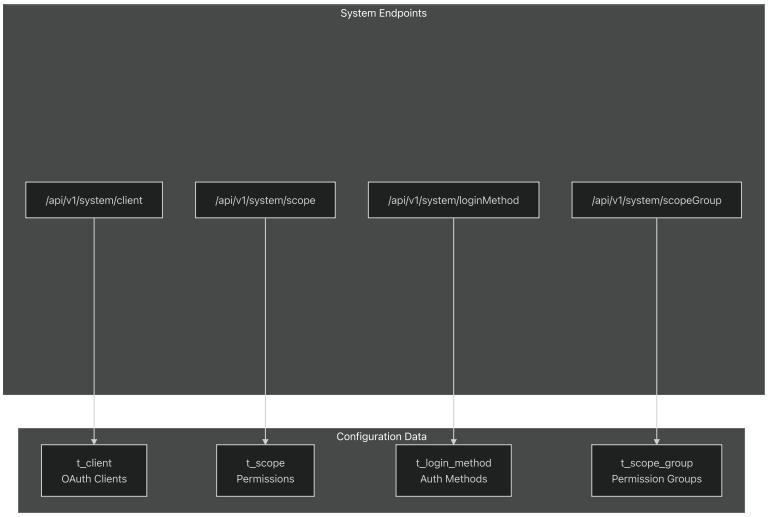
The System APIs provide endpoints for:

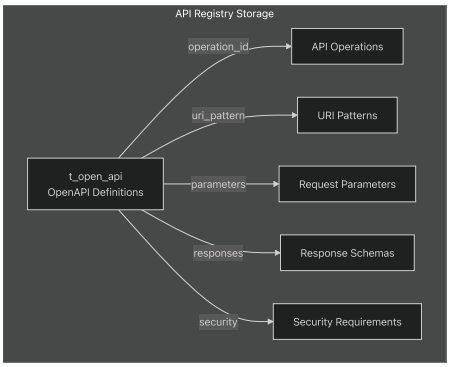
- Querying client configurations and authentication methods
- Retrieving scope and permission definitions
- System-to-system communication for configuration synchronization
- Internal service discovery and API registry management

These endpoints are typically used by other TradeX services to obtain configuration data and maintain consistency across the platform.

API Registry Architecture

The configuration service maintains a centralized API registry using the OpenAPI specification format. This registry stores detailed metadata about all available endpoints across the TradeX platform.





Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L25-L40)

dbmigration/20200403_t_open_api.sql25-40 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L25-L40)[

README.md132-136](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L132-L136 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L132-L136))

Client Management Endpoints

GET /api/v1/system/client

Retrieves client configuration data for OAuth client applications. This endpoint is used by authentication services to validate client credentials and obtain client-specific settings.

Request Parameters:

- Query parameters for filtering and pagination
- Support for incremental updates based on last modified timestamps

Response Format:

```
{
  "clients": [
    {
      "clientId": "string",
      "clientSecret": "string",
      "scopes": ["array of scope names"],
      "loginMethods": ["authentication methods"],
      "status": "ACTIVE|INACTIVE"
    }
  ],
  "lastUpdated": "timestamp"
}
```

GET /api/v1/system/loginMethod

Returns available authentication methods and their configurations. Used by client applications to determine supported login flows.

Response Data:

- Authentication method types (OAuth, JWT, etc.)
- Method-specific configuration parameters
- Enabled/disabled status per method

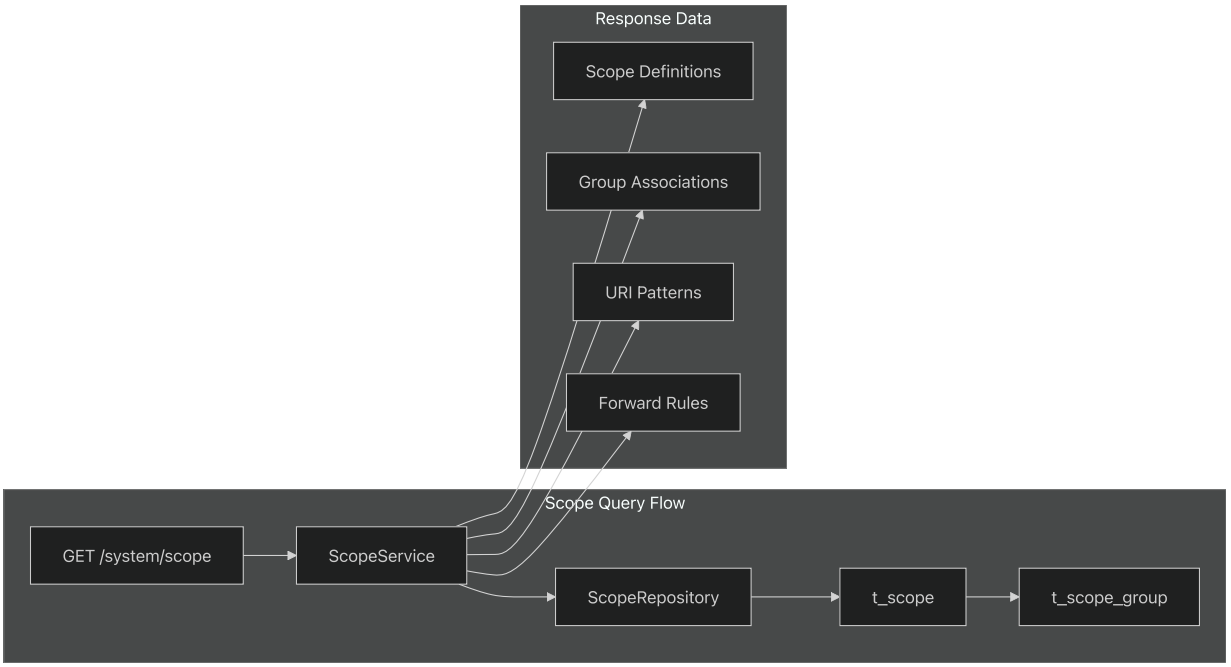
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L132-L136)

README.md132-136 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L132-L136)

Permission System Endpoints

GET /api/v1/system/scope

Retrieves scope definitions and permission mappings. This endpoint provides the complete permission model used across the TradeX platform.



Response Structure:

- `id`: Unique scope identifier
- `name`: Human-readable scope name
- `uriPattern`: URI pattern for endpoint matching
- `forwardType`: Routing configuration type
- `forwardData`: Routing destination data
- `scopeGroupIds`: Associated permission groups

GET /api/v1/system/scopeGroup

Returns scope group definitions and hierarchical permission structures.

Key Features:

- Hierarchical permission grouping
- Role-based access control mappings
- Group inheritance relationships

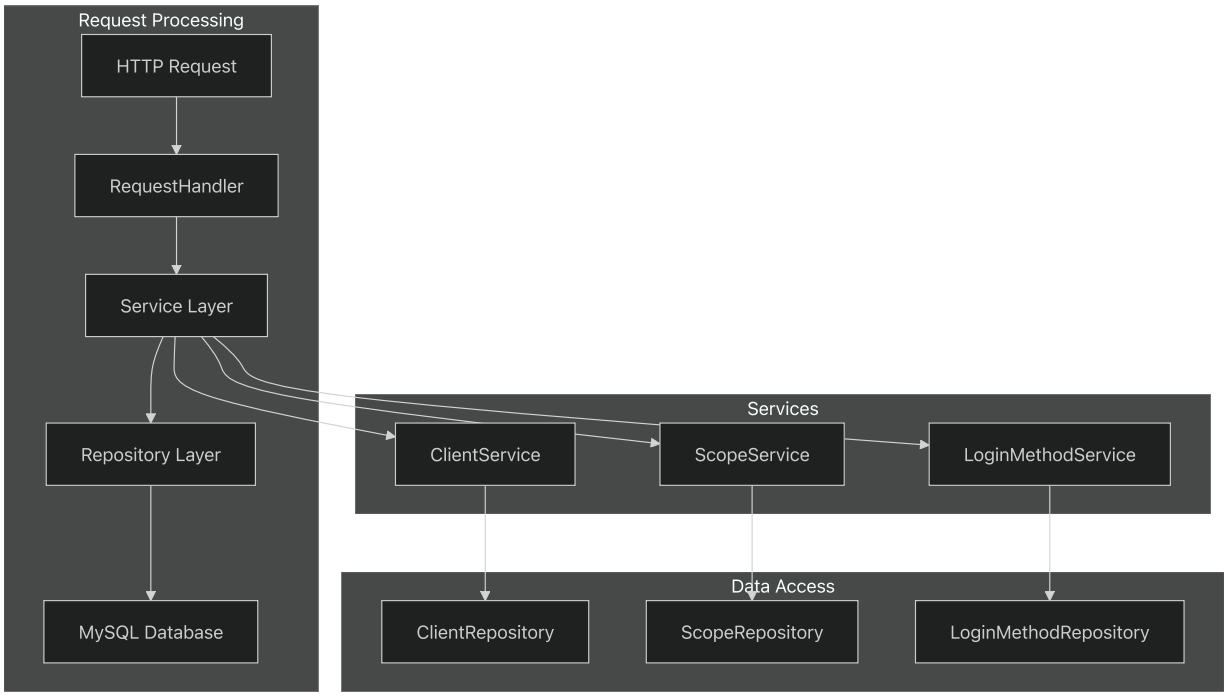
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L132-L136)

README.md132-136 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L132-L136)

Implementation Details

Request Handler Architecture

The system APIs are implemented using a centralized request handler pattern that routes requests to appropriate service layers.



Authentication and Security

System APIs use JWT-based authentication with scope-based authorization:

- **Authentication**: JWT tokens validated against client credentials
- **Authorization**: Scope-based permission checking
- **Rate Limiting**: Applied per client ID
- **Audit Logging**: All system API access logged for security monitoring

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L31-L31)

dbmigration/20200403_t_open_api.sql31 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L31-L31)

Data Models and Schemas

OpenAPI Registry Schema

The `t_open_api` table stores comprehensive API metadata:

Field	Type	Description
`id`	int(11)	Primary key identifier
`operation_id`	varchar(255)	Unique operation identifier
`tags`	json	API grouping tags
`uri_pattern`	varchar(255)	URI matching pattern
`summary`	mediumtext	Operation description
`security`	json	Security requirements
`parameters`	json	Request parameter schemas
`request_body`	json	Request body specifications
`responses`	json	Response format definitions
`version`	varchar(45)	API version information

System Configuration Data

The system APIs primarily serve data from these core tables:

- `t_client`: OAuth client configurations
- `t_scope`: Permission scope definitions
- `t_scope_group`: Permission group hierarchies
- `t_login_method`: Authentication method configurations

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L25-L40)

dbmigration/20200403_t_open_api.sql25-40 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L25-L40)

Error Handling and Status Codes

System APIs return standardized HTTP status codes:

- **200 OK**: Successful data retrieval

- **401 Unauthorized**: Invalid or expired authentication
- **403 Forbidden**: Insufficient permissions
- **404 Not Found**: Resource does not exist
- **429 Too Many Requests**: Rate limit exceeded
- **500 Internal Server Error**: System error occurred

Error responses include detailed error codes and messages for debugging and monitoring purposes.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L1-L265)

README.md1-265 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L1-L265)[

dbmigration/20200403_t_open_api.sql1-468](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L1-L468
(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L1-L468))

Admin APIs

Relevant source files

- [

  README.md](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md
  (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md))

- [

  dbmigration/20200403_t_open_api.sql](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql
  (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql))

This document describes the administrative endpoints provided by the TradeX Configuration Service for managing system configuration, permissions, localization, and content. These APIs are designed for backend administrators and internal services to manage the platform's operational parameters.

For system-level configuration queries, see System APIs. For public-facing configuration endpoints, see Public APIs. For AI Advisor specific administration, see AI Advisor APIs.

Overview

The Admin APIs provide comprehensive administrative functionality across five main areas:

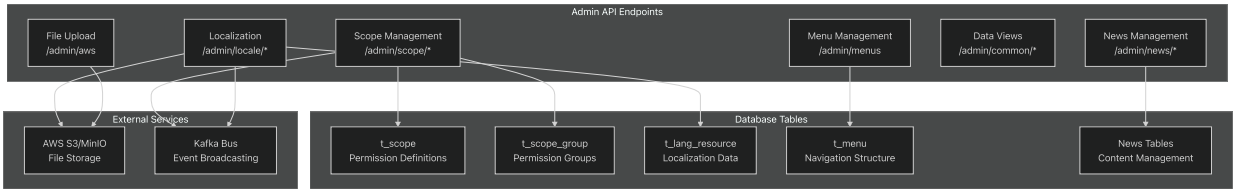
- **Scope Management**: Permission and access control administration
- **Localization Management**: Multi-language resource administration
- **Menu Management**: Dynamic navigation system configuration

- **Content Management**: News and report administration
- **System Utilities**: Data views and file upload management

All admin endpoints require appropriate authentication and administrative privileges.

API Categories

Admin API Architecture

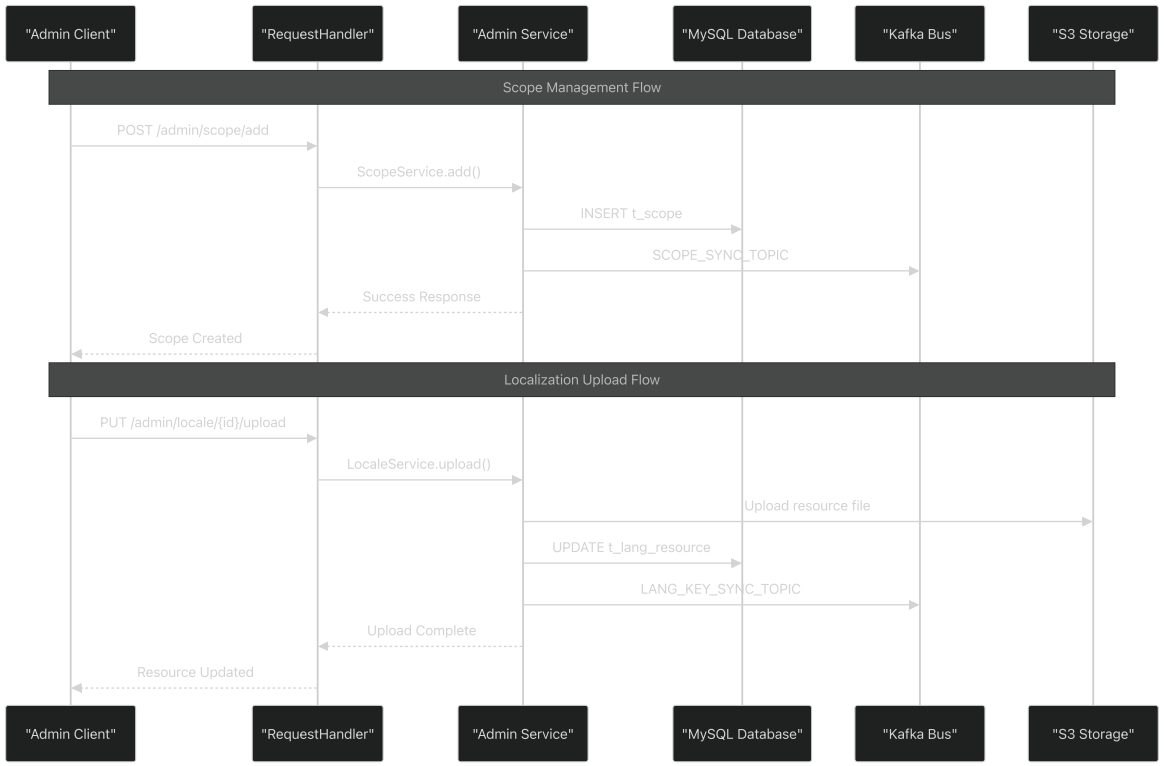


Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L129-L156)

README.md129-156 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L129-L156)[

dbmigration/20200403_t_open_api.sql128-147](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L128-L147
(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L128-L147))

Request Processing Flow



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L128-L147)

dbmigration/20200403_t_open_api.sql128-147 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L128-L147)

Scope Management APIs

The scope management endpoints provide CRUD operations for the permission system that controls access to TradeX platform features.

Core Endpoints

Method	Endpoint	Description
`GET`	`/api/v1/admin/scope`	List all permission scopes with filtering
`POST`	`/api/v1/admin/scope`	Create new permission scope
`PUT`	`/api/v1/admin/scope/{scopeId}`	Update existing scope
`DELETE`	`/api/v1/admin/scope/{scopeId}`	Delete permission scope

Request/Response Structure

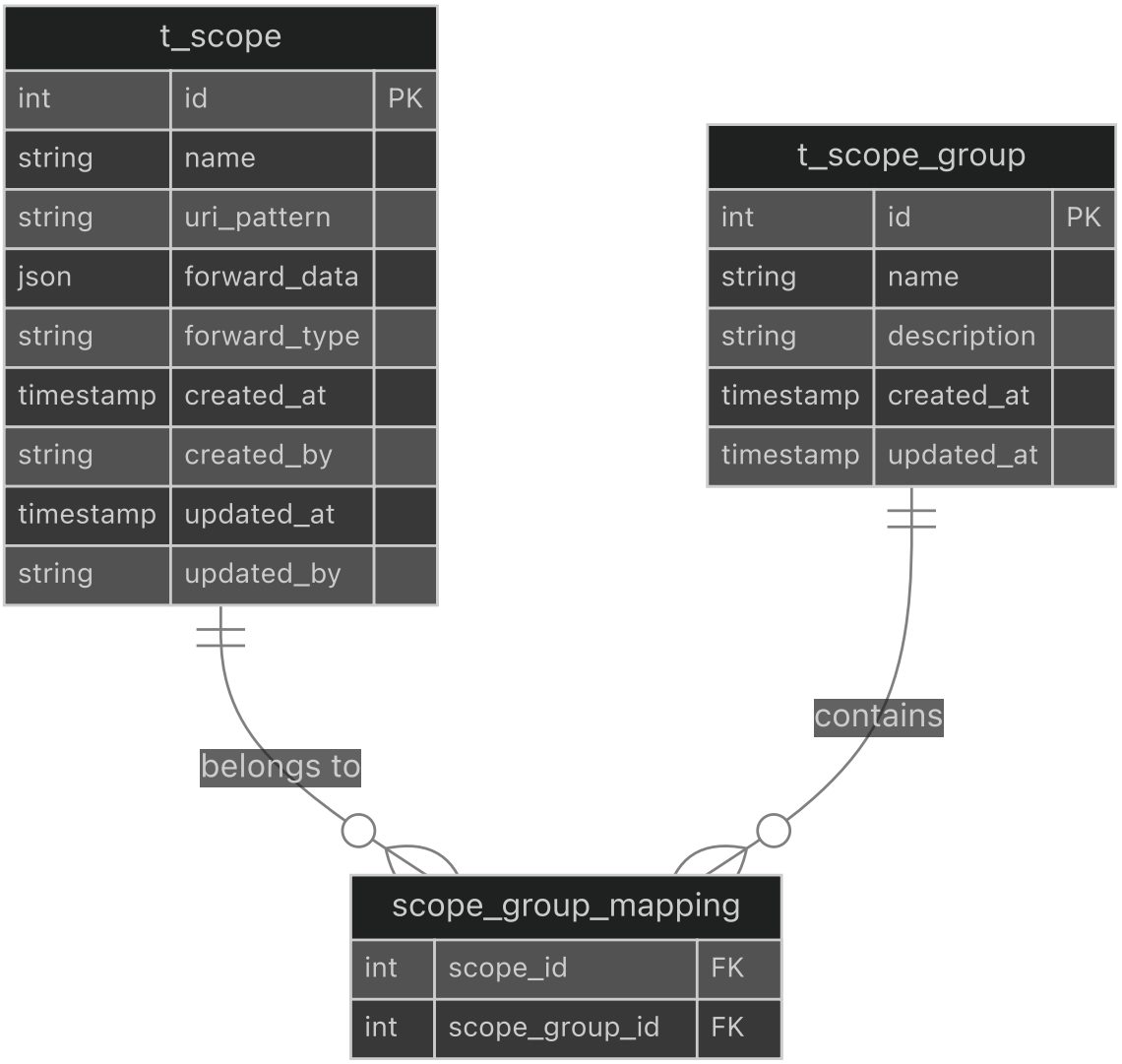
GET /api/v1/admin/scope supports filtering by:

- `name`: Scope name pattern matching
- `scopeGroupId`: Filter by permission group
- `uriPattern`: URI pattern matching
- `forwardType`: Forward mechanism type
- `lastSequence`: Pagination cursor
- `fetchCount`: Results per page

POST /api/v1/admin/scope requires:

```
{
  "name": "string",
  "uriPattern": "string",
  "forwardData": "object",
  "forwardType": "string",
  "scopeGroupIds": ["number"]
}
```

Scope Data Model



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L140-L145)

dbmigration/20200403_t_open_api.sql140-145 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L140-L145)

Localization Management APIs

These endpoints manage multi-language resources and content translation across the TradeX platform.

Resource Management

Method	Endpoint	Description
`GET`	`/api/v1/admin/locale/resource`	Get all language resource namespaces
`GET`	`/api/v1/admin/locale/{namespaceId}/key`	Get translation keys for namespace
`POST`	`/api/v1/admin/locale/{namespaceId}/key`	Add new translation key
`PUT`	`/api/v1/admin/locale/{keyId}/{lang}`	Update translation value
`DELETE`	`/api/v1/admin/locale/{namespaceId}/key/{keyId}`	Delete translation key
`PUT`	`/api/v1/admin/locale/{namespaceId}/upload`	Upload resource to AWS

Translation Key Management

GET /api/v1/admin/locale/{namespaceId}/key supports:

- `keyword`: Search in translation keys
- `lastKey`: Pagination cursor
- `fetchCount`: Results per page

POST /api/v1/admin/locale/{namespaceId}/key requires:

```
{
  "key": "string"
}
```

PUT /api/v1/admin/locale/{keyId}/{lang} requires:

```
{
  "value": "string"
}
```

Resource Upload Process

PUT /api/v1/admin/locale/{namespaceId}/upload initiates:

```
{
  "lang": "string",
  "version": "string"
}
```

This triggers:

1. Resource file generation from database
2. Upload to S3/MinIO storage
3. Version update in `t_lang_resource`
4. Kafka event broadcast for cache invalidation

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L134-L139)

dbmigration/20200403_t_open_api.sql134-139 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L134-L139)

Menu Management APIs

Manages the dynamic navigation system with role-based access control.

Menu Retrieval

Method	Endpoint	Description
`GET`	`/api/v1/admin/menus`	Get menu structure by role IDs

GET /api/v1/admin/menus accepts:

- `roleIds`: Array of role identifiers for permission filtering

Response includes hierarchical menu structure with:

- Menu items and navigation paths
- Permission-based visibility
- Localized menu labels
- Menu ordering and grouping

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L140-L140)

README.md140 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L140-L140)

Content Management APIs

Provides administration for news articles and analytical reports.

News Report Management

Method	Endpoint	Description
`GET`	`/api/v1/admin/news/report`	Filter and search reports
`POST`	`/api/v1/admin/news/report`	Create new report
`PUT`	`/api/v1/admin/news/report/{reportId}`	Update existing report
`DELETE`	`/api/v1/admin/news/report/{reportId}`	Delete report

Report Filtering

GET /api/v1/admin/news/report supports:

- `lang`: Language filter
- `sourceId`: Content source filter
- `symbolCode`: Stock symbol association
- `keyword`: Text search in titles/content
- `lastSequence`: Pagination cursor
- `fetchCount`: Results per page

Report Data Structure

POST /api/v1/admin/news/report requires:

```
{
  "sourceId": "string",
  "title": "string",
  "link": "string",
  "symbolList": ["string"]
}
```

Reports automatically include:

- Publication timestamp
- Language detection
- Image URL extraction
- Source metadata

- Stock symbol associations

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L129-L133)

dbmigration/20200403_t_open_api.sql129-133 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L129-L133)

System Utility APIs

Data View Access

Method	Endpoint	Description
`GET`	`/api/v1/admin/common/dataview`	Generic data retrieval by view code

GET /api/v1/admin/common/dataview requires:

- `code` : View identifier
- `fetchCount` : Number of records to return
- `lastSequence` : Pagination cursor

File Upload Management

Method	Endpoint	Description
`GET`	`/api/v1/admin/aws`	Get signed upload URL for internal files

GET /api/v1/admin/aws requires:

- `key` : Target file path in storage
- `serviceName` : Service identifier for storage organization

Returns AWS S3 pre-signed POST policy for secure file uploads including:

- Upload URL and credentials
- Policy documents and signatures
- Expiration timestamps
- Storage bucket information

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L128-L131)

dbmigration/20200403_t_open_api.sql128-131 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L128-L131)

Authentication and Security

All admin endpoints require:

- Valid JWT authentication token
- Administrative role permissions
- Service-specific scope authorizations

Security features include:

- Request rate limiting
- Input validation and sanitization
- Audit logging for all modifications
- Role-based access control integration

Error Handling

Admin APIs return standardized error responses:

Status Code	Description
`200`	Successful operation
`401`	Authentication required or invalid
`403`	Insufficient permissions
`404`	Resource not found
`422`	Validation errors
`500`	Internal server error

Error responses include detailed error codes and messages for debugging and user feedback.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L129-L156)

README.md129-156 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L129-L156)[

dbmigration/20200403_t_open_api.sql128-147](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L128-L147

(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L128-L147))

Public APIs

Relevant source files

- [

  README.md](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md
  (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md))

- [

  dbmigration/20200403_t_open_api.sql](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql
  (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql))

This document covers the public-facing API endpoints provided by the TradeX Configuration Service. These endpoints are designed for client applications to retrieve configuration data, localization resources, and informational content without requiring special administrative privileges.

The public APIs handle localization resources, FAQ content, service metadata, and file upload functionality. For administrative configuration management, see Admin APIs. For system-level integration endpoints, see System APIs.

Overview

The public APIs serve four primary functions:

Function	Endpoints	Purpose
Localization	`/api/v1/locale`	Multi-language resource management
FAQ Management	`/api/v1/faq/*`	Service-specific help content
Service Discovery	`/api/v1/common/services`	External service integration metadata
File Operations	`/api/v1/aws`	Signed upload URLs for client files

All public endpoints use JWT-based authentication and return JSON responses with standardized error handling.

Localization API

Get Language Resources

```
GET /api/v1/locale
```

Retrieves the latest version of language resource files for specified microservices.

Parameters:

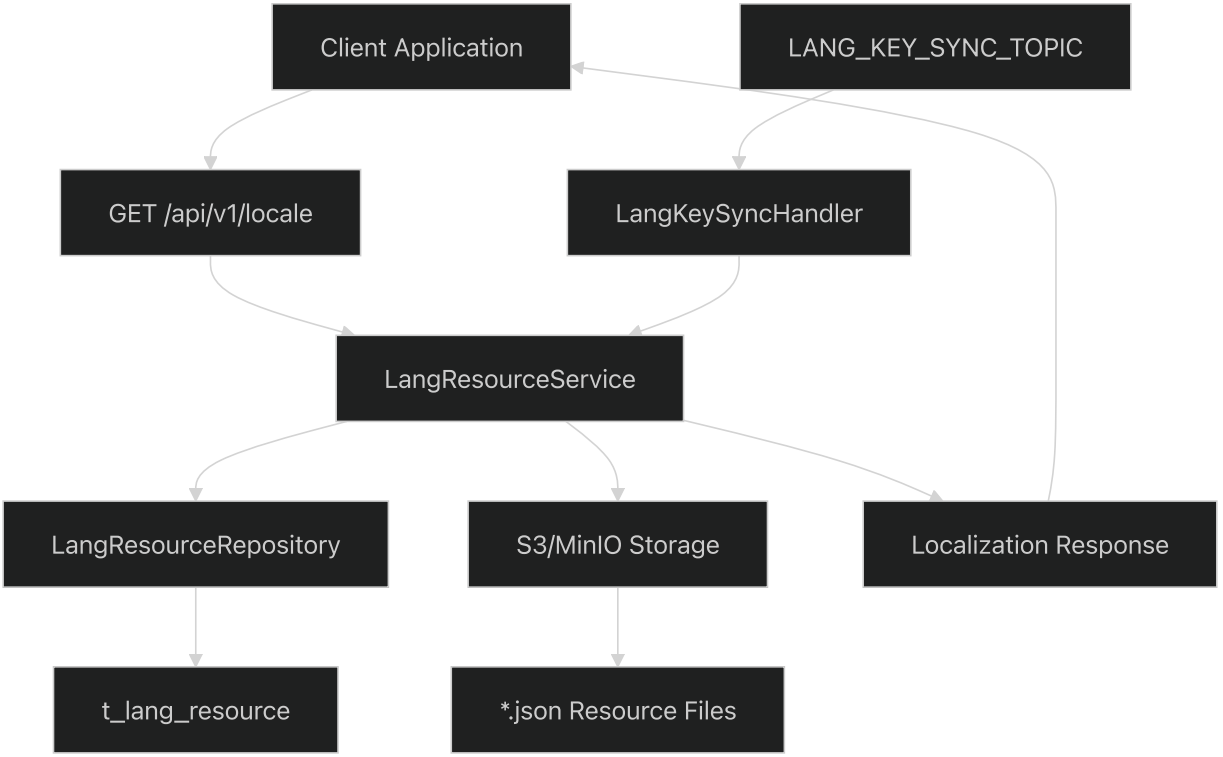
- `msNames` (required): Array of service names requiring localization resources

Response Structure:

```
[
  {
    "lang": "en",
    "files": [
      {
        "url": "https://s3.bucket/path/to/resource.json",
        "namespace": "common"
      }
    ],
    "msName": "tradex-mobile",
    "latestVersion": "1.2.3"
  }
]
```

The localization system supports namespace-based resource organization, allowing services to maintain separate translation contexts for different application modules.

Localization Data Flow



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L68-L68)

dbmigration/20200403_t_open_api.sql68 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L68-L68)[

README.md146-149](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L146-L149 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L146-L149))

FAQ Management API

Get Service FAQs

```
GET /api/v1/faq/{msName}
```

Retrieves categorized FAQ content for a specific microservice.

Parameters:

- `msName` (path, required): The service name to query FAQs for

Response Structure:

```
[
  {
    "name": "Getting Started",
    "faqs": [
      {
        "question": "How do I create an account?",
        "answer": "To create an account, click the Sign Up button..."
      }
    ]
  }
]
```

Submit FAQ Review

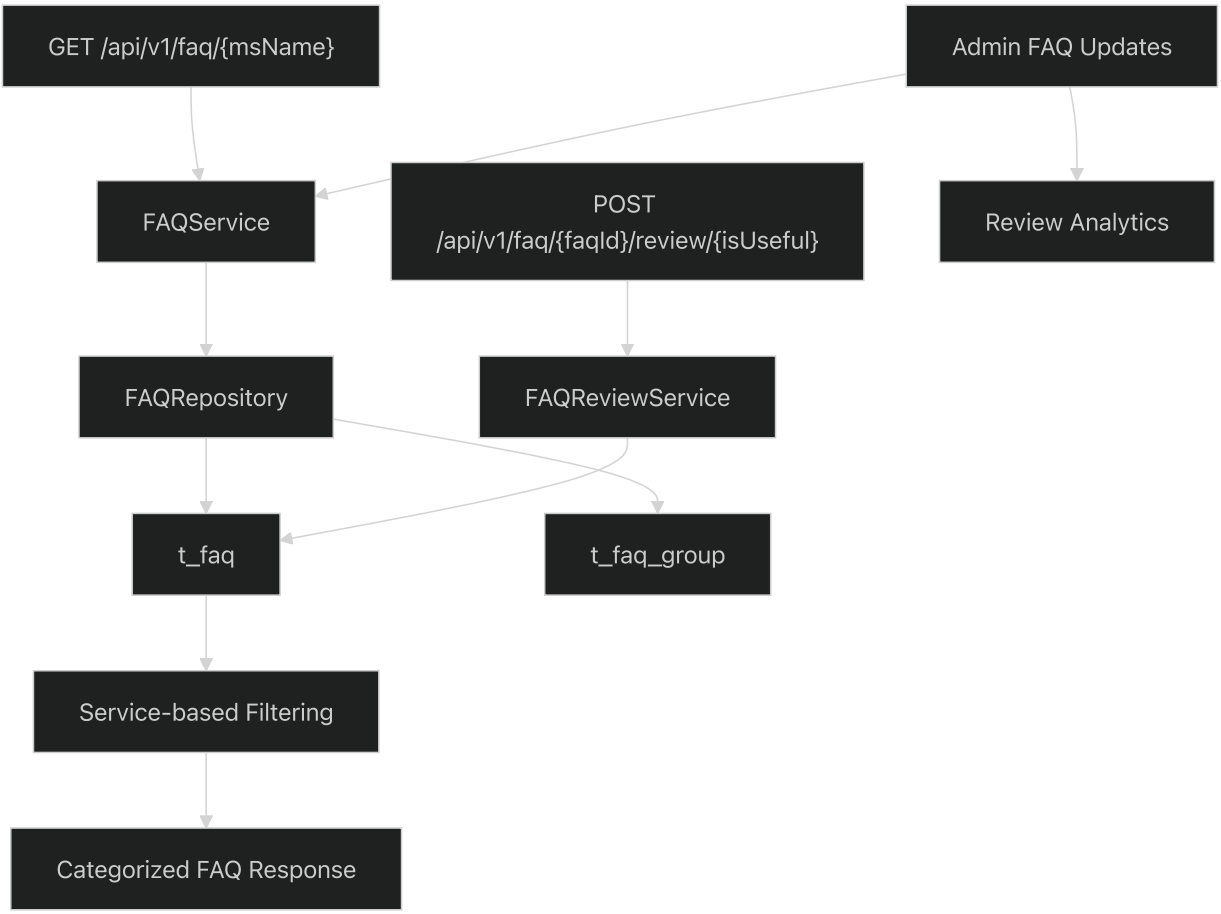
```
POST /api/v1/faq/{faqId}/review/{isUseful}
```

Allows users to rate the helpfulness of FAQ entries for content improvement.

Parameters:

- `faqId` (path, required): Unique identifier of the FAQ entry
- `isUseful` (path, required): Boolean indicating if the FAQ was helpful

FAQ System Architecture



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L71-L72)

dbmigration/20200403_t_open_api.sql71-72 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L71-L72)[

README.md147-149](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L147-L149 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L147-L149))

Service Discovery API

Get External Services

```
GET /api/v1/common/services
```

Returns metadata for external services integrated with the TradeX platform.

Response Structure:

```
[
  {
    "serviceCode": "MARKET_DATA_PROVIDER",
    "serviceName": "Vietnam Stock Exchange",
    "logoUrl": "https://s3.bucket/logos/vse.png",
    "supportEmail": "support@vse.vn",
    "supportPhone": "+84-xxx-xxx-xxxx"
  }
]
```

This endpoint supports service discovery for client applications that need to display information about integrated third-party services or contact support resources.

File Upload API

Get Signed Upload URL

```
GET /api/v1/aws
```

Generates pre-signed URLs for secure file uploads to cloud storage.

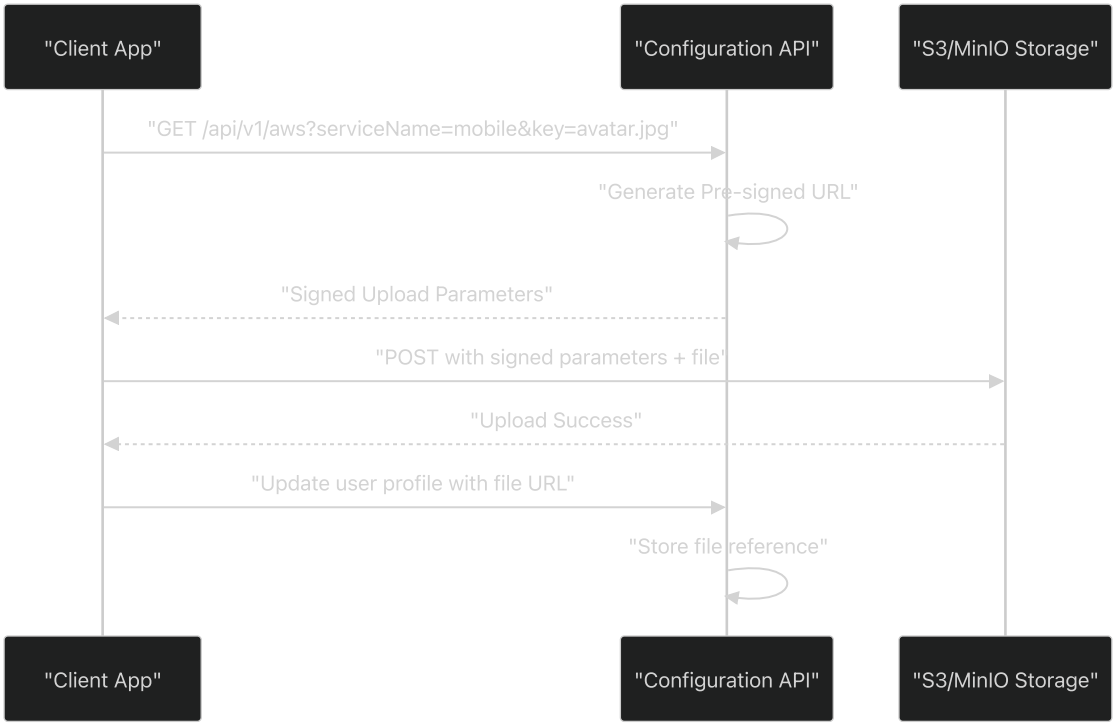
Parameters:

- `serviceName` (required): Service context for file organization
- `key` (required): File path and name for storage

Response Structure:

```
{
  "key": "mobile-app/user-avatars/12345.jpg",
  "bucket": "tradex-uploads",
  "Policy": "eyJ...",
  "X-Amz-Date": "20241201T120000Z",
  "X-Amz-Algorithm": "AWS4-HMAC-SHA256",
  "X-Amz-Signature": "abc123...",
  "X-Amz-Credential": "AKIA.../20241201/us-east-1/s3/aws4_request"
}
```

File Upload Flow



The signed URL approach ensures secure file uploads without exposing storage credentials to client applications.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L70-L70)

dbmigration/20200403_t_open_api.sql70 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L70-L70)[

README.md149](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L149-L149 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L149-L149))

Authentication and Access Control

All public APIs require JWT authentication through the `Authorization` header:

```
Authorization: Bearer <jwt-token>
```

The authentication flow integrates with the OAuth client system managed by the configuration service itself. Public endpoints have minimal scope requirements but still validate token authenticity and expiration.

API Security Model

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L68-L72)

dbmigration/20200403_t_open_api.sql68-72 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L68-L72)[

README.md131-150](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L131-L150 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/README.md#L131-L150))

Error Handling

Public APIs return standardized error responses with appropriate HTTP status codes:

Status Code	Scenario	Response Format
200	Success	JSON data as documented
401	Invalid/expired token	`{"error": "Your access token is invalid or expired"}`
400	Missing required parameters	`{"error": "Missing required parameter: msNames"}`
404	Resource not found	`{"error": "Service not found: invalid-service"}`
500	Internal server error	`{"error": "Internal server error"}`

The error handling follows the OpenAPI specifications defined in the system and provides consistent client experience across all endpoints.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L68-L72)

dbmigration/20200403_t_open_api.sql68-72 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200403_t_open_api.sql#L68-L72)

AI Advisor APIs

Relevant source files

- [

  dbmigration/20200526_all_all_t_open_api_ai_advisor.sql](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql))

- [

dbmigration/20200526_all_all_t_scope_ai_advisor.sql](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql))

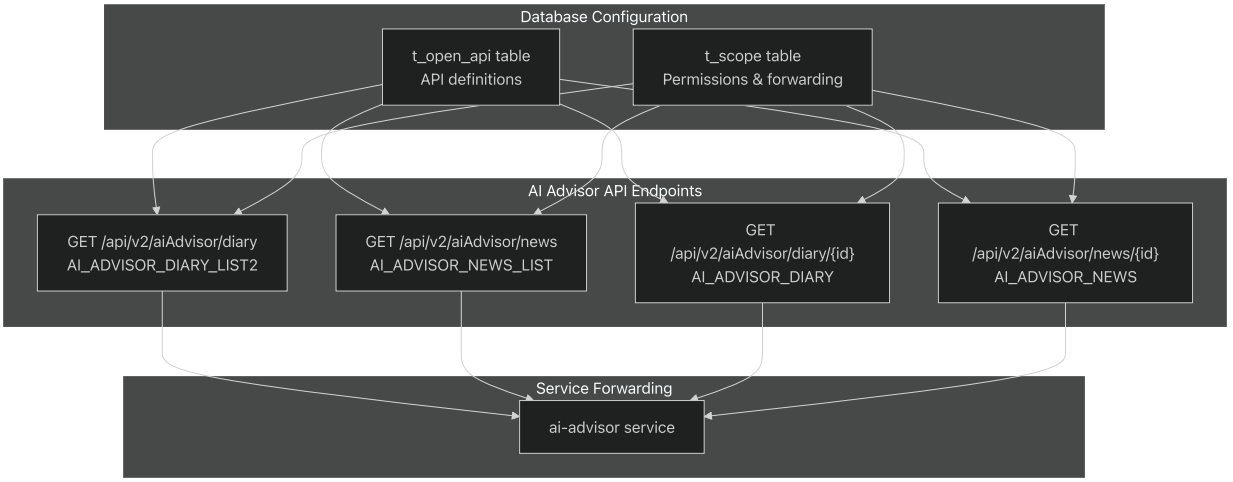
Purpose and Scope

This document covers the AI Advisor API endpoints exposed by the TradeX Configuration Service. These APIs provide access to diary and news content managed by the AI Advisor system. The endpoints documented here are specifically related to content retrieval and are distinct from the administrative functions covered in Admin APIs and system management functions in System APIs.

The AI Advisor APIs serve as a proxy layer, forwarding requests to the dedicated `ai-advisor` service while providing centralized authentication, authorization, and API registry management through the configuration service.

API Endpoint Overview

The AI Advisor APIs consist of four main endpoints that provide access to diary and news content:



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4)

dbmigration/20200526_all_all_t_open_api_ai_advisor.sql1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4)[

dbmigration/20200526_all_all_t_scope_ai_advisor.sql1-4](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L1-L4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L1-L4))

Diary Management APIs

List Diaries

Endpoint: `GET /api/v2/aiAdvisor/diary`
Operation ID: `AI_ADVISOR_DIARY_LIST2`
Security: JWT Authentication Required

Query Parameters

Parameter	Type	Required	Description
`sort`	string	No	Sort specification (e.g., 'id:desc')
`start`	number	No	Start record offset (e.g., 0)
`limit`	number	No	Limit fetch count (e.g., 10)
`fromDate`	string	No	From date filter (YYYYMMDD format)
`toDate`	string	No	To date filter (YYYYMMDD format)
`searchTitle`	string	No	Title search filter

Response Schema

The endpoint returns an array of diary objects with the following structure:

Field	Type	Description
`id`	integer	Unique identifier of the diary
`slug`	string	URL-friendly identifier
`title`	string	Diary title
`author`	string	Author name
`avatar`	string	Author avatar URL
`createdAt`	string	Creation timestamp (YYYYMMDDhhmmss)
`description`	string	Diary description/summary

Get Single Diary

Endpoint: `GET /api/v2/aiAdvisor/diary/{id}`
Operation ID: `AI_ADVISOR_DIARY`
Security: JWT Authentication Required

Path Parameters

Parameter	Type	Required	Description
`id`	number	Yes	Diary identifier

Response Schema

Field	Type	Description
`id`	integer	Unique identifier of the diary
`title`	string	Diary title
`content`	string	Full diary content
`description`	string	Diary description/summary
`slug`	string	URL-friendly identifier
`avatar`	string	Author avatar URL
`author`	string	Author name
`createdAt`	string	Creation timestamp (YYYYMMDDhhmmss)

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L1)

dbmigration/20200526_all_all_t_open_api_ai_advisor.sql1 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L1)[

dbmigration/20200526_all_all_t_open_api_ai_advisor.sql3](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L3-L3 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L3-L3))

News Management APIs

List News

Endpoint: `GET /api/v2/aiAdvisor/news`
Operation ID: `AI_ADVISOR_NEWS_LIST`
Security: JWT Authentication Required

Query Parameters

Parameter	Type	Required	Description
`sort`	number	No	Sort specification
`start`	number	No	Start record offset (e.g., 0)
`limit`	number	No	Limit fetch count (e.g., 10)
`fromDate`	string	No	From date filter (YYYYMMDD format)
`toDate`	string	No	To date filter (YYYYMMDD format)
`searchTitle`	string	No	Title search filter

Response Schema

The endpoint returns an array of news objects with the following structure:

Field	Type	Description
`id`	integer	Unique identifier of the news
`title`	string	News title
`description`	string	News description/summary
`slug`	string	URL-friendly identifier
`source`	string	News source name
`sourceUrl`	string	Original source URL
`avatar`	string	News image/avatar URL
`createdAt`	string	Creation timestamp (YYYYMMDDhhmmss)

Get Single News

Endpoint: `GET /api/v2/aiAdvisor/news/{id}`
Operation ID: `AI_ADVISOR_NEWS`
Security: JWT Authentication Required

Path Parameters

Parameter	Type	Required	Description
`id`	number	Yes	News identifier

Response Schema

Field	Type	Description
`id`	integer	Unique identifier of the news
`title`	string	News title
`content`	string	Full news content
`description`	string	News description/summary
`slug`	string	URL-friendly identifier
`source`	string	News source name
`sourceUrl`	string	Original source URL
`avatar`	string	News image/avatar URL
`createdAt`	string	Creation timestamp (YYYYMMDDhhmmss)

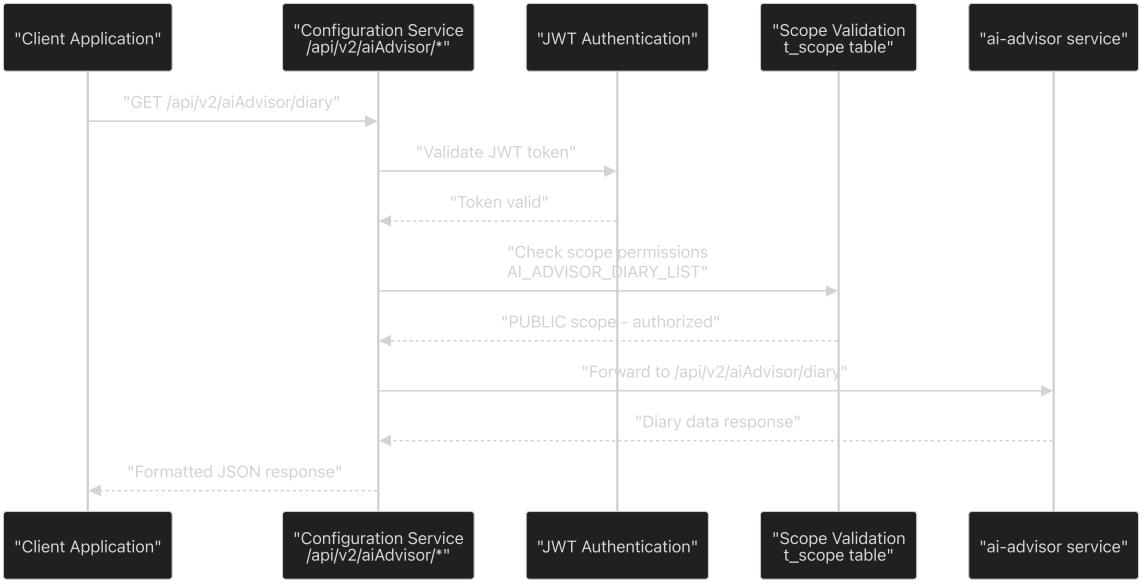
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L2-L2)

dbmigration/20200526_all_all_t_open_api_ai_advisor.sql2 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L2-L2)[

dbmigration/20200526_all_all_t_open_api_ai_advisor.sql4](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L4-L4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L4-L4))

Service Forwarding Architecture

The AI Advisor APIs utilize a service forwarding pattern where the Configuration Service acts as a proxy to the dedicated AI Advisor service:



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L1-L4)

dbmigration/20200526_all_all_t_scope_ai_advisor.sql1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L1-L4)

Permission Configuration

All AI Advisor APIs are configured with public access within the scope system:

Scope Name	URI Pattern	Forward Type	Target Service
`AI_ADVISOR_DIARY_LIST`	`get:/api/v2/aiAdvisor/diary`	CONNECTION	ai-advisor
`AI_ADVISOR_NEWS_LIST`	`get:/api/v2/aiAdvisor/news`	CONNECTION	ai-advisor
`AI_ADVISOR_DIARY`	`get:/api/v2/aiAdvisor/diary/{id}`	CONNECTION	ai-advisor
`AI_ADVISOR_NEWS`	`get:/api/v2/aiAdvisor/news/{id}`	CONNECTION	ai-advisor

All endpoints are mapped to the `PUBLIC` scope group, making them accessible to any authenticated user with a valid JWT token.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L8-L11)

dbmigration/20200526_all_all_t_scope_ai_advisor.sql8-11 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L8-L11)

Database Schema Integration

The AI Advisor APIs are integrated into the configuration system through two main database tables:

OpenAPI Registry (`t_open_api`)

Each endpoint is registered with complete OpenAPI specifications including:

- Operation ID and tags
- URI patterns and HTTP methods
- Parameter schemas and validation rules
- Response schemas and content types
- Security requirements

Scope Management (`t_scope`)

Each endpoint has corresponding scope entries that define:

- Permission requirements
- Service forwarding configuration
- Target service and URI mapping
- Forward type specification (`CONNECTION`)

The scope-to-group mapping ensures all AI Advisor endpoints are publicly accessible while maintaining the centralized permission management system.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4)

dbmigration/20200526_all_all_t_open_api_ai_advisor.sql1-4 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_open_api_ai_advisor.sql#L1-L4)[

dbmigration/20200526_all_all_t_scope_ai_advisor.sql1-11](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L1-L11 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dbmigration/20200526_all_all_t_scope_ai_advisor.sql#L1-L11))

Configuration

Overview

Relevant source files

- [

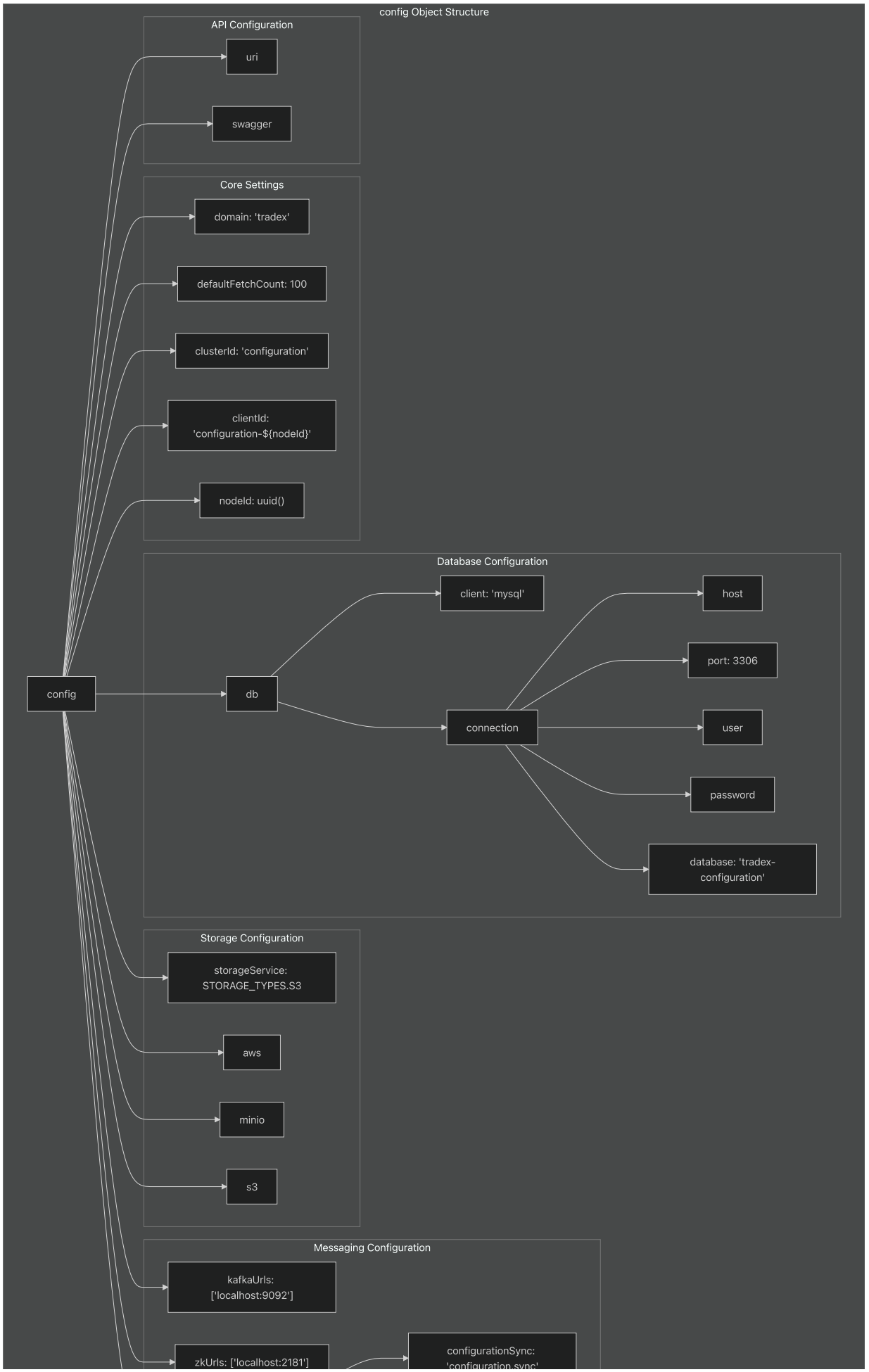
  src/config.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts))

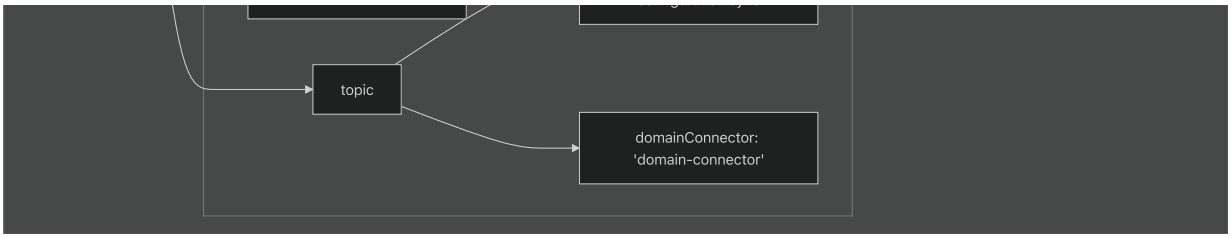
This document covers the configuration management system of the TradeX Configuration Service, including the central configuration object structure, external configuration loading mechanisms, and the various configuration categories for database, storage, messaging, and API services. For environment-specific configuration and runtime parameter management, see Environment Setup.

Configuration Architecture

The configuration system is built around a central configuration object defined in `src/config.ts` that provides structured settings for all service components. The system supports both default configuration values and external configuration overrides through a dynamic loading mechanism.

Configuration Object Structure

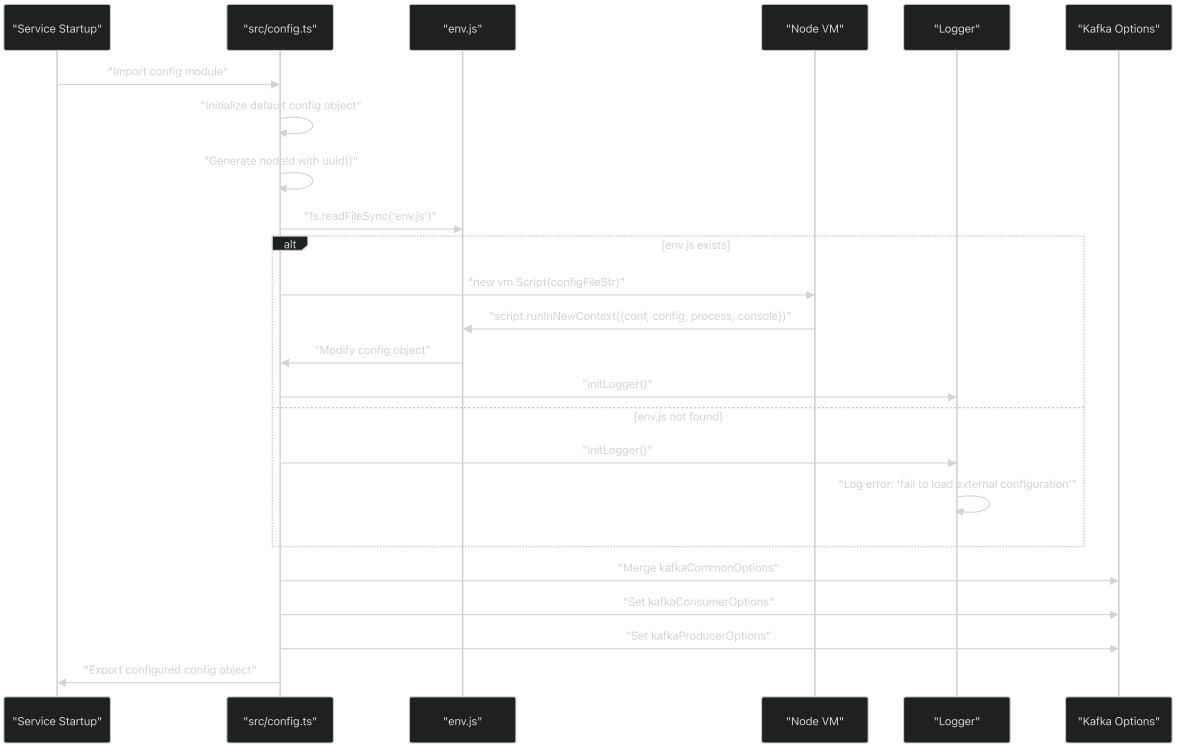

config Object Structure
API Configuration
uri
swagger
Core Settings
domain: 'tradex'
defaultFetchCount: 100
clusterId: 'configuration'
clientId: 'configuration-${nodeId}'
nodeId: uuid()
Database Configuration
client: 'mysql'
host
port: 3306
config
db
connection
user
password
database: 'tradex-configuration'
Storage Configuration
storageService: STORAGE_TYPES.S3
aws
minio
s3
Messaging Configuration
kafkaUrls: ['localhost:9092']
zkUrls: ['localhost:2181']
configurationSync: 'configuration.sync'



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L11-L273)

src/config.ts11-273 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L11-L273)

Configuration Loading and Initialization Process



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L284-L308)

src/config.ts284-308 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L284-L308)

Database Configuration

The database configuration section defines MySQL connection parameters and is structured under the `db` property of the main configuration object.

Property	Value	Description
`client`	`"mysql"`	Database client type
`connection.host`	AWS RDS endpoint	Database server hostname
`connection.port`	`3306`	MySQL connection port
`connection.user`	`"techxdev"`	Database username
`connection.database`	`"tradex-configuration"`	Target database name

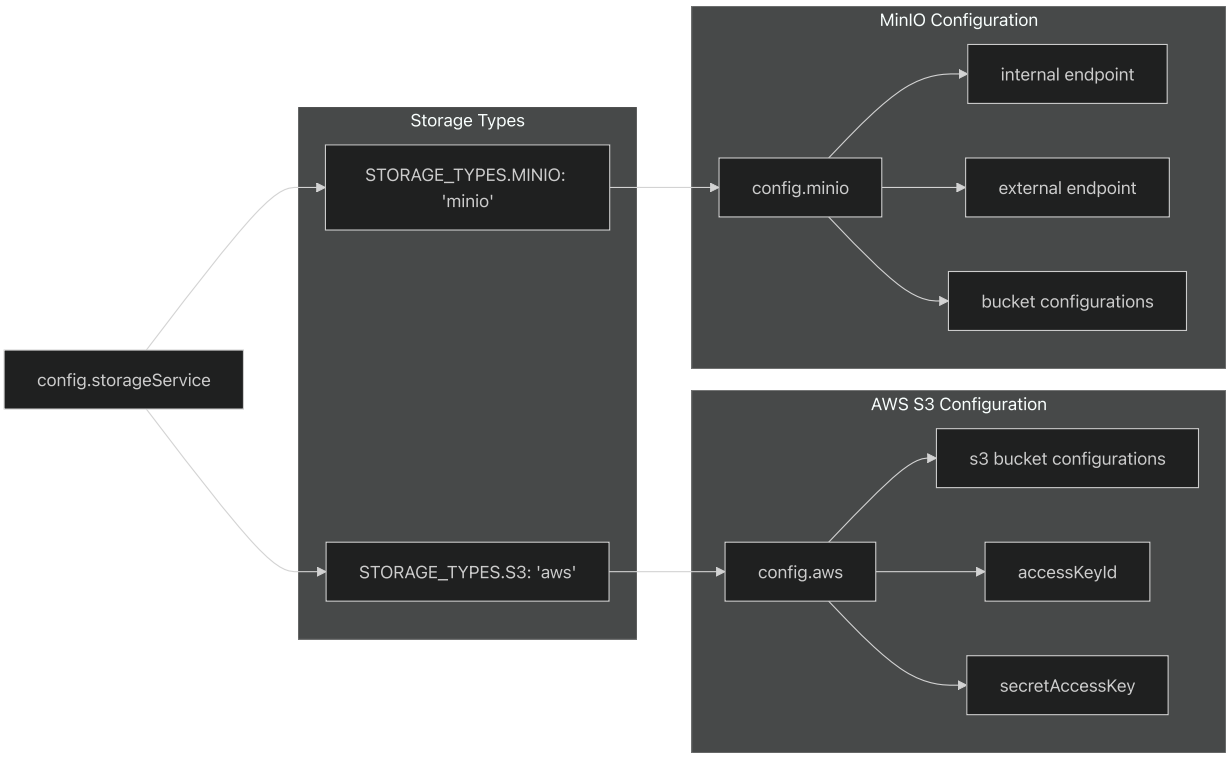
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L14-L23)

src/config.ts14-23 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L14-L23)

Storage Configuration

The service supports dual storage backends through the `STORAGE_TYPES` constant and related configuration sections.

Storage Type Selection



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L7-L10)

src/config.ts7-10 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L7-L10)[

src/config.ts52](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L52-L52)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L52-L52)%5B)

src/config.ts68-123](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L68-L123)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L68-L123)%5B)

src/config.ts124-223](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L124-L223 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L124-L223))

Bucket Configurations

Both AWS S3 and MinIO configurations include specialized bucket settings for different content types:

Bucket Purpose	Path	Content Type	Max Size
`announcement`	`announcement/`	`text/html`	2 MiB
`analysisReport`	`analysis_report/`	`application/pdf`	2 MiB
`langResource`	`lang_resource/`	`application/json`	2 MiB
`public`	`avatar/`	`image/`	2 MiB
`dbExport`	`json_file/`	`application/json/`	2 MiB

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L72-L121)

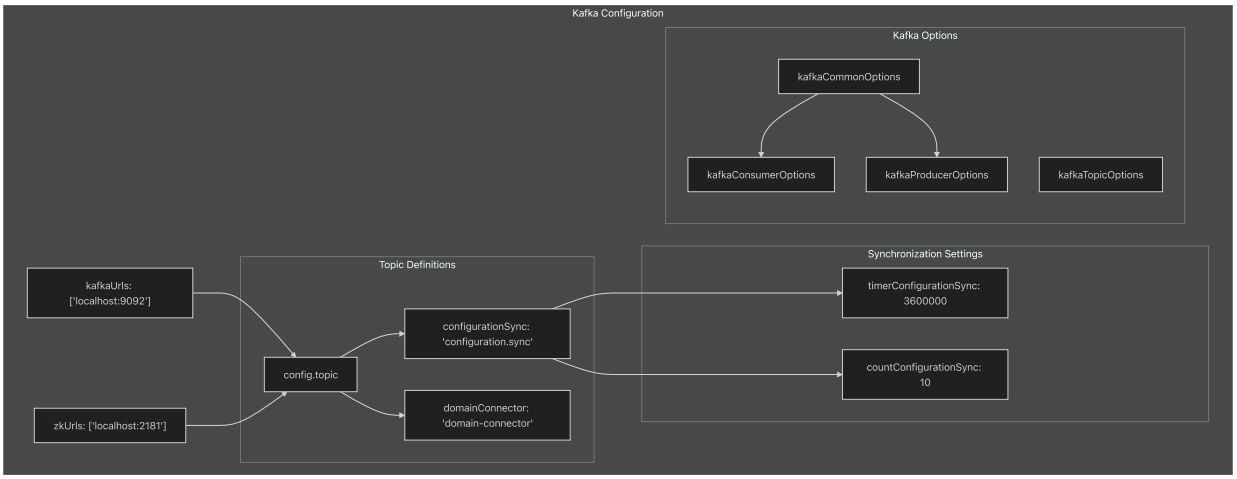
src/config.ts72-121 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L72-L121)[

src/config.ts155-199](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L155-L199 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L155-L199))

Messaging Configuration

The Kafka messaging configuration includes broker URLs, topic definitions, and consumer/producer options.

Kafka Topics and Synchronization



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L50-L51)

src/config.ts50-51 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L50-L51)[

src/config.ts53-58](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L53-L58)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L53-L58)%5B)

src/config.ts233-236](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L233-L236)[(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L233-L236)%5B)

src/config.ts300-307](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L300-L307 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L300-L307))

Logging Configuration

The logging system uses a dual-appender approach with both console and file output, configured through the `logger` property.

Logger Configuration Structure

Component	Setting	Value
Console Appender	`type`	`"console"`
File Appender	`type`	`"file"`
	`filename`	`"/logs/application.log"`
	`compression`	`true`
	`maxLogSize`	`10485760` (10 MB)
	`backups`	`10`
Default Category	`appenders`	`["application", "file"]`
	`level`	`"info"`

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L24-L40)

src/config.ts24-40 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L24-L40)

Service Logging Properties

The service-specific logging configuration includes:

- `serviceName`: `"configuration-service"`
- `format`: `"FLAT"` (supports `"FLAT"` or `"JSON"`)
- `transport`: Empty array for additional transports

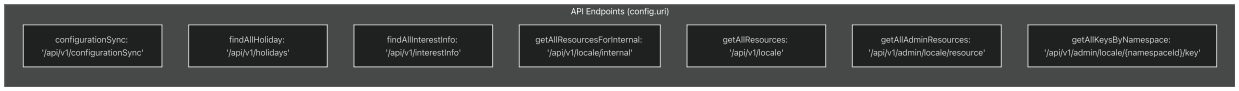
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L41-L45)

src/config.ts41-45 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L41-L45)

API and Service Configuration

API Endpoint Definitions

The `uri` configuration object defines standard API endpoints used throughout the service:



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L59-L67)

src/config.ts59-67 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L59-L67)

Swagger Documentation Configuration

The OpenAPI documentation is configured through the `swagger` property:

Property	Value	Description
`server`	`"localhost:3000"`	Default server URL
`version.v1`	`"/api/v1"`	API version 1 path
`version.v2`	`"/api/v2"`	API version 2 path
`header.openapi`	`"3.0.0"`	OpenAPI specification version
`header.info.title`	`"Rest API Specification"`	API documentation title

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L237-L266)

src/config.ts237-266 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L237-L266)

External Configuration Override

The configuration system supports runtime overrides through an external `env.js` file that can modify the base configuration object dynamically.

Override Mechanism

The override process uses Node.js VM module to execute JavaScript code in a controlled context:

1. **File Loading**: `fs.readFileSync("env.js", "utf8")` attempts to load the external configuration
2. **Script Execution**: `vm.Script` creates a new script context
3. **Context Injection**: The script runs with access to `conf`, `config`, `process`, and `console` objects
4. **Configuration Modification**: The external script can modify the configuration object directly

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L284-L298)

src/config.ts284-298 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L284-L298)

This mechanism allows for environment-specific configuration without requiring code changes or rebuilds, enabling different settings for development, staging, and production environments.

Environment Setup

Relevant source files

- [

  .nvmrc](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.nvmrc
  (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.nvmrc))

- [

  entrypoint.sh](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh
  (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh))

- [

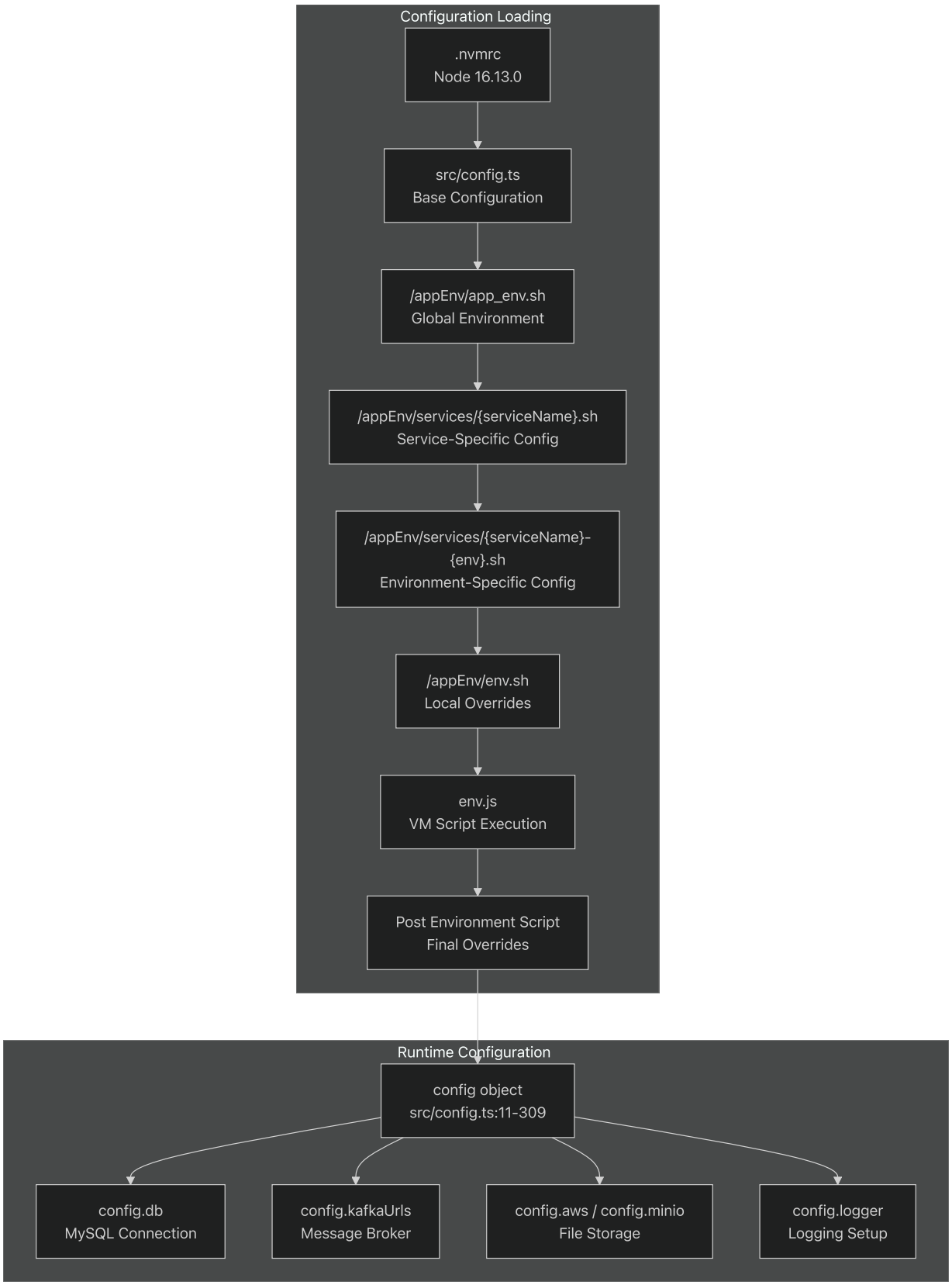
  src/config.ts](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts
  (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts))

This document covers the environment-specific configuration system, service orchestration process, and runtime parameter management for the TradeX Configuration Service. It explains how environment variables are loaded, how the service startup process works, and how configuration is managed across different deployment environments.

For broader configuration details including database and messaging setup, see Configuration. For Docker-specific deployment configuration, see Service Orchestration.

Configuration Sources and Loading Hierarchy

The TradeX Configuration Service employs a multi-layered configuration system that loads settings from various sources in a specific order of precedence.



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.nvmrc#L1-L1)

.nvmrc1 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.nvmrc#L1-L1)[

src/config.ts11-309](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L11-L309)[(https://github.com/nhsv-

tradex/configuration/blob/8c92cdcd/src/config.ts#L11-L309)%5B)

entrypoint.sh37-62](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L37-L62 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L37-L62))

Service Orchestration Process

The service startup is orchestrated through the `entrypoint.sh` script, which manages environment loading and service initialization.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L7-L90)

entrypoint.sh7-90 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L7-L90)

Runtime Configuration Management

The configuration system provides a centralized configuration object that can be dynamically modified through environment scripts.

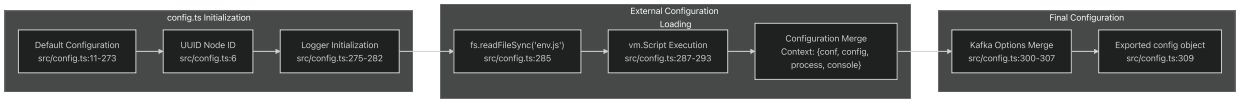
Core Configuration Structure

Configuration Area	Property Path	Description
Database	`config.db.connection`	MySQL connection parameters
Messaging	`config.kafkaUrls`, `config.zkUrls`	Kafka and Zookeeper endpoints
Storage	`config.aws.s3`, `config.minio`	AWS S3 and MinIO configuration
Logging	`config.logger.config`	Log4js configuration structure
Service Identity	`config.clusterId`, `config.clientId`	Service identification
API Configuration	`config.swagger`	OpenAPI/Swagger settings

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L11-L273)

src/config.ts11-273 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L11-L273)

Configuration Loading Process



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L284-L309)

src/config.ts284-309 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L284-L309)

Environment Variable System

The service uses a hierarchical environment variable loading system that allows for flexible configuration across different deployment environments.

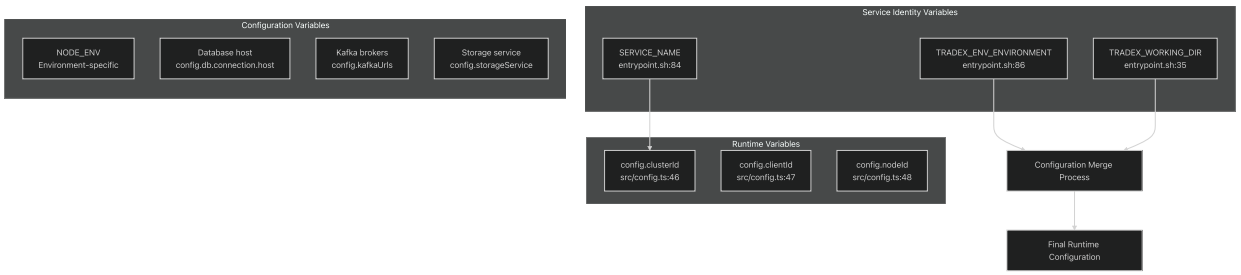
Environment Script Execution Order

Order	Script Path	Purpose	Execution Context
1	`/appEnv/app_env.sh`	Global environment variables	Always executed
2	`/appEnv/services/{serviceName}.sh`	Service-specific configuration	If file exists
3	`/appEnv/services/{serviceName}-{env}.sh`	Environment-specific overrides	If file exists
4	`/appEnv/env.sh`	Local development overrides	If file exists
5	`deployment-resources/{serviceName}-{env}/index.sh`	Deployment resources	If directory exists
6	`${postEnvScriptName}`	Post-environment script	If variable set

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L37-L82)

entrypoint.sh37-82 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L37-L82)

Key Environment Variables



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L84-L87)

entrypoint.sh84-87 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L84-L87)[

src/config.ts46-48](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L46-L48 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config.ts#L46-L48))

Dynamic Configuration Through env.js

The service supports dynamic configuration modification through a `env.js` file that is executed in a VM context, allowing safe runtime configuration changes.

VM Context Configuration

The `env.js` script is executed with access to:

- `conf` - Reference to the configuration object
- `config` - Alias to the configuration object
- `process` - Node.js process object
- `console` - Console logging functions

Configuration Override Example

The VM script execution allows for dynamic property modification:

```
// Example env.js content structure
conf.db.connection.host = process.env.DB_HOST
 |  | conf.db.connection.host; |
|--- | --- |
 | ``` |

Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/src/config

[src/config.ts285-298](https://github.com/nhsv-tradex/configuration/blob/8c92cdc

#### Node.js Version Management

The service uses Node Version Manager (nvm) to ensure consistent Node.js runtime

##### Version Specification

The `.nvmrc` file specifies the required Node.js version:

-   **Required Version**: `16.13.0`
-   **Purpose**: Ensures consistent runtime environment
-   **Usage**: `nvm use` reads this file automatically

Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.nvmrc#L1-

[.nvmrc1](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.nvmrc#L1-I

---

## Development

### Overview

Relevant source files

-   [

    .nvmrc](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.nvmrc)
-   [

    package-lock.json](https://github.com/nhsv-tradex/configuration/blob/8c92cdc
-   [

    package.json](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/pac

This document provides guidance for developers working on the TradeX Configurati

#### Prerequisites and Environment Setup
```

```
The service requires Node.js 16.13.0 as specified in the `.nvmrc` file. This ens

##### Node.js Version Management

![Chart 1](configuration-images/configuration-section17-1.svg)

**Development Environment Setup Flow**

Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.nvmrc#L1-

[.nvmrc1](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.nvmrc#L1-I

#### Dependencies and Package Management

The project uses npm for package management with dependencies categorized into r

##### Dependency Architecture

![Chart 2](configuration-images/configuration-section17-2.svg)

**Package Dependency Structure**

Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/package.js

[package.json19-35](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/p

package.json36-49](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/pa

#### Build Process and Scripts

The build system uses TypeScript compilation with ESLint integration for code qu

##### Build Pipeline

![Chart 3](configuration-images/configuration-section17-3.svg)

**TypeScript Build Pipeline**

The build process follows this sequence:

1.  **Clean**: Removes previous build artifacts using `rimraf`
2.  **Fix**: Automatically fixes ESLint violations
3.  **Compile**: TypeScript compilation to JavaScript
4.  **Output**: Generated files in `build/src/` directory

Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/package.js
```

```
[package.json6-8](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/pac

package.json12-15](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/pa

#### Development Workflow Scripts

The package defines several npm scripts for different development scenarios, fro

##### Script Execution Flow

![Chart 4](configuration-images/configuration-section17-4.svg)

**Development Script Dependencies**

Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/package.js

[package.json5-16](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/pa

#### Git Hooks and Code Quality Integration

The project uses Husky for Git hook management to enforce code quality before co

##### Pre-commit Workflow

![Chart 5](configuration-images/configuration-section17-5.svg)

**Git Hook Integration**

Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/package.js

[package.json50-54](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/p

#### Local Development Setup

For local development, the service provides specific scripts that handle enviror

##### Local Development Process

![Chart 6](configuration-images/configuration-section17-6.svg)

**Local Development Environment Setup**

The `run-local` script performs two key operations:

1.  Copies environment configuration from `src/env.js` to the build directory
2.  Changes to the build directory and starts the service with Node.js

Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/package.js
```

```
[package.json10-11](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/p

#### Binary Distribution

The service is configured as an executable package with the main entry point def

![Chart 7](configuration-images/configuration-section17-7.svg)

**Binary Configuration**

Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/package.js

[package.json55](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/pack

#### Development Dependencies Analysis

The development environment includes comprehensive tooling for TypeScript develo

 | Category | Package | Version | Purpose |
|--- | --- | --- | --- |
 | Type System | `@types/node` | 16.11.7 | Node.js type definitions |
 | TypeScript | `typescript` | 5.3.3 | Core TypeScript compiler |
 |  | `tslib` | 2.6.2 | TypeScript runtime library |
 |  | `tsutils` | 3.21.0 | TypeScript utilities |
 | Linting | `eslint` | 8.56.0 | Core ESLint engine |
 |  | `@typescript-eslint/eslint-plugin` | 6.19.0 | TypeScript-specific rules |
 |  | `@typescript-eslint/parser` | 6.19.0 | TypeScript parser for ESLint |
 | Formatting | `prettier` | 3.2.2 | Code formatting |
 |  | `eslint-config-prettier` | 9.1.0 | ESLint-Prettier integration |
 |  | `eslint-plugin-prettier` | 5.1.3 | Prettier as ESLint plugin |
 | Git Hooks | `husky` | 8.0.3 | Git hook management |
 | Build Tools | `rimraf` | 5.0.5 | Cross-platform rm -rf |

Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/package.js

[package.json36-49](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/p

### Code Quality and Standards

Relevant source files

-   [

    .eslintignore](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.e
-   [

    .eslintrc.js](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.es
```

```
-   [

    .gitignore](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.giti

This document outlines the code quality standards, linting configuration, and de

For information about environment setup and development workflow, see [Environme

#### Purpose and Scope

The TradeX Configuration Service maintains code quality through automated lintir

#### ESLint Configuration Architecture

The service uses a comprehensive ESLint setup that integrates TypeScript analysi

##### Core Configuration Structure

![Chart 1](configuration-images/configuration-section18-1.svg)

**Sources:**[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.eslir

[.eslintrc.js1-44](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.e

The ESLint configuration extends three base configurations: `plugin:prettier/rec

##### Environment and Parser Configuration

The service supports multiple JavaScript environments as defined in[](https://gi

[.eslintrc.js2-6](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.es

 | Environment | Purpose |
|--- | --- |
 | `browser` | Client-side JavaScript compatibility |
 | `node` | Node.js runtime environment |
 | `es6` | ES6 module and syntax support |
 | `es2021` | Latest ECMAScript features |

Parser options in[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.

[.eslintrc.js28-32](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.

##### Active Linting Rules

![Chart 2](configuration-images/configuration-section18-2.svg)

**Sources:**[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.eslir
```

```
[.eslintrc.js33-43](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.

The configuration disables the standard `no-unused-vars` rule in favor of the Ty

[.eslintrc.js34-35](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.

.eslintrc.js36-42](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.e

#### TypeScript Integration

##### Parser and Plugin Configuration

The service uses `@typescript-eslint/parser` as defined in[](https://github.com/

[.eslintrc.js13](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.esl

.eslintrc.js14](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.esli

The parser configuration at[](https://github.com/nhsv-tradex/configuration/blob/

[.eslintrc.js31](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.esl

##### TypeScript-Specific Rules

![Chart 3](configuration-images/configuration-section18-3.svg)

**Sources:**[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.eslir

[.eslintrc.js34-35](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.

The configuration replaces JavaScript's `no-unused-vars` with the TypeScript equ

#### Prettier Integration

##### Configuration Relationship

![Chart 4](configuration-images/configuration-section18-4.svg)

**Sources:**[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.eslir

[.eslintrc.js9-10](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.e

.gitignore5](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.gitignc

The service integrates Prettier through two ESLint extensions: `plugin:prettier/

[.gitignore5](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.gitigr
```

```
#### File Exclusions and Ignore Patterns

##### ESLint Exclusions

The ESLint ignore configuration in[](https://github.com/nhsv-tradex/configuratic

[.eslintignore1-3](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.e

 | Directory | Purpose |
|--- | --- |
 | `node_modules` | Third-party dependencies |
 | `dist` | Build output directory |
 | `build` | Alternative build directory |

##### Git Exclusions

![Chart 5](configuration-images/configuration-section18-5.svg)

**Sources:**[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.gitig

[.gitignore1-6](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.giti

Git exclusions in[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.

[.gitignore1-6](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.giti

#### Development Workflow Integration

##### Quality Check Pipeline

![Chart 6](configuration-images/configuration-section18-6.svg)

**Sources:**[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.eslir

[.eslintrc.js1-44](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.e

.eslintignore1-3](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/.es

The quality assurance workflow ensures all code changes meet established standar

---

## Deployment

### Overview

Relevant source files
```

```
-   [
    dockerfile](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/docke
-   [
    entrypoint.sh](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/er
This document covers the deployment architecture and processes for the TradeX Co
#### Deployment Architecture
The TradeX Configuration Service uses a containerized deployment model with soph
##### Build and Container Strategy
![Chart 1](configuration-images/configuration-section19-1.svg)
Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile
[dockerfile1-57](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dock
##### Environment Configuration Flow
The service implements a hierarchical environment configuration system that loac
![Chart 2](configuration-images/configuration-section19-2.svg)
Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint
[entrypoint.sh1-90](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/e
#### Build Process
The deployment uses a multi-stage Docker build optimized for production efficier
 | Stage | Purpose | Key Operations |
|--- | --- | --- |
 | `base` | Foundation | Install system dependencies, copy package files, instal
 | `builder` | Compilation | Copy source code, install dev dependencies, run `np
 | `production` | Runtime | Copy built application and production node\_modules
The build process installs essential system dependencies required for native Noc
```

bash, git, python3, make, g++

```
Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile

[dockerfile2-33](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dock

#### Environment Configuration System

The `entrypoint.sh` script implements a sophisticated environment loading mechar

##### Configuration Hierarchy

1.  **Global Environment**: `/appEnv/app_env.sh` - Platform-wide settings
2.  **Service Configuration**: `/appEnv/services/$serviceName.sh` - Service-spec
3.  **Environment-Specific**: `/appEnv/services/$serviceName-$env.sh` - Environm
4.  **Local Overrides**: `/appEnv/env.sh` - Local customizations
5.  **Deployment Resources**: `/appEnv/services/deployment-resources/$serviceNam

##### Environment Variables

 | Variable | Source | Purpose |
|--- | --- | --- |
 | `TRADEX_WORKING_DIR` | entrypoint.sh:35 | Application working directory |
 | `SERVICE_NAME` | entrypoint.sh:84 | Service identifier |
 | `TRADEX_ENV_ENVIRONMENT` | entrypoint.sh:86 | Environment designation |

Sources:[](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint

[entrypoint.sh4-90](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/e

#### Health Monitoring

The container includes built-in health monitoring through Docker's `HEALTHCHECK`
```

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD node -e "require('http').get('http://localhost:3000/currentTime (http://localhost:3000/currentTime)', (res) => { process.exit(res.statusCode === 200 ? 0 : 1) })"

	exit 1

Health Check Configuration

Parameter	Value	Purpose
`interval`	30s	Check frequency
`timeout`	3s	Request timeout
`start-period`	5s	Grace period during startup
`retries`	3	Failure threshold
`endpoint`	`/currentTime`	Health check endpoint

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L53-L54)

dockerfile53-54 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L53-L54)

Production Deployment

Container Specifications

- **Base Image**: `node:16-alpine3.11` for minimal footprint and security
- **Exposed Port**: `3000` for HTTP traffic
- **Entry Point**: `/bin/bash /app/entrypoint.sh node build/src/index.js`
- **Working Directory**: `/app`

Security Considerations

The container runs with minimal privileges and includes only essential dependencies. The Alpine Linux base provides a small attack surface while the multi-stage build ensures no development dependencies reach production.

Scalability

The stateless design allows horizontal scaling through container orchestration platforms. Environment configuration is externalized through the hierarchical script system, enabling different configurations per deployment without rebuilding images.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L35-L57)

dockerfile35-57 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L35-L57)[

entrypoint.sh84-90](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L84-L90 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L84-L90))

Docker Configuration

Relevant source files

- [

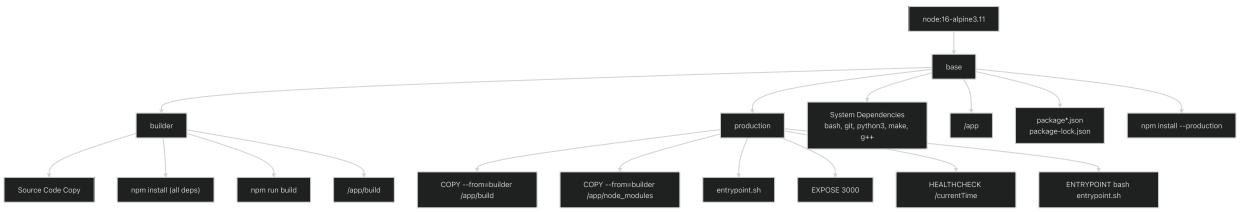
  dockerfile](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile))

This document covers the Docker containerization strategy for the TradeX Configuration Service, including the multi-stage build process, container optimization, and health monitoring configuration. For service startup and orchestration details, see Service Orchestration. For environment-specific configuration, see Environment Setup.

Multi-Stage Build Architecture

The TradeX Configuration Service uses a three-stage Docker build to optimize both build time and final image size. The build leverages Alpine Linux for minimal footprint while supporting native Node.js modules.

Build Stage Flow



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L1-L57)

dockerfile1-57 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L1-L57)

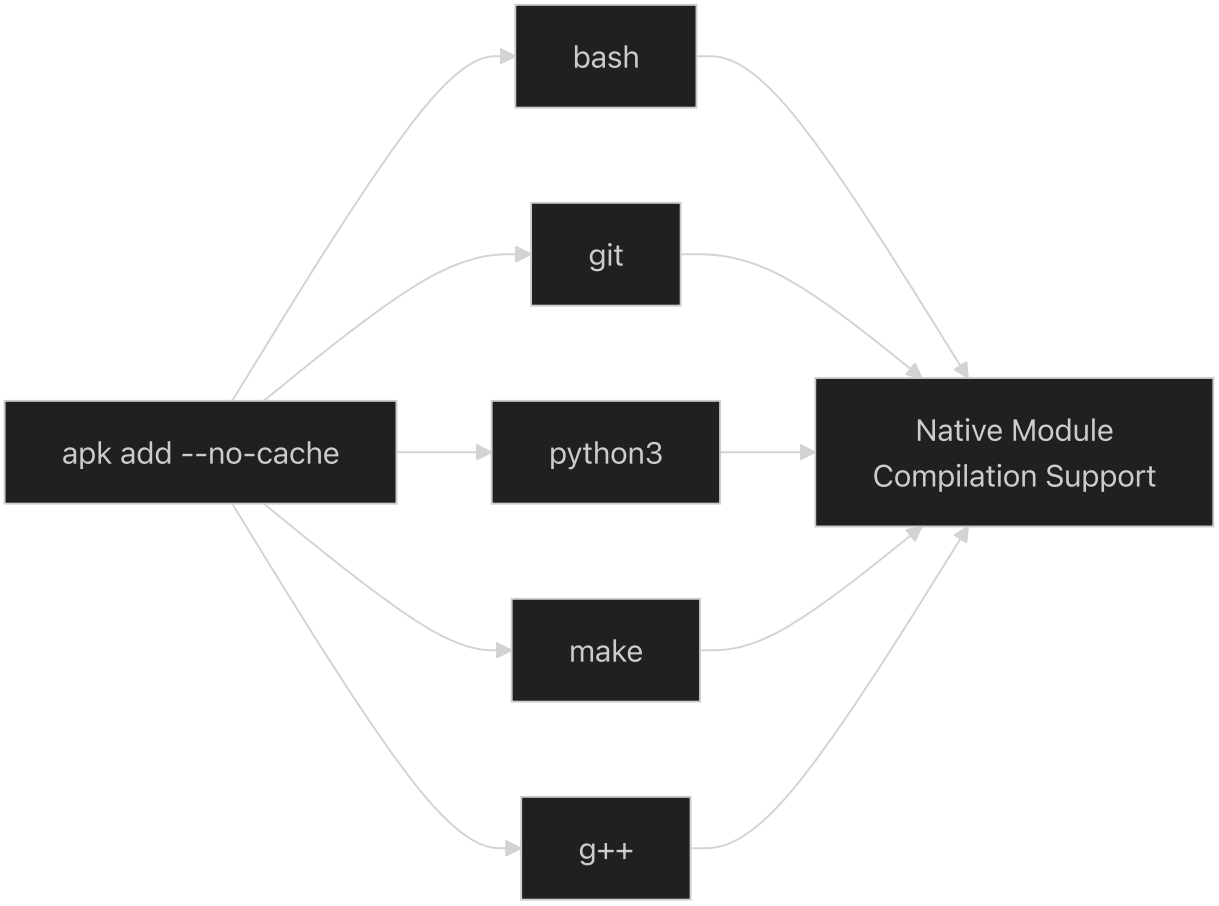
Build Stages

Base Stage Configuration

The base stage establishes the foundation for both builder and production stages, installing necessary system dependencies for native Node.js modules.

Component	Configuration	Purpose
Base Image	`node:16-alpine3.11`	Lightweight Node.js 16 runtime
System Packages	`bash`, `git`, `python3`, `make`, `g++`	Native module compilation support
Working Directory	`/app`	Application root directory
Package Installation	`npm install --frozen-lockfile --production=true`	Production dependencies only

The base stage installs system dependencies required for native module compilation:

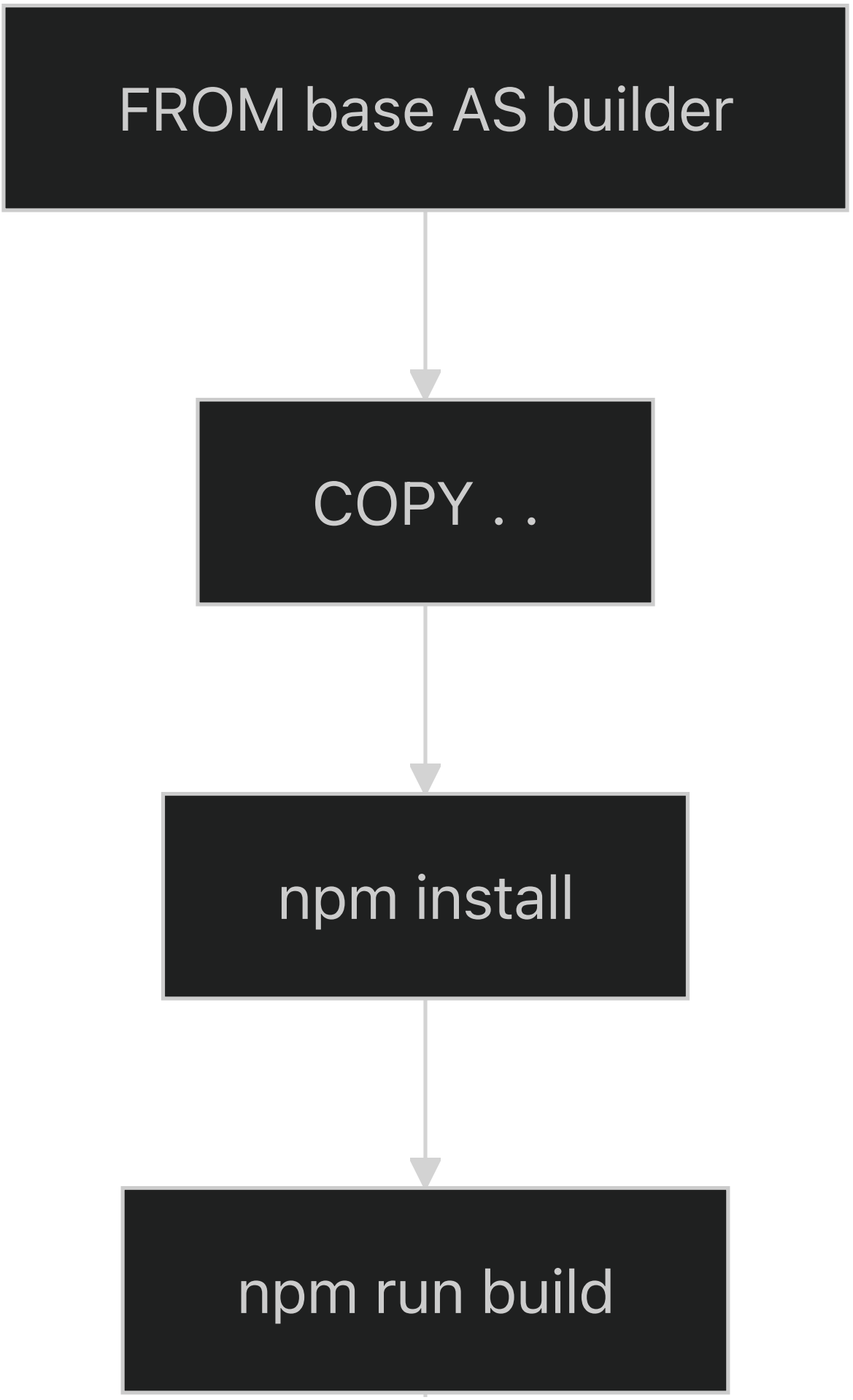


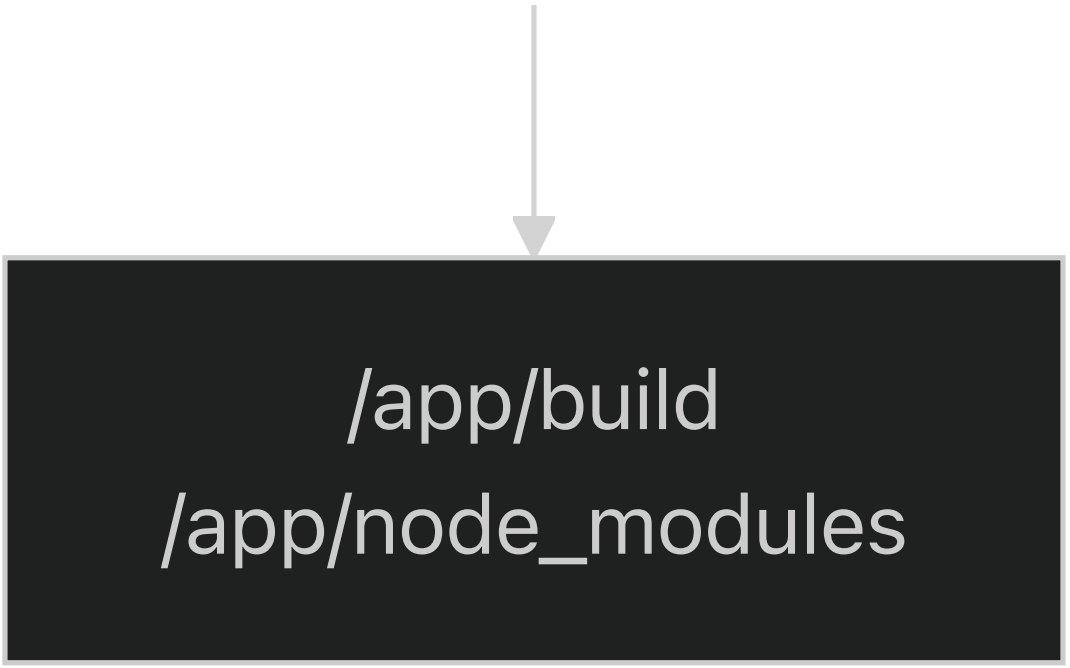
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L2-L21)

dockerfile2-21 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L2-L21)

Builder Stage Configuration

The builder stage handles the complete application build process, including development dependencies and TypeScript compilation.


FROM base AS builder
COPY . .
npm install
npm run build



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L24-L32)

dockerfile24-32 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L24-L32)

Production Stage Configuration

The production stage creates the final optimized container by copying only necessary artifacts from the builder stage.

Artifact	Source	Destination	Purpose
Built Application	`builder:/app/build`	`/app/build`	Compiled JavaScript
Node Modules	`builder:/app/node_modules`	`/app/node_modules`	Runtime dependencies
Entry Script	Host	`/app/entrypoint.sh`	Service orchestration

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L35-L48)

dockerfile35-48 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L35-L48)

Container Runtime Configuration

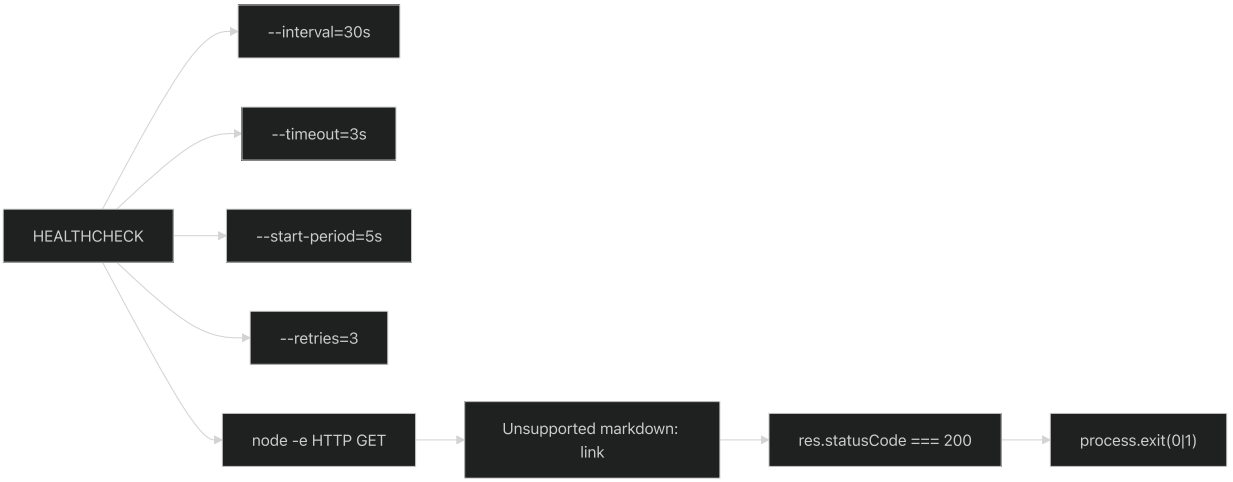
Port Exposure

The service exposes port `3000` for HTTP traffic, configured in (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L50-L50)

dockerfile50 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L50-L50) This port serves all API endpoints documented in API Reference.

Health Check Implementation

The container implements a custom health check using the `/currentTime` endpoint:



The health check parameters:

- **Interval**: 30 seconds between checks
- **Timeout**: 3 seconds maximum response time
- **Start Period**: 5 seconds grace period during startup
- **Retries**: 3 consecutive failures before marking unhealthy

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L53-L54)

dockerfile53-54 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L53-L54)

Entry Point Configuration

The container uses a custom entry point that combines shell orchestration with Node.js execution:

```
ENTRYPOINT ["/bin/bash", "/app/entrypoint.sh", "node build/src/index.js"]
```

The entry point structure delegates to `entrypoint.sh` for environment setup and service orchestration before executing the main application. For detailed orchestration logic, see Service Orchestration.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L56-L56)

dockerfile56 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L56-L56)

Container Optimization Features

Image Size Optimization

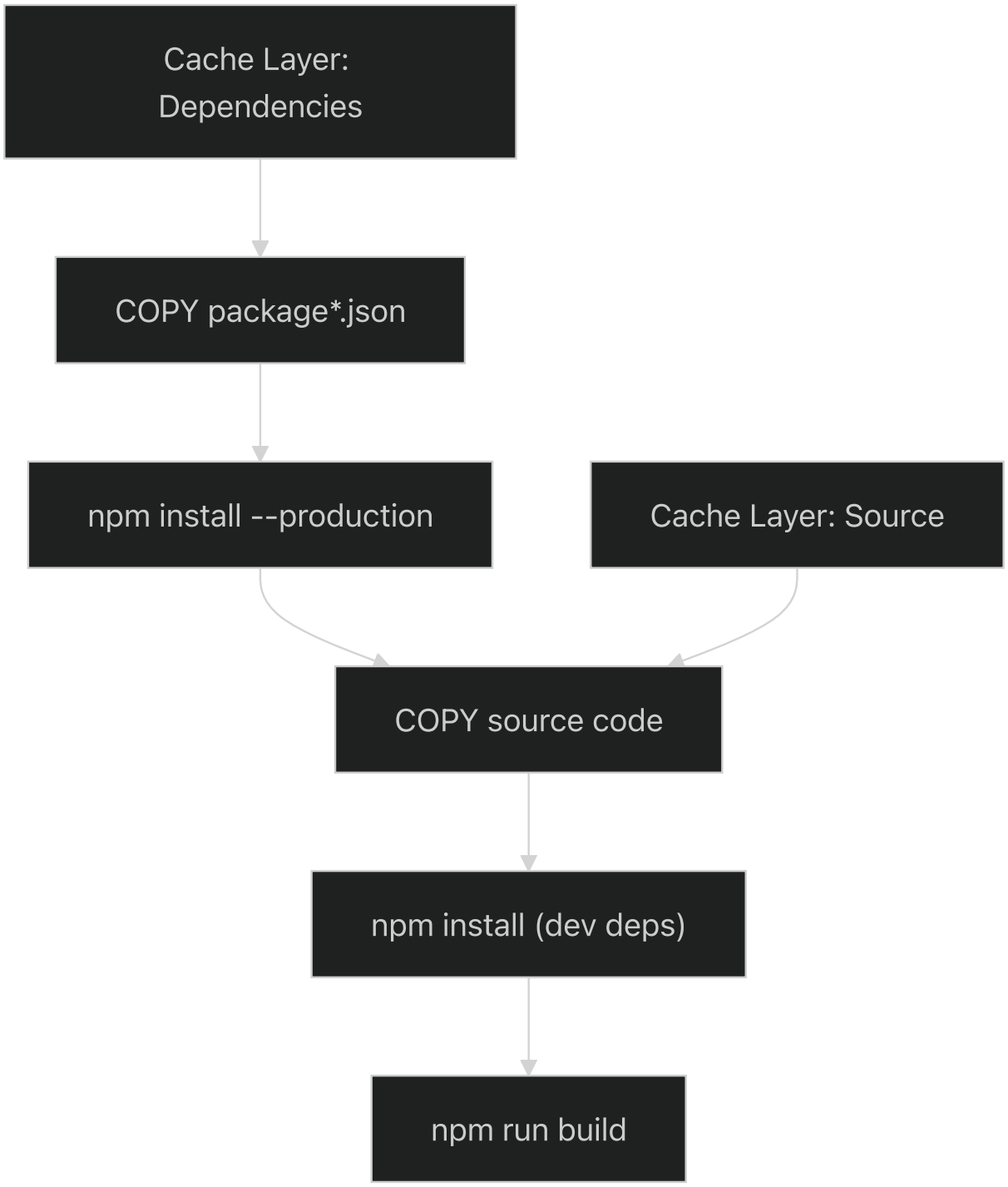
The multi-stage build reduces final image size by:

- Installing only production dependencies in the final stage
- Excluding development tools and source code
- Using Alpine Linux for minimal base image size
- Cleaning package manager cache with `rm -rf /var/cache/apk/*`

Build Cache Optimization

The Dockerfile optimizes build cache efficiency by:

- Copying `package*.json` files before source code
- Installing dependencies before copying application source
- Separating production and development dependency installation



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L17-L21)

dockerfile17-21 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L17-L21)[

dockerfile27-32](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L27-L32
(https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/dockerfile#L27-L32))

Service Orchestration

Relevant source files

- [

  entrypoint.sh](https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh))

This document covers the service startup orchestration system implemented in the TradeX Configuration Service. It details how the `entrypoint.sh` script coordinates environment loading, configuration management, and service initialization during deployment and runtime startup.

For information about Docker container configuration, see Docker Configuration. For environment-specific configuration details, see Environment Setup.

Overview

The service orchestration system is implemented through a single bash script that manages the complete startup lifecycle. The `entrypoint.sh` script serves as the primary entry point for the containerized service, handling environment variable loading, service-specific configuration, and command execution in a structured sequence.

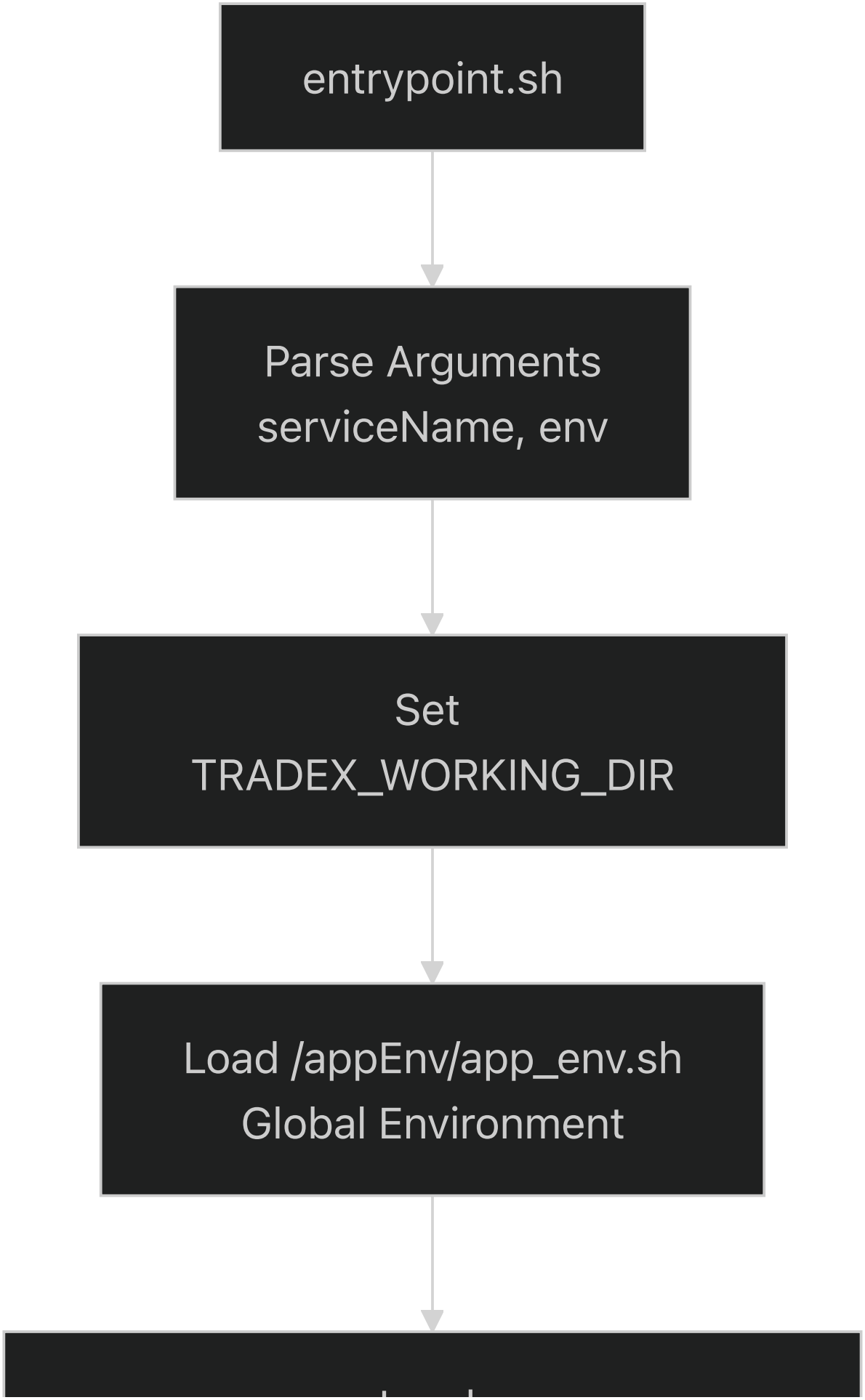
The orchestration system supports multiple deployment environments and service configurations through a hierarchical environment loading strategy that progresses from global settings to service-specific and environment-specific overrides.

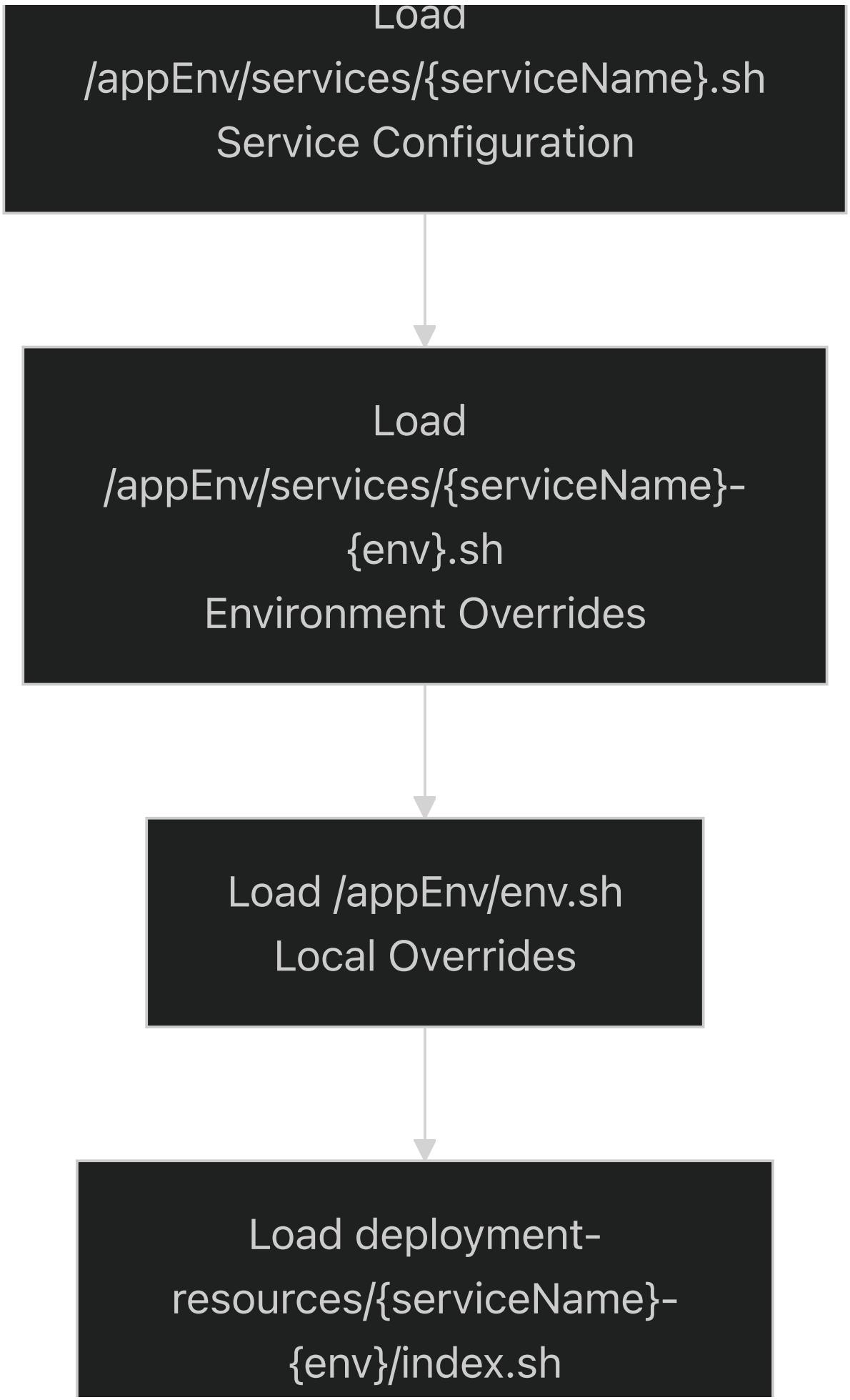
Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L1-L90)

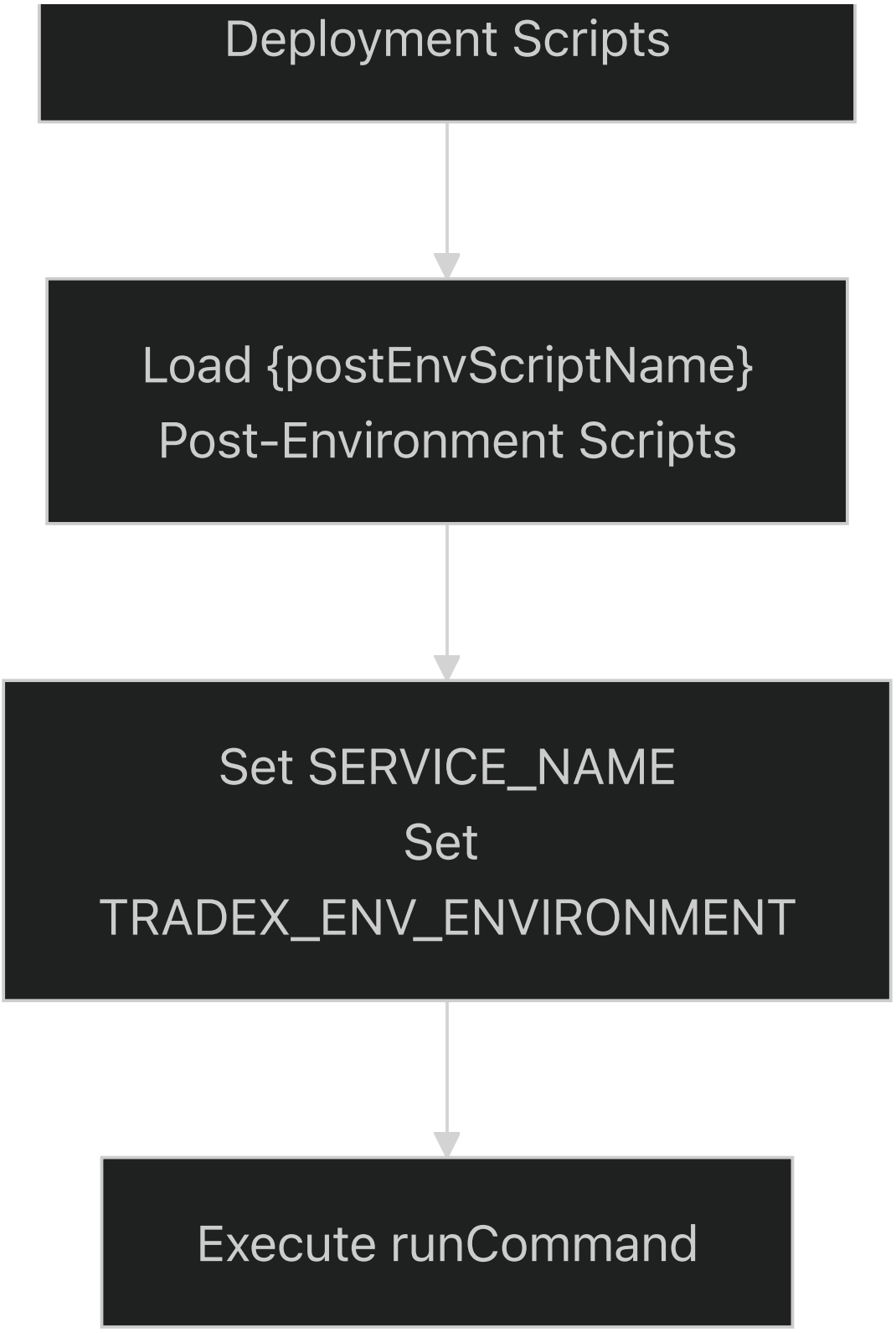
entrypoint.sh1-90 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L1-L90)

Environment Loading Hierarchy

The orchestration system loads configuration through a structured hierarchy of environment files, each serving a specific purpose in the configuration cascade.


entrypoint.sh
Parse Arguments
serviceName, env
Set
TRADEX_WORKING_DIR
Load /appEnv/app_env.sh
Global Environment


Load /appEnv/services/{serviceName}.sh Service Configuration
Load /appEnv/services/{serviceName}-{env}.sh Environment Overrides
Load /appEnv/env.sh Local Overrides
Load deployment-resources/{serviceName}-{env}/index.sh



Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L37-L90)

entrypoint.sh37-90 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L37-L90)

Argument Processing and Parameter Management

The script processes command-line arguments using a standardized parameter parsing approach that supports both short and long-form options.

Parameter	Short Form	Long Form	Purpose
Service Name	`-s`	`--serviceName`	Identifies the specific service configuration to load
Environment	`-e`	`--env`	Specifies the deployment environment (dev, staging, prod)
Command	`PARAMS`	(positional)	The actual command to execute after environment setup

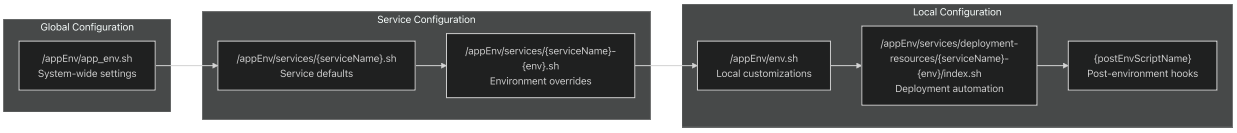
The argument parsing logic preserves positional arguments while extracting named parameters, allowing flexible command specification during container startup.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L7-L30)

entrypoint.sh7-30 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L7-L30)

Configuration File Loading Sequence

The orchestration system loads configuration files in a specific sequence designed to support configuration inheritance and environment-specific overrides.



Configuration Loading Details

1. **Global Environment** (`app_env.sh`): System-wide configuration applied to all services
2. **Service Configuration** (`{serviceName}.sh`): Service-specific defaults and settings
3. **Environment Overrides** (`{serviceName}-{env}.sh`): Environment-specific parameter overrides
4. **Local Environment** (`env.sh`): Local development or deployment-specific customizations
5. **Deployment Resources**: Automated deployment scripts and resource setup
6. **Post-Environment Scripts**: Final configuration hooks executed after environment loading

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L37-L82)

entrypoint.sh37-82 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L37-L82)

Working Directory and Environment Variables

The orchestration system establishes a consistent runtime environment by setting key environment variables and managing the working directory context.

Core Environment Variables

Variable	Source	Purpose
`TRADEX_WORKING_DIR`	Set from `$(pwd)`	Establishes the service's base working directory
`SERVICE_NAME`	Command-line parameter	Identifies the current service for configuration lookup
`TRADEX_ENV_ENVIRONMENT`	Command-line parameter or existing env	Specifies the deployment environment

Directory Management

The script maintains directory context throughout the configuration loading process:

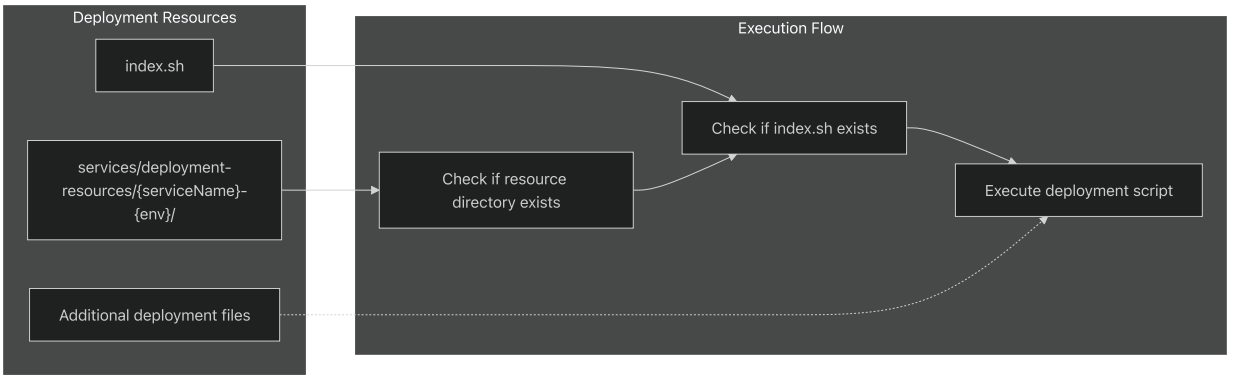
1. Captures initial working directory as `currentDir`
2. Sets `TRADEX_WORKING_DIR` environment variable
3. Navigates to `/appEnv` for configuration loading
4. Returns to original directory before command execution

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L34-L87)

entrypoint.sh34-87 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L34-L87)

Deployment Resource Integration

The orchestration system includes support for deployment-specific resource management through dedicated deployment resource directories.



The deployment resource execution follows this pattern:

1. Check for directory existence at `services/deployment-resources/{serviceName}-{env}`
2. If directory exists, look for `index.sh` script
3. Execute the deployment script with appropriate permissions

4. Return to the base directory for final command execution

Note: There is a typo in the original script at line 68 where `$resourceDir` should be `$resouceDir` to match the variable declaration.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L65-L74)

entrypoint.sh65-74 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L65-L74)

Error Handling and Execution Control

The orchestration script implements several error handling and execution control mechanisms to ensure reliable service startup.

Error Handling Features

- **Strict Error Mode**: `set -e` ensures the script exits on any command failure
- **Unsupported Flag Detection**: Invalid command-line arguments cause immediate script termination
- **Permission Management**: Automatic `chmod +x` execution for all script files
- **Conditional Execution**: File existence checks before attempting to execute configuration scripts

Script Execution Pattern

Each configuration script follows a consistent execution pattern:

1. Check for file existence using `[ -f "filename" ]`
2. Grant execute permissions with `chmod -f +x`
3. Source the script using `. ./filename`
4. Log execution with descriptive echo statements

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L21-L82)

entrypoint.sh21-82 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L21-L82)

Service Startup Command Execution

The final phase of the orchestration process involves executing the actual service startup command with all environment configuration applied.

The command execution follows this sequence:

1. Capture positional arguments as `PARAMS` during argument parsing
2. Store as `runCommand` variable for later execution

3. Execute using `eval $runCommand` after all environment setup is complete

This approach allows the orchestration script to handle complex command structures while maintaining environment variable scope and working directory context.

The execution is logged with timestamp and command details for monitoring and debugging purposes.

Sources: (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L26-L90)

entrypoint.sh26-90 (https://github.com/nhsv-tradex/configuration/blob/8c92cdcd/entrypoint.sh#L26-L90)
